

INFORMATIKA FAKULTATEA
FACULTAD DE INFORMÁTICA

Gradu Amaierako Lana

Informatika Ingeniaritzako Gradua

Software Ingeniaritza

Programazio-ikaskuntzarako web-plataforma: feedback pedagogikoa LLM eta MCP bidez

Josu Aguinaga Bengoetxea

Zuzendariak
Itziar Aldabe Arregi

2026ko ekainaren 21a

Laburpena

Gaur egun, Hizkuntza-eredu Handien (LLM) garapenak eta hedapenak aukera berriak zabaldu ditu hezkuntzan. Gradu Amaierako Lana (GrAL) honen helburua eredu hauen abantailak programazio ikasleen pedagogian integratzea da, pentsamendu kritikoa sustatuz. Horretarako, web-sistema oso bat garatu da, non ikasleek programazio-ariketak egin, egiaztatu eta kontsulatu ditzaketen.

Sistemaren berrikuntza nagusia Model Context Protocol (MCP) protokoloaren erabilera da. MCPri esker, LLMak ikasleari feedback pedagogikoa, erantzun pertsonalizatuak eta ikaslearen ikasketa-mailara egokitutako pistak emateko aukera eskuratzen da, modu eraginkor eta seguru batean.

Garapen Jasangarrirako Helburuak

Garapen Jasangarrirako Helburuak (GJH) Nazio Batuen Erakundeak (NBE) 2015ean ezarritako 17 helburu biltzen ditu eta haien helburu nagusia “peace and prosperity for people and the planet (bakea eta oparotasuna jendea eta planetarentzat)” [1] da, ikusi 1 Irudia.



1 Irudia: GJH 2030.

Proiektu honek hainbat GJH barneratzen ditu:

4. GJH — Kalitatezko Hezkuntza
: Proiektuak helburu nagusi gisa ikasleen —edo nahi duen edozein pertsonen— ikaskuntza hobetzea du: pentsamendu kritikoa bultzatuz eta denbora errealean berrelikadura pedagogikoa eskainiz.

9. GJH — Industria, Berrikuntza eta Azpiegitura
: Proiektuak hainbat berrikuntza teknologiko erabiltzen ditu ikasleen prestakuntza akademikoa aberatsa eskaintzeko. Erabiliko diren teknologia berri nagusiak Adimen Artifiziala (AA), LLM eta MCP dira: azken hamarkadan AA eta, batez ere, LLMak, asko hobetu dira eta MCPrekin konbinatuz pedagogian oso erabilgarria izan daitezke.

Zehazki, konpromiso hauekin erlazionatuta dago proiektua:

- **Konpromisoa 25. Erronka berriei heltzeko curriculuma** (4. GJH)

- **Konpromisoa 35. Enpresa + unibertsitatea harremana sendotzea eta Lehen Hezkuntza (LH) + Unibertsitatea prestakuntza sustatzea.** (4. GJH eta 9. GJH)

Gaien aurkibidea

Gaien aurkibidea v

Irudien aurkibidea ix

Taulen aurkibidea xi

Kodeen aurkibidea xii

1 Sarrera 1
- 1.1 Testuingurua eta arazoaren deskribapena . . . 1
- 1.2 Proposatutako irtenbidea: sistemaren deskribapena . . . 1
- 1.3 Helburu orokorra . . . 2
- 1.4 Memoriaren antolaketa . . . 2

2 Aurrekariak 3
- 2.1 Motibazioa . . . 3
- 2.2 Adimen artifiziala eta hizkuntza-eredu handiak . . . 4
  - 2.2.1 Tokenizazioa eta sarreraren prozesamendua . . . 4
  - 2.2.2 Transformer arkitektura eta arreta mekanismoa . . . 4
  - 2.2.3 Entrenamendu faseak . . . 5
- 2.3 Zer da MCP? . . . 5
  - 2.3.1 Arkitektura eta datu-geruza . . . 5
  - 2.3.2 Garraio-geruza . . . 6
  - 2.3.3 MCP elementu nagusiak: prompts, resources eta tools . . . 7
- 2.4 Artearen egoera . . . 8
  - 2.4.1 Ikaskuntza plataforma orokorrak . . . 8
  - 2.4.2 Ataza zehatzetarako tresnak . . . 9

3 Planifikazioa 11
- 3.1 Irismena . . . 11
  - 3.1.1 Helburu zehatzak . . . 11
  - 3.1.2 Eskakizunak . . . 11
  - 3.1.3 Baztertutakoa . . . 12
  - 3.1.4 Emangarriak . . . 13
  - 3.1.5 Lanaren Deskonposizio Egitura (LDE) . . . 14
  - 3.1.6 Suposamenduak . . . 15
- 3.2 Atazak egiteko denboraldiak eta emangarriak bukatzeko datak . . . 16
  - 3.2.1 Egin beharreko atazen deskribapena . . . 16

3.2.2 Atazen garapen denboraldiak eta menpekotasunak . . . 19
3.2.3 Proiektuaren garapen mugarriak . . . 19
3.3 Ataza bakoitzari emango zaion denboraren estimazioak . . . 19
3.4 Informazio sistema eta komunikazio sistema . . . 20
3.4.1 Informazio sistema . . . 20
3.4.2 Komunikazio sistema . . . 22
3.5 Arriskuen analisia . . . 23

4 Sistemaren betekizunen katalogoa 25
4.1 Informazio betekizunak . . . 25
4.2 Betekizun funtzionalak . . . 26
4.2.1 Aktoreen definizioa . . . 26
4.2.2 Erabilera-kasuen diagramak . . . 26
4.2.3 Sistemaren erabilera kasuak . . . 28
4.2.4 Sistemaren sekuentzia-diagramak . . . 43
4.3 Betekizun ez-funtzionalak . . . 43
4.4 Trazabilitate matrizea . . . 45

5 Arkitektura eta diseinua 47
5.1 Aplikazioaren arkitektura orokorra . . . 47
5.2 Arkitekturaren iterazioak . . . 48
5.2.1 Lehenengo iterazioa: oinarrizko arkitektura . . . 48
5.2.2 Bigarren iterazioa: eskalagarritasuna . . . 49
5.3 Datu-basearen diseinua . . . 49
5.3.1 Gorde beharreko ariketen informazioa . . . 51
5.4 Backendaren endpointak . . . 51
5.5 MCP komunikazioa . . . 51
5.6 Kode exekuzio MCP zerbitzariaren tresnak eta funtzionamendu adibidea . 52
5.6.1 Eskema komuna eta funtzionamendu-fluxua . . . 52
5.6.2 Adibide praktikoa: Python ariketa baten ebaluazioa . . . 53
5.7 Feedback pedagogikoaren diseinua eta LLMaren testuingurua . . . 56
5.7.1 Eskaera eskema . . . 56
5.7.2 Feedbackaren sorkuntza-fluxua . . . 56

6 Inplementazioa 59
6.1 Datu-basearen inplementazioa . . . 59
6.1.1 Mantengarritasuna . . . 59
6.1.2 Hasierako datuen karga . . . 60
6.2 Backendaren inplementazioa . . . 60
6.2.1 Entitateen arteko komunikazioa . . . 61
6.2.2 Segurtasuna eta middleware konfigurazioa . . . 61
6.2.3 Kautotzea . . . 62
6.2.4 Geruza banaketa: CSR patroia . . . 63
6.3 Kode exekuzio MCP zerbitzariaren inplementazioa . . . 63
6.3.1 Docker irudien sorrera . . . 63
6.3.2 Kodearen exekuzioa . . . 63
6.3.3 Segurtasun kontrola . . . 64
6.4 Frontendaren inplementazioa . . . 66

6.4.1 Kautotze kudeaketa . . . 68
6.4.2 Webgunearen orrialdeak . . . 68

7 Jarraipen eta kontrola **73**
7.1 Irismenaren kudeaketa . . . 73
7.2 Garapen denboren kudeaketa . . . 73
7.3 Arriskuen kudeaketa . . . 75

8 Ondorioak **77**
8.1 Emaitzak eta ondorioak . . . 77
8.2 Ikasitakoa . . . 78
8.3 Hobekuntzak eta hurrengo urratsak . . . 78

A Sekuentzia-diagramak **81**
A.1 BF-1: Erregistratu . . . 81
A.2 BF-2: Saioa hasi . . . 81
A.3 BF-3: Saioa itxi . . . 81
A.4 BF-5: Profila ikusi . . . 81
A.5 BF-6: Datu pertsonalak editatu . . . 81
A.6 BF-7: Pasahitza aldatu . . . 81
A.7 BF-8: Ikasketa maila aldatu . . . 82
A.8 BF-9: Kontua ezabatu . . . 82
A.9 BF-10: Saiakerak ezabatu . . . 82
A.10 BF-11: Mezuak ezabatu . . . 82
A.11 BF-12: Aurrerapena begiratu . . . 82
A.12 BF-13: Ariketak zerrendatu . . . 82
A.13 BF-14: Ariketak bilatu . . . 82
A.14 BF-15: Ariketa aukeratu . . . 82
A.15 BF-16: Ariketa lengoaia aldatu . . . 82
A.16 BF-17: Saiakerak ikusi . . . 83
A.17 BF-18: Saiakera bidali . . . 83
A.18 BF-19: Saiakera ireki . . . 83
A.19 BF-20: Laguntza eskatu . . . 83

B Backendaren endpointak **103**

C Erabilitako tresnak eta teknologiak **109**
C.1 Programazio-lengoaiak, exekuzio-tresnak eta datu-baseak . . . 109
C.2 Liburutegiak . . . 110
C.3 Garapen-tresnak . . . 112
C.4 Kudeaketa-tresnak . . . 112

D AA sortzailearen erabilera deklarazioa **115**

E Bileren aktak **117**
E.1 Bilera akta (2026-02-23) . . . 117
E.2 Bilera akta (2026-02-27) . . . 118
E.3 Bilera akta (2026-03-06) . . . 118
E.4 Bilera akta (2026-03-13) . . . 118

E.5 Bilera akta (2026-03-27) 119
E.6 Bilera akta (2026-04-10) 119
E.7 Bilera akta (2026-04-17) 120
E.8 Bilera akta (2026-04-24) 121
E.9 Bilera akta (2026-04-30) 121
E.10 Bilera akta (2026-05-13) 122

Akronimoa **123**

Bibliografia **125**

Irudien aurkibidea

1 GJH 2030. iii

2.1 MCPko osagarrien arkitektura. 6

3.1 Proiektuaren LDE diagrama. 14
3.2 Proiektuaren Gantt diagrama (1/2). 20
3.3 Proiektuaren Gantt diagrama (2/2). 21

4.1 Aplikazioaren erabilera-kasuen diagrama. 27

5.1 Aplikazioaren arkitektura orokorra. 48
5.2 Proiektuaren datu-basearen eredu erlazionalaren diagrama UMLz. 50
5.3 MCPren jokaera aplikazioan. 54

6.1 `run_python_exercise` tresnaren deiaren exekuzio-fluxuaren sekuentzia-diagrama. 65
6.2 Frontendaren ariketak orrialdeko GUIa. 69
6.3 Frontendaren lan-ingurune orrialdeko GUIa. 70
6.4 Frontendaren ikaslearen aurrerapen orrialdeko GUIa. 70
6.5 Frontendaren profila orrialdeko GUIa. 71

A.1 BF-1 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.1 Taula. . 84
A.2 BF-2 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.2 Taula. . 85
A.3 BF-3 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.3 Taula. . 86
A.4 BF-5 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.5 Taula. . 87
A.5 BF-6 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.6 Taula. . 88
A.6 BF-7 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.7 Taula. . 89
A.7 BF-8 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.8 Taula. . 90
A.8 BF-9 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.9 Taula. . 91
A.9 BF-10 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.10 Taula. 92
A.10 BF-11 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.11 Taula. 93
A.11 BF-12 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.12 Taula. 94
A.12 BF-13 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.13 Taula. 95
A.13 BF-14 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.14 Taula. 96
A.14 BF-15 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.15 Taula. 97
A.15 BF-16 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.16 Taula. 98
A.16 BF-17 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.17 Taula. 99
A.17 BF-18 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.18 Taula. 100
A.18 BF-19 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.19 Taula. 101

A.19 BF-20 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.20 Taula. 102

Taulen aurkibidea

3.1 Proiektuko garapen mugarriak. 19
3.2 Proiektuko lan-atazen garapen denbora estimazioak. 22
3.3 Proiektuan zehar gerta daitezkeen arriskuen taula. 24

4.1 Erregistratzea barneratzen duen BF-1 betekizuna. 28
4.2 Saioa hastea barneratzen duen BF-2 betekizuna. 29
4.3 Saioa ixtea barneratzen duen BF-3 betekizuna. 30
4.4 Pasahitza berreskuratzea barneratzen duen BF-4 betekizuna. 31
4.5 Profila ikustea barneratzen duen BF-5 betekizuna. 32
4.6 Datu pertsonalek aldaketa barneratzen duen BF-6 betekizuna. 32
4.7 Pasahitza aldatzea barneratzen duen BF-7 betekizuna. 33
4.8 Ikasketa maila aldatzea barneratzen duen BF-8 betekizuna. 34
4.9 Kontua ezabatzea barneratzen duen BF-9 betekizuna. 35
4.10 Saiakerak ezabatzea barneratzen duen BF-10 betekizuna. 35
4.11 Mezuak ezabatzea barneratzen duen BF-11 betekizuna. 36
4.12 Aurrerapena begiratzea barneratzen duen BF-12 betekizuna. 37
4.13 Ariketak zerrendatzea barneratzen duen BF-13 betekizuna. 37
4.14 Ariketak bilatzea barneratzen duen BF-14 betekizuna. 38
4.15 Ariketa aukeratzea barneratzen duen BF-15 betekizuna. 38
4.16 Ariketaren lengoaia aldatzea barneratzen duen BF-16 betekizuna. 39
4.17 Saiakerak ikustea barneratzen duen BF-17 betekizuna. 40
4.18 Saiakera bidaltzea barneratzen duen BF-18 betekizuna. 40
4.19 Saiakera irekitzea barneratzen duen BF-19 betekizuna. 41
4.20 Laguntza eskatzea barneratzen duen BF-20 betekizuna. 42
4.21 Sistemaren segurtasuna barneratzen duen BEF-1 betekizuna. 43
4.22 Sistemaren eskalagarritasuna barneratzen duen BEF-2 betekizuna. 44
4.23 Sistemaren mantengarritasuna barneratzen duen BEF-3 betekizuna. 44
4.24 Sistemaren erabilerraztasuna barneratzen duen BEF-4 betekizuna. 45
4.25 Sistemaren eramangarritasuna barneratzen duen BEF-5 betekizuna. 45
4.26 Helburu zehatzen eta betekizunen trazabilitate matrizea. 45

5.1 Backendak izango dituen endpoint nagusiak. 52

7.1 Proiektuko lan-atazen garapen denbora errealak eta desbiderapenak. 74

Kodeen aurkibidea

5.1 Kode exekuzio MCP zerbitzarien tresnen sarrera eskema orokorra. 53
5.2 Kode exekuzio MCP zerbitzariaren tresnen erantzunaren eskema. 53
5.3 Kode exekuzio MCP zerbitzariaren tresnen erantzunaren `text` parametroaren eskema. 53

6.1 Prisma erabiliz `Erabiltzailea` taularen inplementazioa. 60
6.2 Zod erabiliz erabiltzaileak saioa hasteko bidali behar dituen datuen eskemaren inplementazioa. 61

KAPITULUA 1

Sarrera

1.1 Testuingurua eta arazoaren deskribapena

Azken urteetan LLM ereduak asko hobetu dira eta gaur egun edonork erabil ditzake egunerokoatazak zailtasun handirik gabe burutzeko. Hala ere, hezkuntzaren esparruan eredu hauek erronka handiak ekarri dituzte ikasleen irakaskuntzan eta pentsamendu kritikoaren garapenean, maiz, ikasleek egin beharreko ikasketa-atazak LLMak egiten baitituzte haien ordez.

Erronka hauei aurre egiteko ekimen ezberdinak sortu dira. Ekimen nagusiak ikasleek entregatutako ariketen soluzioak AAk idatzitakoak diren hautemateko detekzio softwareak dira. Lan honetan egingo den ekimenak beste ikuspuntu bat hartzen du: programazio ikasleei laguntza pertsonalizatua emateko LLM teknologia aztertu eta horretan oinarritutako web-plataforma bat eraikitzea.

1.2 Proposatutako irtenbidea: sistemaren deskribapena

GrAL honek LLMen berrikuntza pedagogiarako erabiltzea du helburu. Helburu honekin web-plataforma bat eraikiko da programazio-ikasleei laguntza emateko. Bertan, ikasleek programazio ariketak egin eta exekutatzeko aukera izango dute. Horrez gain, edozein momentuan txatbot adimendu bat erabiliz AAri kontsultak egiteko aukera izango dute, egiten ari diren ariketara eta testuingurura egokitutako laguntza jasotzeko.

MCP protokoloa erabiliz AAk ikasleari ez dizkio erantzun zehatzak emango, baizik eta egiten ari den ariketaren zailtasun maila eta ikaslearen ikaskuntza maila kontuan hartuta feedback pertsonalizatua eskainiko dio: ikaskuntza autonomoa sustatzen duen feedback pedagogikoa. Honi esker, ikasleek nahi adina saiakera egin ditzakete ariketa egoki egiteko, eta erantzun egokira gerturatuz joango dira AAk itzultzen dituen pistak erabilita. Azken finean, sistemak programazio-prozesuan laguntzen die, ariketaren soluzioa zuzenean eman gabe.

1.3 Helburu orokorra

GrAL honen helburu orokorra programazio ikasleentzako web-plataforma bat diseinatzea eta inplementatzea da, LLM ereduak eta MCP protokoloa erabiliz ikaskuntza autonomoa sustatzen duen feedback pedagogikoa eskaintzeko.

1.4 Memoriaren antolaketa

Memoria modu honetan antolatuta dago. Sarreran (kapitulu hau) GrAL honetako proiektua zergatik egin den eta zer garatu den labur azaltzen da. Aurrekariak kapituluan (2 kapitulua) proiektuan erabili diren teknologia nagusiak eta artearen egoera azaltzen da. Planifikazioa kapituluan (3 kapitulua) proiektuaren arrakasta bermatzeko informazio bilketa egin da. Sistemaren betekizunen katalogoa kapituluan (4 kapitulua) sistemak bete behar dituen informazio betekizunak, betekizun funtzionalak eta betekizun ez-funtzionalak bildu dira.

Arkitektura eta diseinua kapituluan (5 kapitulua) sistemaren arkitektura orokorra, datu-basearen diseinua, backend zerbitzariaren endpointak, MCP zerbitzariaren funtzionamendua eta LLMari bidalitako testuingurua zehaztu dira. Inplementazioa kapituluan (6 kapitulua) datu-basearen, backend zerbitzariaren, MCP zerbitzariaren eta frontendaren inplementazio erabakiak adierazi dira. Jarraipen eta kontrola kapituluan (7 kapitulua) proiektuaren denbora desbideratzeak eta jarraipen erabakiak azaltzen dira. Ondorioak kapituluan (8 kapitulua) lortutako emaitzak eta etorkizunean egiteko falta dena azaltzen dira.

Eranskinetan, betekizun funtzionalen sekuentzia-diagramak, backendaren endpointen zehaztapenak, lanaren garapenerako erabilitako tresnen zerrenda, AA sortzailearen erabileraren deklarazioa eta bileren aktak aurkitu daitezke. Amaieran, memorian erabili diren akronimoen zerrenda eta bibliografia ikuskatu daitezke.

KAPITULUA 2

Aurrekariak

Proiektuaren garapena hasi aurretik, kapitulu honetan proiektuan zehar erabili diren teknologiei eta kontzeptuei testuingurua ematen zaie. Kapituluaren helburu nagusia irakurleak proiektua hobeto ulertu ahal izateko beharrezkoak diren atazak barneratzea da.

Lehenik eta behin, sortu den aplikazioa deskribatzen da eta garapenaren motibazioa zehazten da. Ondoren, LLM eta MCP teknologiak azaltzen dira. Azkenik, artearen egungo egoera aztertzen da eta jada existitzen diren tresnak alderatzen dira.

2.1 Motibazioa

Azken urte hauetan ikertzaileek gero eta LLM boteretsuagoak garatu dituzte. Memoria hau idazten ari den unean, OpenAI-ren GPT-5.4 [2], Anthropic-en Claude Opus 4.6 [3] eta Google-n Gemini 3.1 Pro [4] eskuragarri daude, besteak beste. Tresna hauek egun oroko atazak egiteko oso baliagarriak dira. Adibidez, ulertzen ez diren kontzeptuak azaltzeko edo galdera orokorrak egiteko.

Eredu hauen hedapenak, ordea, erronka nabarmena dakar hezkuntzara. Gaur egungo ikaskuntza-prozesuek oinarri handia dute ikasleek zereginak beren kabuz egitean. Baina LLM ereduak gero eta eskuragarriagoak daudenez, ikasleek ariketen emaitzak zuzenean lortzeko erabiltzen dituzte askotan, jasotako kodea aztertu edo egiaztatu gabe.

Arazo hau gaur egun pedagogiaren erronka nagusienetako bat bihurtu da, eta jada hainbat ekimen interesgarri garatu dira. Ekimen nagusienetarikoen artean, testua jaso eta AA batek idatzi duen ala ez itzultzen duten tresnak daude, hots, sailkapen bitarreko arazo bat ebatzi nahi duten softwareak. Tresna hauek hainbat diseinu ezberdinetan oinarritzen dira: filigranak, estatistikoak eta neurona-sareetan oinarritutakoak [5]. Memoria hau idazterakoan, hezkuntzaren esparruan ekimen hau bultzatzen duten software hauen artean GPTZero[1], Turnitin[2], OriginalityAI[3], DetectGPT[4] eta FastDetect[5] aurkitu ditzakegu, besteak

[1] https://gptzero.me/
[2] https://turnitin.com/
[3] https://originality.ai/
[4] https://github.com/eric-mitchell/detect-gpt
[5] https://fastdetect.net/

beste [6].

Lan honetan, ordea, beste ikuspuntu batetik helduko zaio erronkari. Ariketak LLM bidez ebaztea soilik baztertu beharrean, teknologia hori bera erabiliko da ikasleei laguntza pertsonalizatua zuzenean emateko. Ikuspuntu honek ezinbestekoa bihurtzen ari den teknologia pentsamendu kritikoa mantenduz erabiltzeko aukera ematen dio ikasleari, beste lan-tresna bat balitz bezala. Hori lortzeko, MCP protokoloaren inplementazioan sakonduko da, domeinu akademikoa eta hizkuntza-ereduen arteko zubia eraikiz.

2.2 Adimen artifiziala eta hizkuntza-eredu handiak

LLMak giza hizkuntza ulertzea, prozesatzea eta sortzea helburu duen Hizkuntzaren Prozesamenduko —ingelesez, Natural Language Processing (NLP)— ikasketa sakoneko ereduak dira [7]. Hasierako NLP sistemek testuinguruaren eskalagarritasunarekin eta urruneko hitzen arteko informazioari eusteko zailtasunak zituzten. Erronka hori, ordea, 2017an gainditu zen *Transformer* arkitekturaren etorrerarekin; autoarretan (*self-attention*) oinarritutako arkitektura honek sekuentzien prozesamendua erabat eraldatu eta gaur egungo LLMen oinarri teknologikoa finkatu zuen [8, 9].

2.2.1 Tokenizazioa eta sarreraren prozesamendua

Testua eredu batean txertatu aurretik, ordenagailuek ulertzeko moduko zenbaki-balio bihurtu behar da. Prozesu honek hiru urrats nagusi ditu:

- **Tokenizazioa:** Testua unitate txikiagotan (tokenetan) banatzen da, hitzak edo azpihitzak izan daitezkeenak. Token bakoitzari zenbaki-balio bat esleitzen zaio hiztegi baten arabera.
- ***Embedding*-ak:** Token bakoitza espazio bektorial multidimentsional batean irudikatzen da. Horri esker, ereduak hitzen esanahi semantikoa ulertzen du, antzeko kontzeptuak dituzten hitzak espazio horretan gertu kokatzen baitira [10].
- **Posizio-kodetzea (*Positional Encoding*):** Embeddingek esanahia ematen diote testuari, baina ez hitzen ordenaren berri. Hori konpontzeko, gaur egungo eredu nagusiek Rotary Position Encoding (RoPE) mekanismoa erabiltzen dute, autoarreta-algoritmoan biraketa-matrizeak txertatuz hitzen arteko distantzia erlatiboa kalkulatzeko [11].

2.2.2 Transformer arkitektura eta arreta mekanismoa

Transformerren arrakastaren gakoa autoarreta-mekanismoa (*self-attention*) da. Gizakiok irakurtzean gako-hitz batzuei erreparatzen diegun bezala, mekanismo honi esker ereduak sarrera-sekuentzia baten barruko token bakoitzaren eta gainerako guztien arteko lotura logikoak kalkulatzen ditu [8, 12].

Jatorrizko arreta-mekanismoek bi sekuentzia ezberdin alderatzen zituzten bitartean —adibidez, hizkuntzen arteko itzulpenetan—, autoarreta sekuentzia bakarrarekin erabiltzen da. Horrela, ereduak esaldi baten testuinguru osoa aldi berean eta modu eraginkorrean prozesatzeko gaitasuna lortzen du [8].

2.2.3 Entrenamendu faseak

LLM ereduak erabili aurretik entrenatu egin behar dira. Entrenamendu hau bi zatitan banatzen da: aurre-entrenamendua eta doikuntza. Lehenik ereduaren aurre-entrenamendua egiten da, eta honen ondoren, doikuntza.

1. **Aurre-entrenamendua:** Ereduak hizkuntzaren egitura orokorra ikasten du testu-multzo masiboak aztertuz (liburuak, webguneak, kodea). Algoritmo auto-gainbegiratu baten bidez, ereduak testuinguru baten arabera hurrengo token egokia iragartzen ikasten du [10].
2. **Doikuntza (*Fine-tuning*):** Behin hizkuntzaren oinarria edukita, eredua zeregin edo testuinguru espezifiko batekin entrenatzen da —adibidez, txatbot laguntzaile izateko—. Fase honek erantzunak eremu zehatz horretara mugatzen eta zehaztasuna hobetzen laguntzen du [10].

LLMek, doikuntza jaso arren, muga garrantzitsu bat dute: entrenatuta dauden datuetara mugatuta daude eta ezin dute isolatuta dauden kanpoko sistemekin —adibidez, datu-baseekin edo kode-exekutatzaileekin— modu natiboan komunikatu. Muga hori gainditzeko eta ereduei testuinguru dinamikoa zein ekintzarako gaitasuna emateko sortu da MCP protokoloa.

2.3 Zer da MCP?

MCP Anthropicek 2024an —geroago 2025ean Agentic AI Foundation-ari (AAIF) dohaintzan emana [13]— aurkeztutako estandar berri bat da AA ereduen eta kanpoko informazio sistemen arteko komunikazioa zehazten duena. Estandar hau sortu aurretik, AA erabiltzen zuen eta kanpoko informazioa behar zuen tresna bat informazioa eskaintzen zuen kanpoko tresna batekin konektatu nahi zen aldi bakoitzean, inplementazioa neurrira egin behar zen [14].

2.3.1 Arkitektura eta datu-geruza

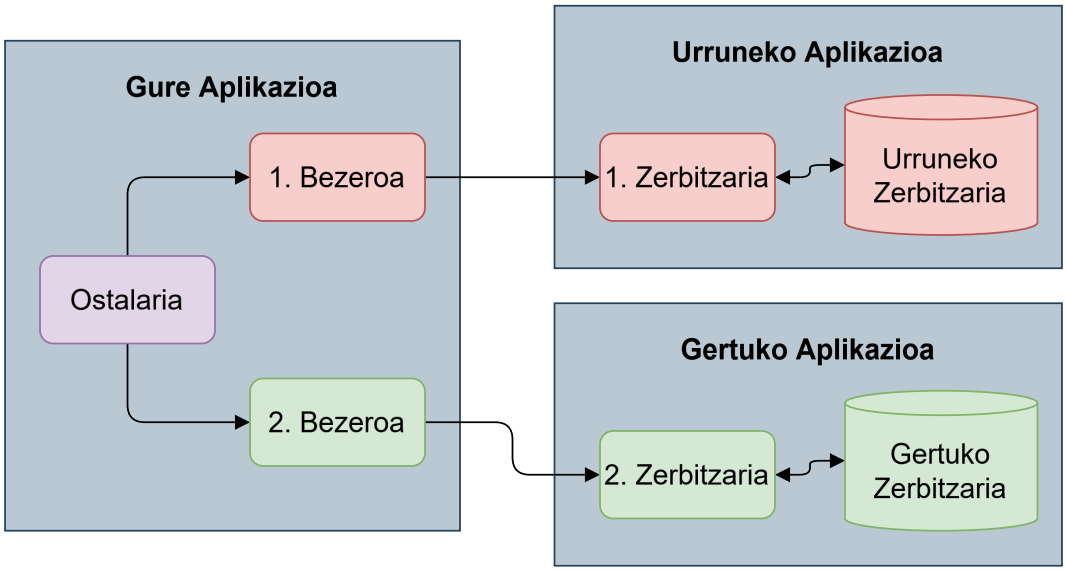
Protokoloak hiru osagarriko arkitektura jarraitzen du, ikusi 2.1 Irudia: ostalaria, bezeroa eta zerbitzaria.

Gure kasuan, garatuko den web-aplikazioaren backendak ostalari gisa jokatuko du. Honek, iturri ezberdinetako testuingurua lortzeko MCP bezeroak sortuko ditu, eta bezero hauek dagozkien MCP zerbitzariekin —adibidez, ariketak exekutatzen dituen zerbitzariarekin— konektatuko dira. Arkitektura honi esker, eginkizun ezberdinen isolamendua eta segurtasuna bermatzen da.

Protokoloak bi geruza definitzen ditu: datu-geruza eta garraio-geruza. Lehenengoak bezeroaren eta zerbitzariaren arteko komunikazioa zehazten du. Bigarrenak bezeroaren eta zerbitzaria komunikatu ahal izateko beharrezkoak diren mekanismoak eta bideak definitzen ditu [15].

Komunikazio hau bideratzeko, protokoloak datu-geruza bat definitzen du. Bezeroaren eta zerbitzariaren arteko datu-trukerako JSON RPC (JSON-RPC) 2.0[6] espezifikazioan —

[6] https://www.jsonrpc.org/specification



2.1 Irudia: MCPko osagarrien arkitektura.

JavaScript Object Notation (JSON) erabiltzen duen Remote Procedure Call (RPC) protokolo bat— oinarritzen da. Protokolo honek lau eginkizun garrantzitsu definitzen ditu [15]:

Bizi-zikloaren kudeaketa
Zerbitzariaren eta bezeroaren arteko konexioa kudeatzen du eta parte-hartzaileen gaitasunen negoziazioa zehazten du.

Zerbitzariaren ezaugarriak
Zerbitzariek oinarrizko funtzionalitateak eskaintzen dituzte: tresnak —ingelesez, *tools*— AA ekintzetarako, baliabideak —ingelesez, *resources*— testuinguru-datuak emateko eta *prompt*ak bezeroarentzat elkarreragin-txantiloi gisa.

Bezeroaren ezaugarriak
Zerbitzariei bezeroari eskaerak egiteko gaitasuna ematen die: ostalariko LLMari erantzunak eskatzea, erabiltzailearen sarrera jasotzea eta erregistro mezuak bidaltzea.

Ezaugarri osagarriak
Beste hainbat gaitasun zehazten ditu, hala nola, denbora errealeko jakinarazpenak eta iraupen luzeko eragiketen aurrerapen jarraipena.

2.3.2 Garraio-geruza

Garraio-geruzak bezeroa eta zerbitzaria komunikatu ahal izateko beharrezkoak diren mekanismoak definitzen ditu, komunikazio segurua bermatuz. Horretarako, MCPek bi garraio mekanismo inplementatzen ditu, bat bezeroa eta zerbitzaria gailu berdinean badaude, eta beste bat gailu ezberdinetan badaude [15].

STDIO garraioa
Bezeroa eta zerbitzaria gailu berdinean daudenean sarrera eta irteera estandarrak —ingelesez, Standard Input/Output (STDIO)— erabiliz prozesuen interkomunikazioa

ahalbidetzen da. Kasu honetan bezeroak MCP zerbitzaria bezeroaren gailuan bertan hasieratu behar du, beraz, mekanismo hau erabiltzen duten MCP zerbitzariek MCP bezero bakarra izango dute, hots, 1 . . 1 erlazioa mantenduko dute.

Streamable HTTP garraioa
Bezeroa eta zerbitzaria gailu ezberdinetan daudenean Hypertext Transfer Protocoleko (HTTP) POST eskaerak erabiltzen ditu elkarren arteko komunikazioa egiteko. Bien arteko autentifikazioa bermatzeko tokenak, Application Programming Interface (API) giltzak edo goiburu pertsonalizatuak onartzen ditu. Horrez gain, Server-Sent Events-ak (SSE) onartzen ditu denbora errealeko datuak bidaltzeko.

2.3.3 MCP elementu nagusiak: prompts, resources eta tools

MCP protokoloko bertsio gehienetan erabilgarri dauden elementu nagusiak zerbitzariek eskaintzen dituzten oinarrizko funtzionalitateak dira. Oinarrizko elementuak hiru dira, eta elementu bakoitza entitate desberdinak erabakitzen du noiz erabili.

Tresnak (Tools)
Tresnak LLM ereduek deitu ditzakeen eta ekintzak garatzen dituzten funtzioak dira. Tresna bakoitzak, tresnaren izena, tresna noiz erabili behar den adierazten duen deskribapena eta tresna erabiltzeko pasa behar zaion sarreraren eskema definitzen ditu. Tresnak LLM ereduak erabakitzen du noiz erabili.

Baliabideak (Resources)
Baliabideak ostalariak ereduari bidaltzeko eskuratu ditzakeen irakurketa-soileko datuak dira. Baliabide bakoitza Uniform Resource Identifier (URI) bat erabilita definitzen da. Baliabideak ostalariak erabakitzen du noiz erabili.

Promptak (Prompts)
Promptak[7] ereduaren erantzuna egituratzeko erabiltzen diren txantiloiak dira. Promptek txantiloia pertsonalizatzeko argumentuak definitu ditzakete. Promptak erabiltzaileak erabakitzen du noiz erabili.

Adibide praktiko gisa, GrAL honetan erabiliko den alderdi bat aztertuko da. Kode exekuzio MCP zerbitzariak kodea exekutatzeko hiru tresna desberdin eskainiko ditu, tresna bat inplementatuko den programazio-lengoaia bakoitzeko. Backendak, ostalari gisa, tresnak erabiltzeko MCP bezero bat sortuko du. Gero, ikaslearen kodea edo LLM ereduaren kodea exekutatzeko programazio-lengoaiari dagokion tresna deituko da, eta exekuzioaren emaitza erantzun gisa eskuratuko da.

Laburbilduz, hiru elementu nagusi hauen erabilerak hizkuntza-ereduekin elkarrizketa segurua mantentzen dute: tresnek exekutatu daitezkeen funtzioak eskaintzen ditu, baliabideek irakurketa-soileko informazioa eskuragarri jartzen dute, eta promptek ereduaren erantzunari forma ematen die. Protokoloaren oinarrizko elementuak ezagututa, ezinbestekoa da gaur egun hezkuntzan eta programatzean erabiltzen diren soluzioak aztertzea. Hurrengo atalean artearen egoerako tresna nagusiak aztertuko dira.

[7] Garrantzitsua da aipatzea MCPren testuinguruan, *prompt* terminoak zerbitzariak eskaintzen dituen elkareragin-txantiloiak direla, ez erabiltzaileak LLMari sarrera gisa bidaltzen dion testua.

2.4 Artearen egoera

Atal honetan gaur egun erabiltzen diren ikaskuntza plataforma eta tresna nagusiak aztertuko dira. Zehazki, ikaskuntza bultzatzeko AA nola inplementatzen duten aztertuko da.

2.4.1 Ikaskuntza plataforma orokorrak

Lehenengo, ikaskuntza orokorra helburu duten eta mundu mailan zabalduta dauden plataformak aztertuko dira, AA teknologia nola inplementatu duten azaltzeko.

2.4.1.1 Moodle

Moodle Martin Dougiamasek 2002an sortutako kode irekiko ikaskuntza plataforma birtuala da. Plataformak aukera ematen du Moodle instantzia propioak exekutatzeko eta ikastaroen bidez ikaskuntza kudeatzeko. Ikastaroak blokeen bitartez osatzen dira, eta bloke bakoitzak hainbat funtzionalitate izan ditzake: testak, dokumentuak, galdetegiak, informazio blokeak, entregagaiak, foroak, etab [16].

Sistemak bi erabiltzaile-rol nagusi bereizten ditu: irakasleak —edukiak aldatzeko eta ebaluatzeko baimenarekin— eta ikasleak —edukiak kontsultatu eta zereginak burutzeko baimenarekin— [17]. Gaur egun, plataformak 500 milioi erabiltzaile eta 55 milioi ikastaro baino gehiago ditu mundu osoan [18].

Moodleek 4.5 bertsioan AA gehitu du plataforman, baina erabilera oso mugatua du. Batetik, irakasleek aukera dute ataza berri bat sortzerakoan hasierako testu batetik beste testuak eta irudiak sortzeko. Bestetik, ikasleek aukera dute testu atazetan edukiaren laburpena sortzea. Ondorengo 5.0 bertsioan, irakasleek LLM eredua aldatzeko aukera inplementatu da, baita ere erabiltzaileek AArekin egindako elkarekintzen txostena sortzea. Azkenik, ikasleek aukera dute testu atazetan edukiaren azalpena sortzea [19]. Hala ere, plataformak ez du programazio ariketen soluzioak exekutatzeko ingurunerik, ezta feedback pertsonalizatua zuzenean itzultzeko aukerarik.

2.4.1.2 Canvas

Canvas Instructure-ek 2011an sortu zuen kode irekiko ikaskuntza plataforma birtuala da, orduko beste sistema batzuen erabilgarritasun-arazoei eta intuizio-faltari aurre egiteko jaio zena. Plataforma honen ezaugarriak Moodleek eskaintzen dituenaren oso antzekoak dira, baina erabiltzeko erraztasuna hobetzen du eta pertsonalizatzeko gaitasuna gutxitzen du. Horrez gain, garrantzi handia ematen dio beste aplikazioekin integrazio zuzena lortzeari [20, 21].

2026an hainbat AA tresna bildu dituzte eta plataforman integratu dute: IgniteAI. Tresna honek hainbat funtzionalitate eskaintzen dizkie irakasleei eta ikasleei. Batetik, irakasleentzat, atazak automatikoki sortzeko laguntza eskaintzen du, notak jartzeko ikasleen atazak ebaluatzen lagundu eta ikasleen eztabaiden laburpenak egin ditzake. Bestetik, ikasleentzat, ikastaroko atazak aurkitzen laguntzen du eta atazak edo mezuak itzultzeko aukera ematen du [22]. Beraz, Canvasen inplementatutako AAk ikuspegi administratiboa du nagusiki, eta Moodleko inplementazioaren antzera, ez du aukerarik ikaslearen kodearen feedback zuzena emateko.

2.4.1.3 Google Classroom

Classroom Googlek 2014ean sortutako kode itxiko ikaskuntza plataforma birtuala da. Beste plataforma birtualen antzera, irakasle eta ikasle rolak daude. Irakasleek ikastaroak sortu ditzakete eta bertara ikasleak gehitu. Plataformak irakasleari hainbat elementu gehitzeko aukera ematen dio: lanak, testak, galderak eta dokumentuak. Azkenik, ikasleek igotako entregagaiak puntuatzeko aukera ematen dio irakasleari. Beste plataformekin aldaratuz gero, Classroomen indargune nagusienetarikoa Googlek hodeian eskaintzen dituen tresnak —Docs[8], Sheets[9], Slides[10] eta abar— zuzenean barneratuta daudela da [23, 24].

2025ean Googlek Gemini erabilgarri jarri zuen "Google Workspace for Education" kontua zuten irakasle guztiontzat. Tresna honen bitartez irakasleak atazak azkarrago sor ditzakete. Gainera, irakasleak aukera dute sortutako ataza batean "Gem" bat zehazteko. Gemak neurrira egindako Geminiren bertsioak dira eta honen bitartez AA zehaztutako atazan aditu bilakatzen da. Gero, ikasleak gema hau erabili dezake atazaren inguruan gehiago ikasteko [25]. Irakasleak sortutako gem horrek atazan aditu gisa jokatzen duen arren, ikaslearekin interakzioa txatbot modura mugatzen da. Erabili den Gemini bertsioaren arabera kode exekuzioa mugatuta egon daiteke, baina ez du aukerarik ikaslearen aurrerapenaren informazioa erabiliz feedback pertsonalizatua sortzeko.

2.4.2 Ataza zehatzetarako tresnak

Ondoren, ikaskuntza plataforma orokorrak izan gabe, programazioaren irakaskuntzan AA inplementatzen duten ingurune edo tresna espezifikoak aztertuko dira.

2.4.2.1 Gemini Guided Learning

Googlek 2025ean "Guided Learning" modua gehitu zion Geminiri. Modu hau aktibatzean eta erabiltzaileak galdera bat egiten dionean AAri, erabiltzaileak ez du erantzun zuzen bat jasoko, baizik eta elkarrizketa moduko erantzun bat jasoko du. Elkarrizketan zehar Geminik galderak egingo dizkio erabiltzaileari, jasotako erantzunaren arabera azalpenak emango dizkio eta hasierako galdera erantzun ahal izateko hurrengo urratsa zein izango den adieraziko dio [26]. Tresnak ikuspegi pedagogikoa gertu barneratzen du, baina txatbot modura mugatzen da. Gemen antzera, erabili den Gemini bertsioaren arabera kode exekuzioa posiblea izan daiteke, baina ikaslearen inguruko testuingururik ez du.

2.4.2.2 Codecademy eta Datacamp

Codecademy programazio-ikastaro zehatzak (programazio-lengoaiak, tresnak, kontzeptu teknikoak, etab) egiteko plataforma ezaguna da.

Eduki gehienak ordainpekoak dira eta ikasleak ariketak plataforman bertan exekutatu eta egiaztatu ditzake. Ariketak egitean, AA laguntzaile bat integratzen da albo-panel batean, idatzitako kodea eta ariketaren testuingurua kontuan hartuta ikaslearen galderak erantzuteko [27].

Datacamp programazio ikastaro zehatzak egiteko beste plataforma bat da. Plataforma honek Codecademyk eskaintzen dituen ikastaro mota antzekoak eskaintzen ditu, baina

[8] https://docs.google.com
[9] https://sheets.google.com
[10] https://slides.google.com

datuen ingeniaritzari pisu handiagoa ezartzen dio. Plataformak ikastaro guztien lehenengo kapitulua doan eskaintzen du, baina gainontzeko kapituluak egiteko ordaindutako kontu bat behar da. Ikastaroko ariketei dagokienez, Codecademyren antzeko egitura jarraitzen dute eta AA laguntzaile propioa ere eskaintzen du [28].

Bi plataformek ikaslearen kode exekuzioa inplementatzen duten arren, eskaintzen duten AAk txatbot konbentzional gisa jokatzen du; hots, pentsamendu kritikoa bultzatu beharrean, erantzun eta soluzio zuzenak emateko joera du.

2.4.2.3 LeetCode

LeetCode programazio-ariketak egiteko plataforma erabilienetarikoa da, hilabetero 30 milioi bisita baino gehiago jasotzen baititu [29]. Plataformak doako kontuak eta ordaindutakoak eskaintzen ditu. Ordaindutakoek konpainia handiek elkarrizketetan erabiltzen dituzten ariketak eskuragarri dituzte. Doakoek, ordea, beste ariketa guztiak eskuragarri dituzte. Proba unitarioen bidez kodea automatikoki ebaluatzeko sistema du eta "Ask Leet" izeneko AA eskaintzen du.

GrAL honetan proposatzen den plataformara gehien gerturatzen den soluzioa da. Hala ere, plataformak bi muga ditu: sistema itxia denez MCP protokoloaren erabilera ezezaguna da eta eskaintzen duen AAk ariketen soluzio osoak itzultzen ditu, balio pedagogikoa jaitsiz [30]. Hala ere, plataformaren ariketa-diseinua oso lagungarria izango da GrAL hau egiteko. 3 kapituluan azalduko den bezala, ariketen diseinua GrAL honen irismenetik kanpo utziko da, eta, beraz, ariketa gehienak plataforma honetatik hartuko dira.

KAPITULUA 3

Planifikazioa

Kapitulu honetan, proiektua aurrera eraman ahal izateko planifikazioa zehaztu da. Bertan, lanak izango duen irismena, atazen denboraldiak eta entregagaien amaiera-datak, ataza bakoitza garatzeko emandako denboren estimazioak, proiektua garatzeko erabiliko informazio eta komunikazio sistemak eta arriskuen analisia azaltzen dira.

3.1 Irismena

Lehenik eta behin, proiektua garatu ahal izateko irismena noraino definitu zehaztu beharko da.

3.1.1 Helburu zehatzak

Honakoak dira proiektu hau garatzeko bete beharko diren helburu zehatzak:

HZ1 Programazio-ikasleentzat web-plataforma bat diseinatzea eta garatzea (ariketak egiteko editorea, laguntza txata eta aurrerapen *dashboard* bat integratuko dituena), ikaskuntza autonomoa sustatzen duen berrelikadura pedagogiko egokitua eskaintzeko.

HZ2 Hiru geruzako bezero-zerbitzari oinarrizko arkitektura modularra inplementatzea. Arkitekturaren tartean MCP geruza espezializatua kokatuko da.

HZ3 MCP geruza integratzea arkitekturan; web-sistemaren eta LLMen arteko testuingurutruke estandarizatua ahalbidetzeko.

HZ4 Ikaslearen mailara egokitutako prompt pedagogikoen egitura diseinatzea, LLMaren erantzunak eta laguntza-maila datu-basean gordetako ikaslearen informazioaren arabera mugatzeko.

3.1.2 Eskakizunak

Garatu beharreko softwareak ondoko eskakizunak bete behar ditu:

Frontenda
Single Page Application (SPA) bat izango da. Honetarako, React *framework*a erabiliko da. Aplikazioak hiru osagai nagusi integratuko ditu: Monaco kode editorea, laguntza txata integratua eta ikaslearen aurrerapena erakutsiko duen dashboarda.

Backenda
Representational State Transfer (REST)-API arkitektura jarraituko du. Node.js ingurunea eta Express frameworka erabiliz eraikia. Backendak honako eginkizunak bete beharko ditu: erabiltzailearen autentifikazioa, ariketen kudeaketa eta MCP orkestrazioa kudeatuko du. Azken honen helburua, frontendak jasotako kodea MCP zerbitzariari transmititzea da, testuingurua LLMaren APIari bidaltzea eta LLMaren feedback egokitua erabiltzaileari atzera bidaltzea.

MCP geruza
MCP geruzak zerbitzari bat du.

- Ariketen eta proben exekuzioak ingurune seguruetan isolatzeko, kode exekuzio MCP zerbitzari bat inplementatuko da. Zerbitzari honek kodea Docker ingurune isolatu batean exekutatuko du, exekuzio emaitzak eta errore txostenak eskuratuz.
- Kode exekuzio zerbitzaria baliabide gabe ez gelditzeko, exekutatuko diren proben exekuzio denbora mugatuta egongo da.

Datu-biltegiratzea
Aplikazioaren funtzionamendua bermatzeko datu-base batean datuak gordeko dira. Ikasteko aukera bezala, PostgreSQL Relational Database Management System-a (RDBMS) erabiliko da. Bertan, erabiltzailearen kontua, erabiltzaileen saio-tokenak, ikasleen informazioa, programazio-lengoaiak, ariketen enuntziatuak, ariketen metadatuak, ikasleen elkarrizketak, ikasleen ebazpenak, ikasleen saiakerak eta proben exekuzioen emaitzak gordeko dira.

Ariketen-kudeaketa eta mailakatzea
Ariketen aldetik, ariketak hainbat programazio-lengoaia ezberdinetan egin ahal izango dira. Arazo berezirik ez badago, ariketa mota guztiak aukeratutako programazio-lengoaia guztietan egiteko aukera egongo da. Hasiera batean sortuko diren ariketek zailtasun maila ezberdinak izango dituzte, ikasleek beraren mailara egokitzen diren ariketak eskuragarri izateko. Ikasleak berak ikasketa maila aldatzeko aukera izango du. Maila hau datu-basean gorde beharko da eta backendak erabili beharko du LLMari galderak egitean, ikaslearen laguntza maila mugatzeko.

Kontuen kudeaketa
Kontuen aldetik, erabiltzaileak aukera izan behar du bere kontuaren gainean aldaketak egiteko: erabiltzailea, pasahitza, maila, gordetako metrikak eta elkarrizketak.

3.1.3 Baztertutakoa

Proiektuaren bideragarritasuna bermatzeko eta lan-karga eremu onargarrian mantentzeko, ondokoak uzten dira irismenetik kanpo.

Mugikorretarako aplikazioa
Webgunea mugikorrean erabili ahal izateko eraikiko da, baina ez da mugikorreko aplikaziorik egingo.

Datu-base banatuak
Aplikazioaren eta erabiltzaileen datuak ez dira gordeko hainbat leku fisikoetan. Dena datu-base batean egingo da, segurtasun kopiak eginez beharrezkoa balitz.

Diseinuaren kanpo ebaluazioa
Frontendeko interfazeen diseinuak egitean ez da kanpoko pertsonen berrelikadurarik kontuan hartuko. Prototipoak proiektuaren garatzailearen eta zuzendariaren artean bakarrik berrikusiko dira.

Ariketen sorkuntza
Eskuragarri jarriko diren ariketen enuntziatu gehienak LeetCodetik hartuko dira GrALaren lan-karga murrizteko.

Erantzunen kalitatearen bermatzea
LLMak itzultzen dituen erantzunen kalitatea ez dugu bermatuko eta asumituko dugu jasotako erantzunak lagungarriak direla.

API kontsumoen eta kostuen kudeaketa
Proiektuan kostu gehigarriak saihesteko, doako API *key* bat erabiliko da.

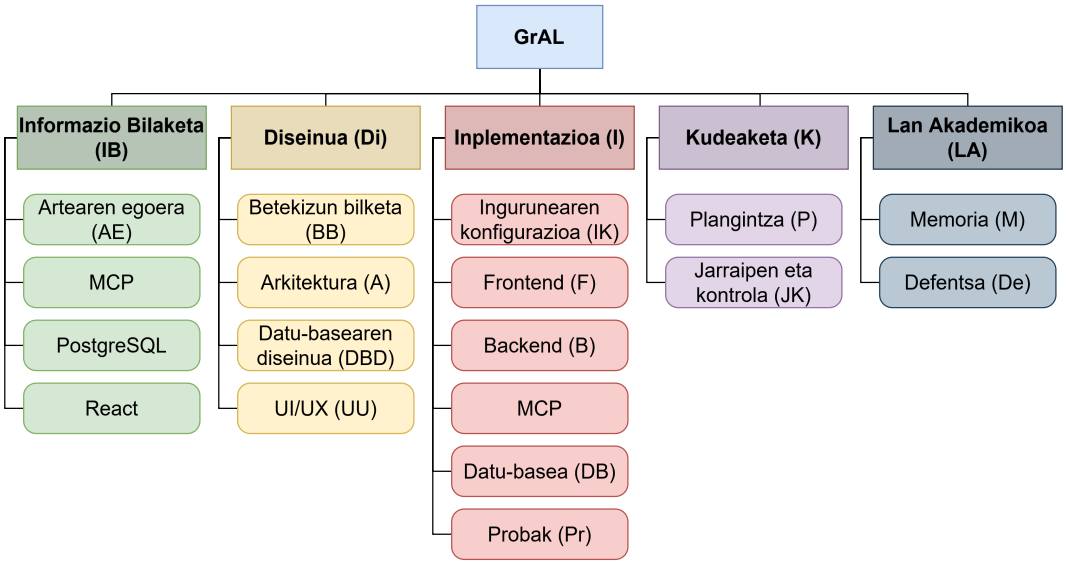
3.1.4 Emangarriak

Proiektuaren garapenak bi emangarri kategoria sortuko ditu: web-plataformari lotutako kodea eta dokumentazio akademikoa:

- **Bezeroaren aplikazioa (Frontend)**: erabiltzaileak sistemarekin elkarreragiteko React aplikazioa.
- **Zerbitzari nagusia (Backend)**: Node.js eta Express bidez osatutako APIa, sistemaren barne logika, erabiltzaile kudeaketa eta LLM orkestrazioa.
- **MCP zerbitzaria**: ikasleen kodea ingurune isolatuan exekutatzeko eta testuinguru estandarizatua eraikitzeko garatutako zerbitzaria.
- **Datu-basearen instantzia**: ariketak eta erabiltzaileen informazioa gordetzen duen datu-basea.

Horrez gain, dokumentazioari dagozkion emangarri hauek ere sortuko dira:

- **Memoria**: dokumentu hau bera, aplikazioaren diseinua, inplementazioa eta jarraipena azaltzen duen dokumentua.
- **Aurkezpena**: proiektuaren defentsa egiteko erabiliko diren diapositibak.



3.1 Irudia: Proiektuaren LDE diagrama.

3.1.5 Lanaren Deskonposizio Egitura (LDE)

Proiektuan zehar egin beharreko lana modu hierarkikoan antolatzeko Lanaren Deskonposizio Egitura (LDE) diagrama bat sortu da, ikusi 3.1 Irudia. Proiektua 5 lan-adar nagusitan banatzen da eta adar bakoitzak hainbat lan-pakete barneratzen ditu. Lan-pakete bakoitzak helburu zehatz bat betetzeko ataza espezifikoak biltzen ditu.

Informazio Bilaketa (IB)

Lan-adar honetako paketeak proiektuaren garapenerako beharrezkoak diren oinarri teknikoak eta ezagutza teknologikoak eskuratzea dute helburu nagusia.

- **Artearen egoera (AE)**: gaur egungo hezkuntza plataformak eta programazio ikaskuntza tresnak aztertzea.
- **MCP**: MCP protokoloaren funtzionamendua, arkitektura eta zehaztapenak aztertzea eta ikastea.
- **PostgreSQL**: PostgreSQL datu-base erlazionalaren ezaugarriak eta funtzionamendua aztertzea.
- **React**: React frameworkaren bidez SPA arkitekturak inplementatzen ikastea.

Diseinua (Di)

Aplikazioaren inplementazioak jarraitu beharreko arkitektura, datu-egitura eta interfazearen diseinuari lotutako lan-paketeak biltzen ditu.

- **Betekizun bilketa (BB)**: proiektuak barneratuko dituen helburu, muga eta eskakizun tekniko eta funtzionalak zehaztu.
- **Arkitektura (A)**: proiektuaren osagai guztien arkitektura eta komunikazio moduak diseinatu.
- **Datu-basearen diseinua (DBD)**: erabiltzaileen kontuak, eta saiakeren eta ariketen datuak gordeko dituen datu-basearen eskema diseinatu.

- **UI/UX (UU)**: web-plataformaren frontendaren itxura eta osagaiak definitu.

Inplementazioa (I)
Diseinu dokumentuan oinarrituz, aplikazioaren garapen teknikoa eta programazio-lanak biltzen dituen adar nagusiaren lan-paketeak.

- **Ingurunearen konfigurazioa (IK)**: proiektuak erabiliko dituen tresna guztiak, dependentziak eta Docker ingurunea konfiguratzea.
- **Frontend (F)**: Diseinatutako frontenda inplementatzea.
- **Backend (B)**: Node.js eta Express bidez REST API zerbitzaria eta testuinguru orkestratzailea inplementatzea.
- **MCP geruza**: Docker bidez kodea isolatuta exekutatuko duen MCP zerbitzaria inplementatzea.
- **Datu-basea (DB)**: Diseinatutako datu-basea eraiki eta hasierako ariketa-datuekin kargatzea.
- **Probak (Pr)**: Sistemaren funtzionalitatea eta osagaien arteko integrazioa bermatzeko proba unitarioak sortzea.

Kudeaketa (K)
Proiektuaren garapen egokia, epeak eta helburuak betetzen direla bermatzeko koordinazio-lanak barneratzen ditu.

- **Plangintza (P)**: proiektuaren irismena, atazak eta denbora estimazioak zehaztea.
- **Jarraipen eta kontrola (JK)**: proiektuaren garapenaren jarraipena garapen egokia bermatzeko.

Lan Akademikoa (LA)
Adar honek GrAL guztietan eskatzen diren betekizunak biltzen ditu.

- **Memoria (M)**: proiektuaren alderdi teknikoa eta metodologikoa biltzen dituen memoria idaztea.
- **Defentsa (De)**: proiektu amaierako aurkezpena eta defentsarako materiala prestatzea.

3.1.6 Suposamenduak

Proiektuaren garapena burutzeko, bi suposamendu egiten dira.

S1 LLMaren erantzunak dagokion testuinguruarekiko egokiak izango dira. Erantzunak ahalik eta onenak izan daitezen, LLMari egin beharreko galdera ahalik eta zehatzena izan beharko da. 3.1.3 azpiatalean azaldu den bezala, ez da konponketa-zereginik zehaztuko.

S2 Erabiliko den doako LLMaren APIak ez du arazorik emango eta beti eskuragarri izango dugu.

3.2 Atazak egiteko denboraldiak eta emangarriak bukatzeko datak

Atal honetan aurreko atalean azaldutako lan-paketeei dagozkien lan-atazak deskribatzen dira. Lan-atazak lan-pakete baten deskonposaketa da, eta ataza bakoitza lan-paketea osatzeko urrats bat da. Gainera, proiektuaren garapenean ordena mantentzeko, lan-atazen garapen denboraldiak zehazten dira, lehenengo zein atazak egin behar diren jakiteko. Azkenik, proiektuko emangarrien garapen mugarriak zehazten dira.

3.2.1 Egin beharreko atazen deskribapena

Jarraian, lan-adarretan banatuta, 3.1 Irudian definitu diren lan-pakete bakoitzean egin behar diren atazak zerrendatzen dira.

Informazio Bilaketa (IB) adarra:

- **IB.AE** (Artearen egoera) lan-paketea:

  IB.AE.A1. Jada existitzen diren hezkuntza plataformak eta programazio ikaskuntza tresnak aztertzea.

  IB.AE.A2. Aztertutako tresnen desberdintasunak alderatzea.

  IB.AE.A3. Memoriaren artearen egoera kapitulua idaztea.

- **IB.MCP** lan-paketea:

  IB.MCP.A1. Protokoloaren dokumentazioa irakurtzea eta aztertzea.

  IB.MCP.A2. Zerbitzari eta bezero inplementazio adibideak aztertzea.

  IB.MCP.A3. Memorian MCP protokoloaren oinarri teknikoa azaltzea.

- **IB.PostgreSQL** lan-paketea:

  IB.PostgreSQL.A1. PostgreSQL datu-basearen erabilera dokumentazioa aztertzea.

  IB.PostgreSQL.A2. Node.js ingurunearekin konektatzeko liburutegiak eta diseinuak aztertzea.

- **IB.React** lan-paketea:

  IB.React.A1. React frameworkaren dokumentazioa irakurri.

  IB.React.A2. Monaco kode editorea Reacten integratzeko moduak ikastea.

  IB.React.A3. SPA adibideak ikuskatzea.

Diseinua (Di) adarra:

- **Di.BB** (Betekizun bilketa) lan-paketea:

  Di.BB.A1. Proiektuak bete beharko dituen helburu zehatzak eta eskakizunak finkatzea.

Di.BB.A2. Sistemaren erabiltzaile profilak zehaztea.

Di.BB.A3. Sistemaren helburuak zehaztea.

Di.BB.A4. Sistemaren informazio betekizunak zehaztea.

Di.BB.A5. Sistemaren betekizun funtzionalak zehaztea.

Di.BB.A6. Sistemaren betekizun ez-funtzionalak zehaztea.

Di.BB.A7. Trazabilitate matrizea osatzea.

- **Di.A** (Arkitektura) lan-paketea:

Di.A.A1. Goi-mailako arkitektura-diagrama diseinatzea (frontenda, backenda, MCP geruza, LLMa eta datu-basea).

Di.A.A2. MCP zerbitzariaren interfazeak eta erantzukizunak definitzea.

Di.A.A3. REST APIaren *endpoint*ak zehaztea.

Di.A.A4. Backend eta MCP geruza arteko komunikazio protokoloa definitzea.

- **Di.DBD** (Datu-basearen diseinua) lan-paketea:

Di.DBD.A1. Sistemaren entitate nagusiak identifikatzea.

Di.DBD.A2. Datu-basearen entitate-erlazio diagrama diseinatzea.

Di.DBD.A3. Beharrezkoak diren murriztapenak zehaztea.

- **Di.UU** (UI/UX) lan-paketea:

Di.UU.A1. Orri nagusiaren *wireframe*a sortzea.

Di.UU.A2. Zuzendariarekin prototipoa berrikustea eta beharrezkoa bada eguneratzea.

Inplementazioa (I) adarra:

- **I.IK** (Ingurunearen konfigurazioa) lan-paketea:

I.IK.A1. Garapen inguruneak konfiguratzea.

I.IK.A2. Git errepositorioa sortu eta GitHub bidezko bertsio-kontrola konfiguratzea.

I.IK.A3. Kodea isolatuta exekutatzeko Docker ingurunea konfiguratzea.

- **I.F** (Frontend) lan-paketea:

I.F.A1. React aplikazioa sortzea.

I.F.A2. Autentifikazio pantaila sortzea.

I.F.A3. Konfigurazio aukerak gehitzea.

I.F.A4. Monaco kode editorea txertatzea.

I.F.A5. Laguntza txataren osagaia inplementatzea.

I.F.A6. Ikaslearen aurrerapena erakutsiko duen dashboarda inplementatzea.

- **I.B** (Backend) lan-paketea:

  I.B.A1. Express zerbitzaria sortzea.

  I.B.A2. Autentifikazio endpointak inplementatzea.

  I.B.A3. Ariketak kudeatzeko endpointak inplementatzea.

  I.B.A4. MCP bezeroa eta testuinguru orkestratzailea inplementatzea.

  I.B.A5. LLM ereduaren hornitzailearen APIarekin komunikazio-logika integratzea.

  I.B.A6. Datu-basearekin konexioa ezartzea.

- **I.MCP** lan-paketea:

  I.MCP.A1. Docker ingurunean kodea exekutatuko duen MCP zerbitzaria garatzea.

- **I.DB** (Datu-basea) lan-paketea:

  I.DB.A1. PostgreSQL instantzian diseinatutako entitate-erlazio diagrama inplementatzea.

  I.DB.A2. Hasierako ariketa multzoa eta proba-kasuak datu-basean kargatzea.

- **I.Pr** (Probak) lan-paketea:

  I.Pr.A1. Backendeko endpointen proba unitarioak idaztea.

  I.Pr.A2. MCP zerbitzarien eta orkestratzailearen arteko integrazio-probak burutzea.

Kudeaketa (K) adarra:

- **K.P** (Plangintza) lan-paketea:

  K.P.A1. Eskakizunak identifikatzea, hasierako erabakiak hartzea eta zuzendariarekin dudak ebaztea.

  K.P.A2. Proiektuaren garapena sustatzeko hasierako plangintza idaztea.

- **K.JK** (Jarraipen eta kontrola) lan-paketea:

  K.JK.A1. Jarraipen informazioa eskuratzea.

  K.JK.A2. Estimatutakoarekin informazioa alderatzea eta arriskuak identifikatzea.

  K.JK.A3. Baldintzak bermatzea proiektuak arrakasta izan dezan.

  K.JK.A4. Jarraipen eta kontrol txostena idaztea.

Lan Akademikoa (LA) adarra:

- **LA.M** (Memoria) lan-paketea:

  LA.M.A1. Proiektua garatu ahala memoria idaztea.

3.1 Taula: Proiektuko garapen mugarriak.

Ataza	Data
I.A.M	2026-06-21
I.A.De	2026-06-29 - 2026-07-15

LA.M.A2. Memoria idaztea amaitzen denean dokumentua guztiz berrikusi eta aurkitzen diren akatsak zuzentzea.

- **LA.De** (Defentsa) lan-paketea:

  LA.De.A1. Defentsarako diapositibak prestatzea.

  LA.De.A2. Demo bat prestatzea.

  LA.De.A3. Aurkezpena eta entseguak prestatzea.

  LA.De.A4. GrALa defendatzea.

3.2.2 Atazen garapen denboraldiak eta menpekotasunak

Proiektuan zehar egin beharko diren atazen ordena, ataza bakoitzak izango duen garapen denboraldiak eta atazen arteko menpekotasunak 3.2 Irudia eta 3.3 Irudiko Gantt diagrama zatituan ikusi daiteke.

Lan-adarrak kolore ezberdinez marraztuta daude. Honek asko laguntzen du ikusten proiektuaren garapenaren kudeaketa nola eramango den. Proiektua diseinu adarrarekin hasiko da (horiz identifikatuta). Diseinuaren fase erdian, informazio bilaketari ekingo zaio (berdez identifikatuta). Bi adar hauek antzeko amaiera denboraldia izango dute, eta amaitzear daudenean, inplementazio adarrari hasiera emango zaio (gorriz identifikatuta).

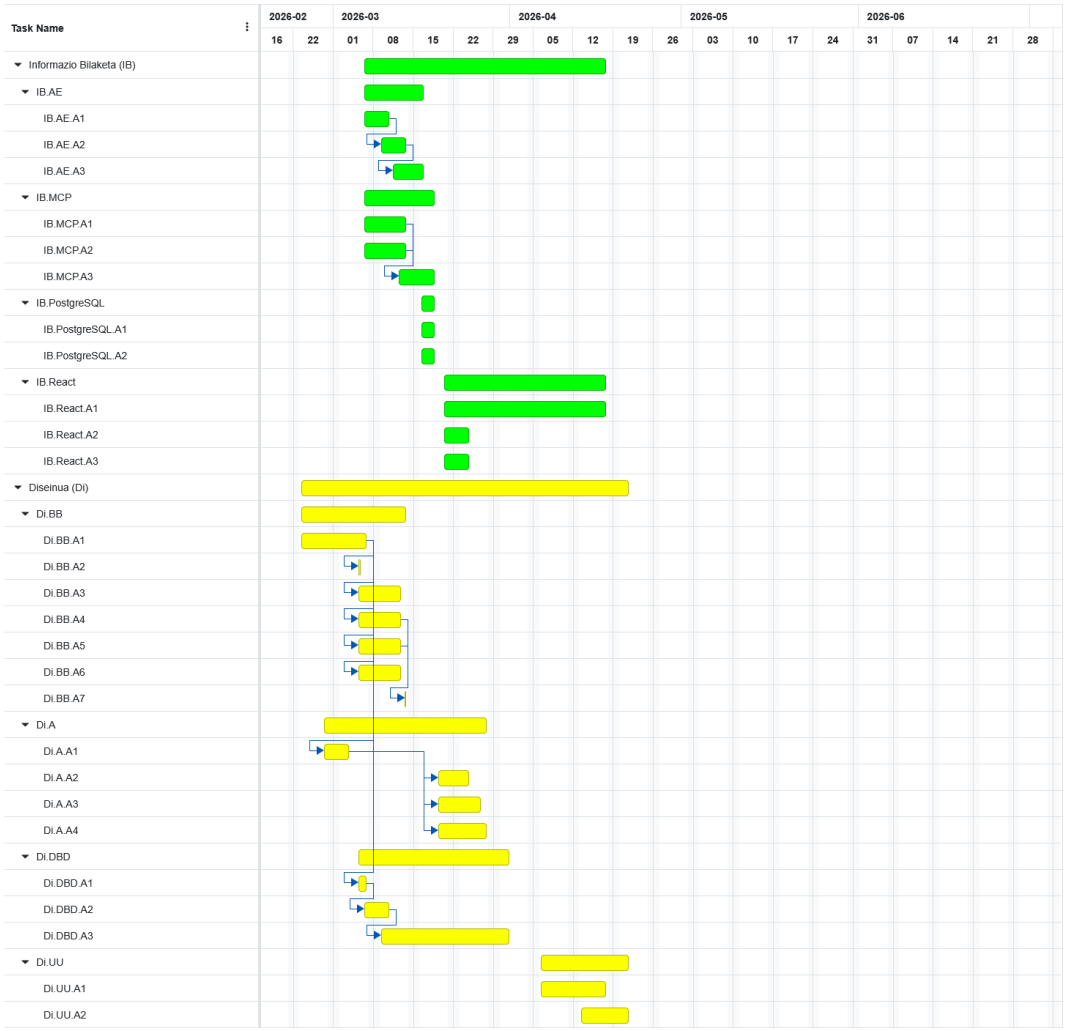
Inplementazio adarraren amaieraren eta GrALa Archivo Digital para la Docencia y la Investigación (ADDI) plataformara igotzeko egunaren artean tarte zabala aurreikusi da. Tarte horri esker, proiektuan zehar arazoak edo atzerapenak izanez gero, konponketa lanak egiteko malgutasuna bermatzen da. Azkenik, bai kudeaketa adarra (morez identifikatuta), bai lan akademikoaren adarra (urdinez identifikatuta), lauhileko osoan zehar modu jarraituan burutuko dira.

3.2.3 Proiektuaren garapen mugarriak

Jada esan dugu proiektuak hainbat emangarri sortuko dituela. Egilea eta zuzendaria soilik direnez pertsona interesatu bakarrak, emangarriek ez dute mugarri finko zehatzik. Hala ere, proiektuaren bideragarritasuna bermatzeko, bi mugarri kritiko ezarri dira: memoria ADDI plataformara igotzeko muga eta proiektuaren defentsa, ikusi 3.1 Taula. Aipatzeko da proiektuaren defentsaren data unibertsitateak finkatzen duela memoria entregatu ondoren.

3.3 Ataza bakoitzari emango zaion denboraren estimazioak

Proiektuan definitu den lan-ataza bakoitza, ikusi 3.1 Irudia, garatzeko beharko den denbora estimazioak 3.2 Taulan zehaztu dira, eta aurreko atalean azaldutako Gantt diagraman oinarritu dira.



3.2 Irudia: Proiektuaren Gantt diagrama (1/2).

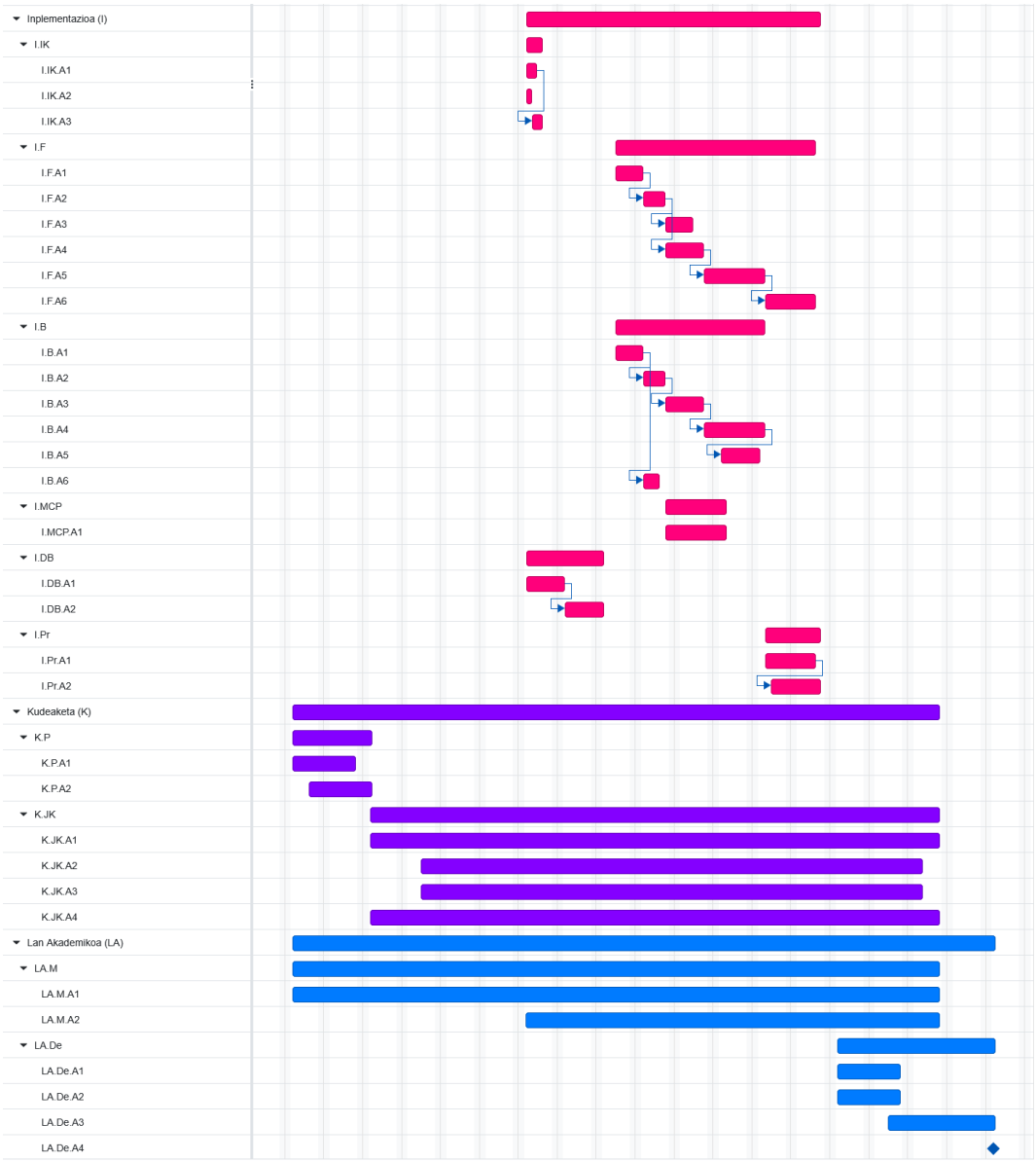
3.4 Informazio sistema eta komunikazio sistema

Proiektuaren garapenaren zehar orden ona mantentzeko ezinbestekoa da zehaztea informazioa nola gordeko den —*informazio sistema*— eta interesdunen arteko komunikazioa nola egingo den —*komunikazio sistema*—.

3.4.1 Informazio sistema

GrAL honen informazio sistema hodeiko 3 azpiegiturek eta tokiko biltegiratze batek osatzen dute. Sistema honen helburua dokumentazioaren eta kodearen bertsio kontrola eta segurtasuna bermatzea da.

Izendapena
: Proiektuko fitxategi guztien fitxategi-izenek dagokien lan-adarraren eta lan-paketearen identifikatzailea barneratuko dute. Gainera, lan-pakete guztiek dagokien lan-adarraren izena izango duen karpetan gordeko dira.



3.3 Irudia: Proiektuaren Gantt diagrama (2/2).

Azpiegiturak eta tresnak

Proiektua garatzeko erabiliko diren azpiegiturak honakoak dira:

- Microsoft OneDrive: eguneroko fitxategiak partekatutako karpeta batean kudeatuko dira.
- Overleaf (LaTeX): lanaren memoria garatzeko erabiliko da.
- GitHub: proiektuaren iturburu-kodea gordetzeko erabiliko da.
- Garatzailearen ordenagailua: hodeian dauden azpiegituren informazioaren segurtasun kopiak gordetzeko erabiliko da.

3.2 Taula: Proiektuko lan-atazen garapen denbora estimazioak.

Lan-paketea	Denbora (h)
Informazio Bilaketa (IB) adarra	40
IB.AE	10
IB.MCP	15
IB.PostgreSQL	5
IB.React	10
Diseinua (Di) adarra	38
Di.BB	15
Di.A	10
Di.DBD	8
Di.UU	5
Inplementazioa (I) adarra	150
I.IK	5
I.F	45
I.B	55
I.MCP	20
I.DB	10
I.Pr	15
Kudeaketa (K) adarra	30
K.P	15
K.JK	15
Lan Akademikoa (LA) adarra	70
LA.M	60
LA.De	10
Guztira	328

3.4.2 Komunikazio sistema

Proiektuaren komunikazio sistema hiru komunikazio kanal ezberdinek osatzen dute. Kanal guztien helburu nagusia GrALaren egilearen eta zuzendariaren harremana da:

Online bilerak
: Microsoft Teams plataformaren bidez burutuko dira, astero, ostiraletan 12:00etan. Bilerak ordu beteko gehienezko iraupena izango dute. Proiektuaren jarraipena egiteko erabiliko da.

Bilera presentzialak
: Presentzialtasuna beharrezkoa baldin bada zuzendariaren bulegoan bilerak egingo dira, eta online bileren ordutegi berbera izan dezakete.

Posta elektronikoa
Lauhilabetean zehar komunikazio asinkronoa egiteko erabiliko da.

3.5 Arriskuen analisia

Proiektuaren garapena egokia bermatu nahi dugu. Horretarako, atal honetan proiektuan zehar gerta daitezkeen arriskuak zerrendatuko eta analizatuko dira, ikusi 3.3 Taula.

Arriskuak zerrendatzeak helburu nagusi bat du: proiektuaren garapenean sor daitezkeen *oztopoak saihestea*. Honetarako, zerrendatutako arrisku bakoitzari prebentzio eta kontingentzia plana zehaztu zaio.

A1. Erabilera mugak
Baliteke erabiliko den LLMaren APIak arazoak ematea ezarritako erabilera mugekin edo zerbitzua guztiz erortzea.

Prebentzioa
Batetik, mugak saihesteko doako APIak edo lokalak erabiliko dira. Bestetik, zerbitzua bermatzeko sistemak hainbat zerbitzu ezberdin erabiltzeko prestatu beharko da.

Kontingentzia
Erabilera muga gaindituz gero edo zerbitzaria eroriz gero, beste API bat aurkitu beharko da.

A2. Teknologia berrien ikaskuntza
Proiektua garatu ahal izateko LLMarekin komunikazioak egin behar dira. Hau lortzeko MCP zerbitzariak eta bezeroak inplementatu beharko dira. Teknologia hau oso berria da eta pertsonalki ezezaguna. Honek ekarri dezake planifikazioan egindako denboren estimazioak oso okerrak izatea eta beste atazak asko atzeratzea.

Prebentzioa
Proiektuaren garapenaren martxa ez gelditzeko, MCP protokoloari lan-pakete propio bat emango zaio eta proiektuaren hasieratik protokoloaren funtzionamendua aztertzen hasiko da.

Kontingentzia
Esperotako denbora baino gehiago behar bada, beste atazak atzeratu beharko dira. Honek plangintza berrantolatzea ekar dezake eta beharrezkoa balitz betekizunak mugatzea, proiektua aurkeztu ahal izateko lanaren entregatze datan.

A3. Fitxategien eta iturburu-kodearen segurtasuna
Proiektuak lauhilabeteko iraupena izango du. Denbora luze samarra da proiektua garatzeko eta oso garrantzitsua da fitxategiak ez galtzea lanean ari garen bitartean. Informazioa galduz gero galdutako guztia berregin beharko da eta plangintza asko atzeratu dezake. Okerrenean, proiektu osoa galdu dezakegu.

Prebentzioa
Fitxategiak guztiz ez galtzeko proiektua garatu ahala segurtasun kopiak egingo dira, bai lokalean, bai hodeian. Gainera, Git erabiliko da iturburu-kodearen historia aurreratua izateko.

3.3 Taula: Proiektuan zehar gerta daitezkeen arriskuen taula.

Kodea	Arriskua	Probabilitatea	Eragina
A1	Erabilera mugak	Txikia	Txikia
A2	Teknologia berrien ikaskuntza	Altua	Ertaina
A3	Fitxategien eta iturburu-kodearen segurtasuna	Txikia	Handia
A4	Gaixotasuna	Txikia	Ertaina

Kontingentzia

Fitxategiak galduz gero azkeneko segurtasun kopia berreskuratu beharko da eta falta zaizkion aldaketak berriro eskuz inplementatu. Ordea, informazio galera partziala izan bada, Git erabili dezakegu aurreko *commit* bat berreskuratzeko.

A4. Gaixotasuna

Aurreko arriskuan azaldu den bezala, proiektuak lauhilabeteko iraupena izango du eta gerta daiteke osasun arazoak izatea. Gaixotuz gero, atazak egiteko denboraldiak atzeratu egin beharko dira.

Prebentzioa

Arrisku hau ezin da guztiz prebenitu eta aukera bakarra osasuna zaintzea izango da.

Kontingentzia

Plangintzako denboraldiak zehazterakoan erabili gabeko tarte bat utziko da garapen mugarriak baino lehenago, gaixotasunarengatik sortutako edozein atzerapen berrantolatu ahal izateko.

KAPITULUA 4

Sistemaren betekizunen katalogoa

Kapitulu honetan sistemak bete behar dituen betekizunak biltzen dira. Hauen artean informazio betekizunak, betekizun funtzionalak eta betekizun ez-funtzionalak azaltzen dira. Betekizun funtzionalen kasuan, erabilera-kasuen diagramak, aktoreen definizioak eta sekuentzia-diagramak ere azaltzen dira. Amaitzeko, informazio betekizunak eta betekizun funtzionalak proiektuaren helburu zehatzekin erlazionatzen dituen trazabilitate matrize bat aurkezten da.

4.1 Informazio betekizunak

Atal honek informazioa ongi tratatu ahal izateko beharrezkoak diren informazio betekizunak biltzen ditu.

IB-1. Ikasleen informazioa
Ikasleen datu pertsonalak. Gorde beharreko datu zehatzak:

- Izena
- Ikasketa maila

IB-2. Kontuen informazioa
Erabiltzaileen kontuei dagokien informazioa. Gorde beharreko datu zehatzak:

- Helbide elektronikoa
- Pasahitzaren *hash*a
- Sortze data
- Saioa
- Gogoko programazio-lengoaia

IB-3. Ariketen informazioa
Plataformako ariketei dagokien informazioa. Gorde beharreko datu zehatzak:

- Izenburua

- Deskribapena
- Testak
- Metadatuak

IB-4. Saiakeren informazioa
Erabiltzaileek ariketen saiakeren gaineko informazioa. Gorde beharreko datu zehatzak:

- Ebazpena
- Erabiltzailearen elkarrizketak
- Nota
- Exekuzio emaitza
- Metadatuak

4.2 Betekizun funtzionalak

Atal honek erabiltzaileek sistemarekin elkarrekintza egin ahal izateko beharrezkoak diren betekizun funtzionalak biltzen ditu. Betekizun funtzionalek sistemaren portaera definitzen dute.

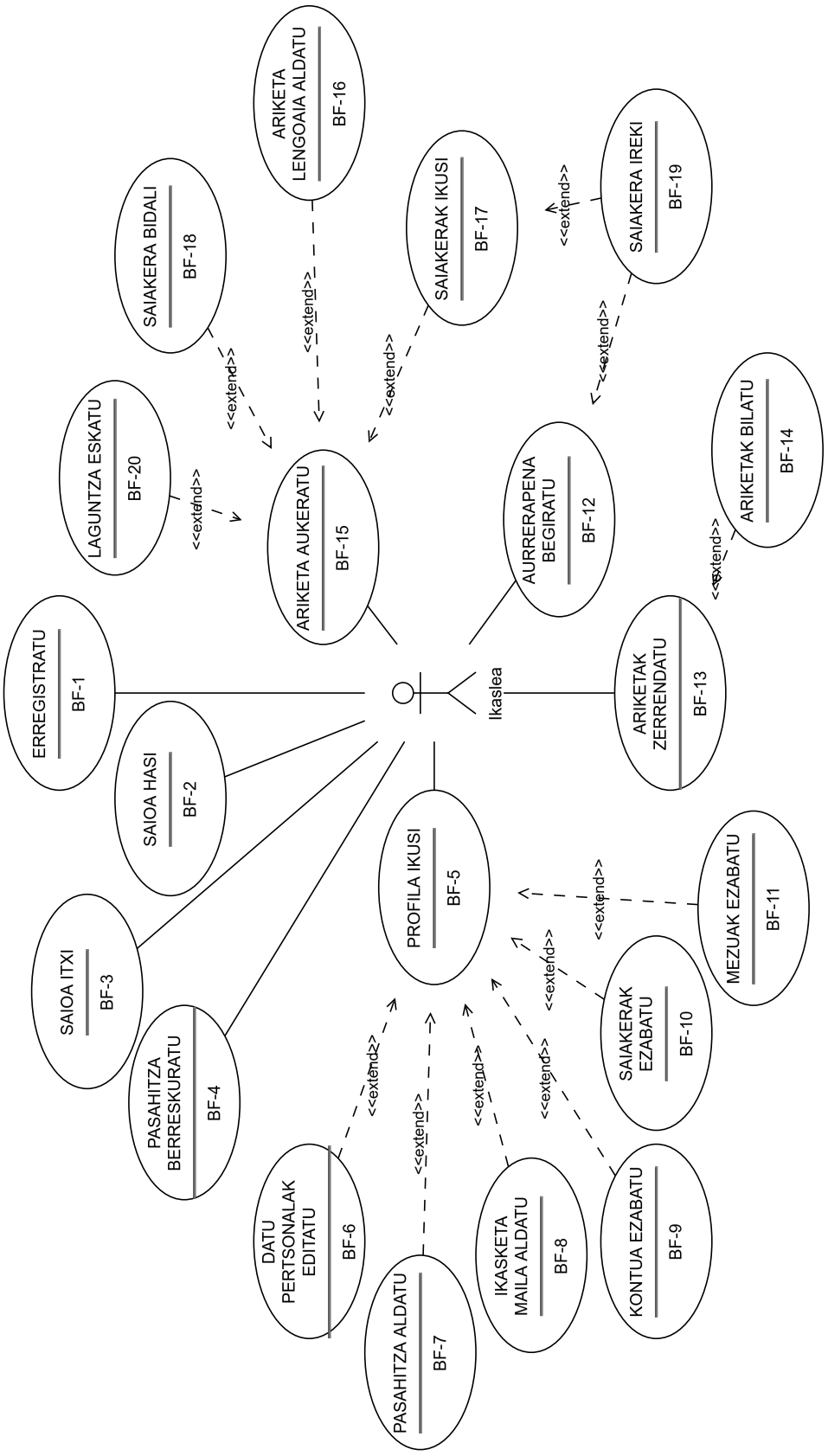
4.2.1 Aktoreen definizioa

Gure sistemak aktore bakarra izango du:

Ikaslea
Aktore honek plataforma ikaskuntzarako erabili nahi duen edozein erabiltzaile adierazten du.

4.2.2 Erabilera-kasuen diagramak

Aplikazioaren betekizun funtzionalak erabilera-kasuen diagraman biltzen dira, ikusi 4.1 Irudia. Bertan aktore bakarra dugu eta erabilera-kasu guztiak aktore bakarrarekin lotuta daude. Irakurketarekin laguntzeko erabilera-kasu bakoitzaren barruan, betekizun funtzionalaren izenaz gain, betekizun funtzionalaren identifikatzailea idatzita dago erdiko marraren azpian.



4.1 Irudia: Aplikazioaren erabilera-kasuen diagrama.

4.2.3 Sistemaren erabilera kasuak

Erabilera-kasuen diagraman bildutako betekizun funtzionalak hurrengo tauletan azaltzen dira, ikusi 4.1 Taulatik 4.20 Taulara.

4.1 Taula: Erregistratzea barneratzen duen BF-1 betekizuna.

Identifikatzailea	**BF-1**
Izena	Erregistratu
Aktoreak	Ikaslea
Deskribapena	Plataforma erabili nahi duen pertsona batek kontu bat sortzen du.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-2, BF-3
Aurrebaldintza	Erabiltzaileak ez du saiorik hasita.
Exekuzio normala	1. Erabiltzaileak erregistratu botoia sakatzen du. 2. Sistemak erregistro txantiloia itzultzen dio erabiltzaileari. 3. Erabiltzaileak txantiloia betetzen du (izena, helbide elektronikoa, pasahitza, pasahitza errepikatu, ikasketa maila) eta sistemari bidaltzen dio. 4. Sistemak balioak egiaztatzen ditu. 5. Sistemak kontu berria sortzen du datu-basean. 6. Sistemak baieztapen-kode bat sortzen du eta erabiltzailearen helbide elektronikora bidaltzen du. 7. Sistemak baieztapen-orria erabiltzaileari itzultzen dio. 8. Erabiltzaileak baieztapen-kodea sartzen du. 9. Sistemak baieztapen-kodea egiaztatzen du. 10. Sistemak erabiltzailearen kontua aktibatzen du. 11. Sistemak erabiltzaileari saioa hasten dio.
Postbaldintza	Kontu berria sortu da eta erabiltzaileak saioa hasita du.

Hurrengo orrian jarraitzen du

4.1 Taula: Erregistratzea barneratzen duen BF-1 betekizuna. (Jarraitua)

Salbuespenak	4a. Jada helbide elektroniko hori duen erabiltzaile bat existitzen da. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 3. pausora itzuli. 4b. Zehaztutako pasahitza ahulegia da. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 3. pausora itzuli. 9a. Baieztapen-kodea ez da egokia edo iraungi da. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 8. pausora itzuli. 9b. Erabiltzaileak baieztapen-kode berria eskatzen du. 1. Itxarote denbora igaro bada sistemak baieztapen-kode berri bat sortzen du eta erabiltzailearen helbide elektronikora bidaltzen du. 2. Exekuzioa 8. pausora itzuli.

4.2 Taula: Saioa hastea barneratzen duen BF-2 betekizuna.

Identifikatzailea	**BF-2**
Izena	Saioa hasi
Aktoreak	Ikaslea
Deskribapena	Plataformara sartu ahal izateko erabiltzaileak saioa hasten du.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-1, BF-3
Aurrebaldintza	Erabiltzaileak ez du saiorik hasita.
Exekuzio normala	1. Sistemak saioa hasteko txantiloia erabiltzaileari bidaltzen dio. 2. Erabiltzaileak txantiloia betetzen du (helbide elektronikoa, pasahitza) eta sistemari bidaltzen dio. 3. Sistemak datuak egiaztatzen ditu. 4. Sistemak JWT bat sortzen du eta erabiltzaileari bidaltzen dio. 5. Erabiltzaileak tokena *cookie* gisa gordetzen du. 6. Erabiltzailea orri nagusira mugitzen da.
Postbaldintza	Erabiltzaileak saioa hasita du.

Hurrengo orrian jarraitzen du

4.2 Taula: Saioa hastea barneratzen duen BF-2 betekizuna. (Jarraitua)

Salbuespenak	3a. Sartutako datuak ez dira egokiak. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 2. pausora itzuli. 3b. Kontua ez dago aktibatuta. 1. Sistemak baieztapen-kode bat sortzen du eta erabiltzailearen helbide elektronikora bidaltzen du. 2. Sistemak baieztapen-orria erabiltzaileari itzultzen dio. 3. Erabiltzaileak baieztapen-kodea sartzen du. 4. Sistemak baieztapen-kodea egiaztatzen du. 4.a. Baieztapen-kodea ez da egokia edo iraungi da. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 3b. 3. pausora itzuli. 4.b. Erabiltzaileak baieztapen-kode berria eskatzen du. 1. Itxarote denbora igaro bada sistemak baieztapen-kode berri bat sortzen du eta erabiltzailearen helbide elektronikora bidaltzen du. 2. Exekuzioa 3b. 3. pausora itzuli. 5. Sistemak erabiltzailearen kontua aktibatzen du. 6. Exekuzioa 4 pausora itzuli.

4.3 Taula: Saioa ixtea barneratzen duen BF-3 betekizuna.

Identifikatzailea	**BF-3**
Izena	Saio itxi
Aktoreak	Ikaslea
Deskribapena	Plataformatik ateratzeko erabiltzaileak saioa ixten du.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-1, BF-2
Aurrebaldintza	Erabiltzaileak saioa hasita du.
Exekuzio normala	1. Erabiltzaileak saioa ixteko botoia sakatzen du. 2. Sistemak JWT access tokena ezabatzen du. 3. Sistemak *refresh* tokena iraungitzen du.
Postbaldintza	Erabiltzaileak ez du hasitako saiorik.

Hurrengo orrian jarraitzen du

4.3 Taula: Saioa ixtea barneratzen duen BF-3 betekizuna. (Jarraitua)

Salbuespenak	—

4.4 Taula: Pasahitza berreskuratzea barneratzen duen BF-4 betekizuna.

Identifikatzailea	**BF-4**
Izena	Pasahitza berreskuratu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak pasahitza ahaztu du eta posta elektronikoaren bitartez berreskuratzen du.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-2
Aurrebaldintza	Erabiltzaileak ez du saiorik hasita.
Exekuzio normala	1. Erabiltzaileak pasahitza berreskuratzeko botoia sakatzen du. 2. Sistemak pasahitza berreskuratzeko txantiloia bidaltzen dio erabiltzaileari. 3. Erabiltzaileak txantiloia (helbide elektronikoa) betetzen du eta sistemari bidaltzen dio. 4. Sistemak helbide elektroniko hori duen kontua existitzen den egiaztatzen du eta eskaera noiz egin den gordetzen du. 5. Sistemak pasahitza berreskuratzeko esteka bidaliko dela esaten duen mezua bidaltzen dio erabiltzaileari. 6. Sistemak pasahitza aldatzeko esteka bat bidaltzen dio erabiltzailearen helbide elektronikora eta iraungitze-data bat ezartzen dio. 7. Erabiltzaileak esteka irekitzen du, pasahitza berria zehazten du (pasahitza eta errepikatu pasahitza) eta sistemari bidaltzen dio. 8. Sistemak pasahitza egiaztatzen du. 9. Sistemak kontuaren pasahitza eguneratzen du. 10. Sistemak kontuaren refresh tokenak ezabatzen ditu.
Postbaldintza	Erabiltzailearen pasahitza aldatu da eta erabiltzaileak irekitako saioak amaitu dira.

Hurrengo orrian jarraitzen du

4.4 Taula: Pasahitza berreskuratzea barneratzen duen BF-4 betekizuna. (Jarraitua)

Salbuespenak	4a. Jasotako helbide elektronikoa duen konturik ez da existitzen. 1. Sistemak pasahitza berreskuratzeko esteka bidaliko dela esaten duen mezua bidaltzen dio erabiltzaileari (ez du zehazten kontua ez dela existitzen segurtasunagatik). 4b. Berreskuratze saiakera bat egin da eta ez da nahiko denbora igaro beste bat egiteko. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 3. pausora itzuli. 7a. Pasahitza aldatzeko esteka iraungi da. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 1. pausora itzuli. 8a. Zehaztutako pasahitza ahulegia da. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 7. pausora itzuli.

4.5 Taula: Profila ikustea barneratzen duen BF-5 betekizuna.

Identifikatzailea	**BF-5**
Izena	Profila ikusi
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak bere kontuaren profila ikusten du.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-6, BF-7, BF-8, BF-9, BF-10, BF-11
Aurrebaldintza	Erabiltzaileak saioa hasita du.
Exekuzio normala	1. Erabiltzaileak profila irekitzeko botoia sakatzen du. 2. Sistemak erabiltzailearen informazio eskuratzen du datu-basetik eta profila bidaltzen dio erabiltzaileari.
Postbaldintza	Erabiltzaileak bere profila ikusgai du.
Salbuespenak	—

4.6 Taula: Datu pertsonalek aldaketa barneratzen duen BF-6 betekizuna.

Identifikatzailea	**BF-6**

Hurrengo orrian jarraitzen du

4.6 Taula: Datu pertsonalek aldaketa barneratzen duen BF-6 betekizuna. (Jarraitua)

Izena	Datu pertsonalak editatu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak bere profilean datu pertsonalak eguneratzen ditu.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-5
Aurrebaldintza	Erabiltzaileak saio hasita du eta bere profila ikusgai du.
Exekuzio normala	1. Erabiltzaileak nahi dituen datu pertsonalak (helbide elektronikoa ezik) aldatzen ditu eta eguneratzeko botoia sakatzen du. 2. Sistemak balioak egiaztatzen ditu. 3. Sistemak erabiltzailearen aldaketak datu-basean eguneratzen ditu. 4. Sistemak erabiltzaileari aldaketak egin direla komunikatzen dio.
Postbaldintza	Erabiltzaileak datu pertsonalak eguneratu ditu.
Salbuespenak	2a. Jasotako balio bat ez da egokia. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 1. pausora itzuli.

4.7 Taula: Pasahitza aldatzea barneratzen duen BF-7 betekizuna.

Identifikatzailea	**BF-7**
Izena	Pasahitza aldatu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak profiletik pasahitza aldatzen du pasahitza berria eta zaharra adieraziz.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-5
Aurrebaldintza	Erabiltzaileak saio hasita du eta bere profila ikusgai du.

Hurrengo orrian jarraitzen du

4.7 Taula: Pasahitza aldatzea barneratzen duen BF-7 betekizuna. (Jarraitua)

Exekuzio normala	1. Erabiltzaileak pasahitza zaharra, pasahitza berria eta errepikatu pasahitza berria idazten ditu eta eguneratzeko botoia sakatzen du. 2. Sistemak balioak egiaztatzen ditu. 3. Sistemak datu-basean kontuaren pasahitza eguneratzen du. 4. Sistemak kontuaren refresh tokenak eta access tokena ezabatzen ditu.
Postbaldintza	Erabiltzailearen pasahitza aldatu da eta erabiltzaileak irekitako saioak amaitu dira.
Salbuespenak	2a. Zehaztutako pasahitza ahulegia da. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 1. pausora itzuli.

4.8 Taula: Ikasketa maila aldatzea barneratzen duen BF-8 betekizuna.

Identifikatzailea	**BF-8**
Izena	Ikasketa maila aldatu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak profiletik ikasketa maila aldatzen du.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-5
Aurrebaldintza	Erabiltzaileak saioa hasita du eta bere profila ikusgai du.
Exekuzio normala	1. Erabiltzaileak bere ikasketa mailarekin lotutako ezarpenak aldatzen ditu eta eguneratzeko botoia sakatzen du. 2. Sistemak balioak egiaztatzen ditu. 3. Sistemak datu-basean erabiltzaileak aldatutako balioak eguneratzen ditu.
Postbaldintza	Erabiltzailearen ikasketa mailarekin lotutako ezarpenak eguneratuta daude.
Salbuespenak	2a. Jasotako balio bat ez da egokia. 1. Sistemak errore-mezua bidaltzen dio erabiltzaileari. 2. Exekuzioa 1. pausora itzuli.

4.9 Taula: Kontua ezabatzea barneratzen duen BF-9 betekizuna.

Identifikatzailea	**BF-9**
Izena	Kontua ezabatu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak profiletik bere aplikazioko kontua betirako ezabatzen du.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-5
Aurrebaldintza	Erabiltzaileak saioa hasita du eta bere profila ikusgai du.
Exekuzio normala	1. Erabiltzaileak kontua ezabatzeko botoia sakatzen du. 2. Sistemak benetan berretsi nahi duen esaten duen mezua bidaltzen dio erabiltzaileari. 3. Erabiltzaileak berresteko botoia sakatzen du. 4. Sistemak erabiltzailearen kontuarekin erlazionatutako datu-baseko taula guztiak ezabatzen ditu. 5. Sistemak kontua ezabatu dela esaten duen mezua bidaltzen dio erabiltzaileari.
Postbaldintza	Erabiltzailearen kontua guztiz ezabatu da.
Salbuespenak	3a. Erabiltzaileak ez du berresten. 1. Erabiltzaileak kontua ezabatzea ezeztatzen du.

4.10 Taula: Saiakerak ezabatzea barneratzen duen BF-10 betekizuna.

Identifikatzailea	**BF-10**
Izena	Saiakerak ezabatu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak profiletik aurreko saiakera guztiak ezabatzen ditu.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-5
Aurrebaldintza	Erabiltzaileak saioa hasita du eta bere profila ikusgai du.

Hurrengo orrian jarraitzen du

4.10 Taula: Saiakerak ezabatzea barneratzen duen BF-10 betekizuna. (Jarraitua)

Exekuzio normala	1. Erabiltzaileak saiakerak ezabatzeko botoia sakatzen du. 2. Sistemak benetan berretsi nahi duen esaten duen mezua bidaltzen dio erabiltzaileari. 3. Erabiltzaileak berresteko botoia sakatzen du. 4. Sistemak erabiltzailearen kontuan gordetako ariketa guztien saiakerak eta elkarrizketak datu-basetik ezabatuko ditu. 5. Sistemak saiakerak ezabatu direla esaten duen mezua bidaltzen dio erabiltzaileari.
Postbaldintza	Erabiltzaileak egindako ariketa guztien saiakerak eta elkarrizketak ezabatu dira.
Salbuespenak	3a. Erabiltzaileak ez du berresten. 1. Erabiltzaileak saiakeren ezabatzea ezeztatzen du.

4.11 Taula: Mezuak ezabatzea barneratzen duen BF-11 betekizuna.

Identifikatzailea	**BF-11**
Izena	Mezuak ezabatu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak profiletik AArekin egindako elkarrizketa guztiak ezabatzen ditu.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-5
Aurrebaldintza	Erabiltzaileak saioa hasita du eta bere profila ikusgai du.
Exekuzio normala	1. Erabiltzaileak elkarrizketak ezabatzeko botoia sakatzen du. 2. Sistemak benetan berretsi nahi duen esaten duen mezua bidaltzen dio erabiltzaileari. 3. Erabiltzaileak berresteko botoia sakatzen du. 4. Sistemak erabiltzailearen kontuko elkarrizketa guztiak datu-basetik ezabatuko ditu. 5. Sistemak elkarrizketak ezabatu direla esaten duen mezua bidaltzen dio erabiltzaileari.
Postbaldintza	Erabiltzaileak izandako elkarrizketak ezabatu dira.
Salbuespenak	3a. Erabiltzaileak ez du berresten. 1. Erabiltzaileak saiakeren ezabatzea ezeztatzen du.

4.12 Taula: Aurrerapena begiratzea barneratzen duen BF-12 betekizuna.

Identifikatzailea	**BF-12**
Izena	Aurrerapena begiratu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak aurrerapen dashboarda ireki dezake ariketen batez besteko emaitzak ikusteko.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-17
Aurrebaldintza	Erabiltzaileak saioa hasita du.
Exekuzio normala	1. Erabiltzaileak dashboarda irekitzeko botoia sakatzen du. 2. Sistemak erabiltzailearen informazioa eskuratzen du datu-basetik eta aurrerapena bidaltzen dio erabiltzaileari.
Postbaldintza	Erabiltzaileak bere aurrerapena ikusgai du.
Salbuespenak	—

4.13 Taula: Ariketak zerrendatzea barneratzen duen BF-13 betekizuna.

Identifikatzailea	**BF-13**
Izena	Ariketak zerrendatu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak ariketen zerrenda ireki dezake egiteko ariketa bat hautatzeko.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-14, BF-15
Aurrebaldintza	Erabiltzaileak saioa hasita du.
Exekuzio normala	1. Erabiltzaileak ariketen zerrenda irekitzeko botoia sakatzen du. 2. Sistemak erabiltzailearen ikasketa maila informazioa eskuratzen du datu-basetik. 3. Sistemak erabiltzailearen ikasketa mailarekin datozen ariketak eskuratzen ditu datu-basetik eta zerrenda bidaltzen dio erabiltzaileari.
Postbaldintza	Erabiltzaileak ariketa-zerrenda ikusgai du.

Hurrengo orrian jarraitzen du

4.13 Taula: Ariketak zerrendatzea barneratzen duen BF-13 betekizuna. (Jarraitua)

Salbuespenak	—

4.14 Taula: Ariketak bilatzea barneratzen duen BF-14 betekizuna.

Identifikatzailea	**BF-14**
Izena	Ariketak bilatu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak ariketen zerrendan iragazki bat aplikatu dezake bilaketa zehatz bat egiteko.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-13, BF-15
Aurrebaldintza	Erabiltzaileak saioa hasita du eta ariketen zerrenda ikusgai du.
Exekuzio normala	1. Erabiltzaileak ariketak iragazteko eskura dauden aukeren artean (titulu bilatzailea, egoera, zailtasuna, lengoaia, etiketa eta kategoria) klik egiten du. 2. Sistemak iragazkiak egiaztatzen ditu. 3. Sistemak datu-basetik ariketak eskuratzen ditu ezarritako iragazkiak erabiliz eta zerrenda bidaltzen dio erabiltzaileari.
Postbaldintza	Erabiltzaileak iragazitako ariketa-zerrenda ikusgai du.
Salbuespenak	—

4.15 Taula: Ariketa aukeratzea barneratzen duen BF-15 betekizuna.

Identifikatzailea	**BF-15**
Izena	Ariketa aukeratu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak ariketen zerrendatik ariketa bat hautatzen du eta ariketa egin ahal izateko tresnak agertzen dira.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-13, BF-14, BF-16, BF-17, BF-18, BF-19, BF-20

Hurrengo orrian jarraitzen du

4.15 Taula: Ariketa aukeratzea barneratzen duen BF-15 betekizuna. (Jarraitua)

Aurrebaldintza	Erabiltzaileak saioa hasita du eta ariketen zerrenda ikusgai du.
Exekuzio normala	1. Erabiltzaileak ariketa-zerrendako ariketa batean klik egiten du. 2. Sistemak erabiltzailearen gogoko programazio hizkuntza datu-basetik eskuratzen du. 3. Sistemak eskatutako ariketa existitzen den egiaztatzen du. 4. Sistemak ebazpen huts bat sortzen dio ikasleari, jada ez badu. 5. Sistemak erabiltzaileak aukeratutako ariketaren informazioa (izenburua, enuntziatua, zailtasun maila, etiketak) eskuratzen du. 6. Sistemak erabiltzailearen gogoko programazio hizkuntzan ariketaren informazio zehatza (programazio-lengoaia, hasierako kodea) eskuratzen du. 7. Sistemak ariketa erabiltzaileari bidaltzen dio.
Postbaldintza	Erabiltzaileak hautatutako ariketa ikusgai du.
Salbuespenak	6a. Erabiltzaileak ez du gogoko lengoaiarik edo hautatutako ariketa ez dago lengoaia horretan aukeragarri. 1. Sistemak datu-basean agertzen den lehenengo programazio-lengoaian ariketaren informazio zehatza eskuratzen du. 2. Exekuzioa 7. pausora itzuli.

4.16 Taula: Ariketaren lengoaia aldatzea barneratzen duen BF-16 betekizuna.

Identifikatzailea	**BF-16**
Izena	Ariketa lengoaia aldatu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak ariketa bat egiteko hautatu duenean, ariketa egiteko programazio-lengoaia aldatu dezake.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-15
Aurrebaldintza	Erabiltzaileak saioa hasita du eta ariketa bat ikusgai du.

Hurrengo orrian jarraitzen du

4.16 Taula: Ariketaren lengoaia aldatzea barneratzen duen BF-16 betekizuna. (Jarraitua)

Exekuzio normala	1. Erabiltzaileak ariketaren lengoaia aldatzeko botoian klik egiten du eta zerrendako lengoaia bati klik egiten dio. 2. Sistemak eskatutako ariketa existitzen den egiaztatzen du. 3. Sistemak ebazpen huts bat sortzen dio ikasleari, jada ez badu. 4. Sistemak erabiltzaileak hautatutako lengoaiarekin ariketaren informazio zehatz berria (hasierako kodea, etab.) eskuratzen du. 5. Sistemak ariketa erabiltzaileari bidaltzen dio.
Postbaldintza	Erabiltzaileak ariketa lengoaia berrian ikusgai du.
Salbuespenak	—

4.17 Taula: Saiakerak ikustea barneratzen duen BF-17 betekizuna.

Identifikatzailea	**BF-17**
Izena	Saiakerak ikusi
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak egindako ariketen aurreko saiakerak ikusi ditzake.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-12, BF-15, BF-19
Aurrebaldintza	Erabiltzaileak saioa hasita du eta ariketa bat ikusgai du.
Exekuzio normala	1. Erabiltzaileak aurreko saiakerak ikusteko botoia sakatzen du. 2. Sistemak erabiltzaileak aukeratuta duen ariketa zehatzean egindako saiakeren informazioa eskuratzen du. 3. Sistemak saiakeren zerrenda bidaltzen dio erabiltzaileari.
Postbaldintza	Erabiltzaileak ariketa zehatz horretan egin dituen aurreko saiakeren informazioa ikusgai dago.
Salbuespenak	—

4.18 Taula: Saiakera bidaltzea barneratzen duen BF-18 betekizuna.

Identifikatzailea	**BF-18**

Hurrengo orrian jarraitzen du

4.18 Taula: Saiakera bidaltzea barneratzen duen BF-18 betekizuna. (Jarraitua)

Izena	Saiakera bidali
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak irekitako ariketa batean kodea idatziko du eta zuzena den jakiteko zerbitzarira bidaliko du.
Lotutako helburuak	HZ1, HZ3
Lotutako betekizunak	BF-15
Aurrebaldintza	Erabiltzaileak saioa hasita du eta ariketa bat ikusgai du.
Exekuzio normala	1. Erabiltzaileak kodea bidaltzeko botoia sakatzen du. 2. Sistemak kodea kode exekuzio MCP zerbitzarira bidaltzen du. 3. Docker ingurunean ariketaren test guztiak exekutatzen dira. 4. Sistemak emaitzak jasotzen ditu eta saiakera datu-basean gordetzen da. 5. Sistemak emaitzak bidaltzen dizkio erabiltzaileari.
Postbaldintza	Erabiltzaileak idatzitako kodea bidali da, testak exekutatu dira, saiakera gorde da eta frontendean exekuzioaren emaitza bistaratu da.
Salbuespenak	—

4.19 Taula: Saiakera irekitzea barneratzen duen BF-19 betekizuna.

Identifikatzailea	**BF-19**
Izena	Saiakera ireki
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak saiakeren zerrendatik saiakera bat hautatuko du eta kode editorean saiakeran bidaliko iturburu-kodea irekiko da.
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-15, BF-17
Aurrebaldintza	Erabiltzaileak saioa hasita du, ariketa bat ikusgai du eta saiakeren zerrenda irekita du, edo saioa hasita du eta aurrerapen dashboardean dago.

Hurrengo orrian jarraitzen du

4.19 Taula: Saiakera irekitzea barneratzen duen BF-19 betekizuna. (Jarraitua)

Exekuzio normala	1. Erabiltzaileak saiakera batean klik egiten du. 2. Sistemak erabiltzaileak kode editorean daukan kodea gordetzen du datu-basean. 3. Sistemak erabiltzaileak hautatutako saiakeraren iturburu-kodea eskuratzen du eta erabiltzaileari bidaltzen dio.
Postbaldintza	Erabiltzaileak hautatutako saiakeraren iturburu-kodea ikusgai du.
Salbuespenak	2a. Erabiltzaileak aurrerapen orritik egin du klik saiakeran. 1. Sistemak ariketa ikusteko pantaila irekitzen dio erabiltzaileari. 2. Sistemak erabiltzaileak hautatutako saiakerari dagokion ariketaren informazioa eskuratzen du datu-basetik. 3. Sistemak erabiltzaileak hautatutako saiakeran iturburu-kodea eskuratzen du. 4. Sistemak ariketaren eta saiakeraren informazioarekin betetzen du pantaila.

4.20 Taula: Laguntza eskatzea barneratzen duen BF-20 betekizuna.

Identifikatzailea	**BF-20**
Izena	Laguntza eskatu
Aktoreak	Ikaslea
Deskribapena	Erabiltzaileak ariketa bat ireki ondoren laguntza eskatzen dio AAri txata erabiliz.
Lotutako helburuak	HZ1, HZ2, HZ3, HZ4
Lotutako betekizunak	BF-15
Aurrebaldintza	Erabiltzaileak saioa hasita du eta ariketa bat ikusgai du.

Hurrengo orrian jarraitzen du

4.20 Taula: Laguntza eskatzea barneratzen duen BF-20 betekizuna. (Jarraitua)

Exekuzio normala	1. Erabiltzaileak txatean AArentzako mezua idazten du. 2. Erabiltzaileak mezua bidaltzen dio sistemari. 3. Sistemak testuingurua eraikitzen du system promptarekin, ariketaren enuntziatuarekin, erabiltzailearen iturburu-kodearekin, exekuzio emaitzekin, aurreko elkarrizketekin eta erabilgarri dauden tresnekin. 4. Sistemak testuingurua AAri bidaltzen dio APIa erabiliz. 5. AAk erantzuna bidaltzen dio sistemari. 6. AAk tresna deia eskatu badu, tresna exekutatu eta erantzuna berrelikatu. 7. Sistemak ikaslearen eta AAren erantzunak gordetzen ditu datu-basean eta erabiltzaileari bidaltzen dizkio.
Postbaldintza	Erabiltzaileak AAren erantzuna txatean ikusgai du eta elkarrizketa berria datu-basean gorde da.
Salbuespenak	5a. APIa ez dago eskuragarri. 1. Sistemak errore mezua bidaltzen dio erabiltzaileari.

4.2.4 Sistemaren sekuentzia-diagramak

Sistemaren funtzionalitate orokorrak eta osagai bakoitzak besteekin elkar lan nola egiten duten erakusten duten sekuentzia-diagramak A Eranskinan ikusi daitezke.

4.3 Betekizun ez-funtzionalak

Sistemak bete beharko dituen betekizun ez-funtzionalak hurrengo tauletan azaltzen dira, ikusi 4.21 Taulatik 4.25 Taulara. Betekizun hauek ez dira beharrezkoak sistemak funtzionatu dezan, baina betetzen badira sistemari onura egingo dio.

4.21 Taula: Sistemaren segurtasuna barneratzen duen BEF-1 betekizuna.

Identifikatzailea	**BEF-1**
Izena	Segurtasuna
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-1, BF-2, BF-4, BF-7, BF-9, BF-18

Hurrengo orrian jarraitzen du

4.21 Taula: Sistemaren segurtasuna barneratzen duen BEF-1 betekizuna. (Jarraitua)

Deskribapena	Kontuak babesteko saio hasteko saiakera kopurua mugatu beharko da, baita ere pasahitza berreskuratzeko saiakerak. Erabiltzailearen pasahitza ez da inoiz gordeko datu-basean, honen hasha gordeko da. Erabiltzaileak erabiliko duen pasahitza ahula ez dela ziurtatu behar da. Erabiltzaileak bidaliko iturburu-kodea Docker ingurunean exekutatu beharko da, ez ostalarian.
Garrantzia	Ezinbestekoa

4.22 Taula: Sistemaren eskalagarritasuna barneratzen duen BEF-2 betekizuna.

Identifikatzailea	**BEF-2**
Izena	Eskalagarritasuna
Lotutako helburuak	HZ2
Lotutako betekizunak	—
Deskribapena	Arkitekturak sistema handitzeko aukera erraztu behar du. Adibidez, ikasleak ez diren erabiltzaile mota bat nahi baldin badugu, erraz inplementatzeko aukera egon behar da.
Garrantzia	Altua

4.23 Taula: Sistemaren mantengarritasuna barneratzen duen BEF-3 betekizuna.

Identifikatzailea	**BEF-3**
Izena	Mantengarritasuna
Lotutako helburuak	HZ2, HZ3
Lotutako betekizunak	—
Deskribapena	Hainbat mailetan banatuta dagoen arkitekturak mantentze-lana errazten du, sistemaren arazoak non dauden identifikatzen laguntzeko.
Garrantzia	Altua

4.24 Taula: Sistemaren erabilerraztasuna barneratzen duen BEF-4 betekizuna.

Identifikatzailea	**BEF-4**
Izena	Erabilerraztasuna
Lotutako helburuak	HZ1
Lotutako betekizunak	BF-1, BF-2, BF-12, BF-13, BF-14, BF-15, BF-20
Deskribapena	Erregistratuta dagoen edozein pertsonak sistema inongo aurreprestakuntzarik gabe erabiltzeko gaitasuna izango du.
Garrantzia	Altua

4.25 Taula: Sistemaren eramangarritasuna barneratzen duen BEF-5 betekizuna.

Identifikatzailea	**BEF-5**
Izena	Eramangarritasuna
Lotutako helburuak	HZ1
Lotutako betekizunak	—
Deskribapena	Aplikazioak gaur egungo edozein web arakatzailean erabili ahal izango da.
Garrantzia	Ertaina

4.4 Trazabilitate matrizea

Azken atal honetan proiektuaren helburu zehatzak eta betekizunak barneratzen dituen trazabilitate matrizea azaltzen da, ikusi 4.26 Taula.

4.26 Taula: Helburu zehatzen eta betekizunen trazabilitate matrizea.

	HZ1	**HZ2**	**HZ3**	**HZ4**
IB-1	X			
IB-2	X			
IB-3	X			
IB-4	X			
BF-1	X			

Hurrengo orrian jarraitzen du

4.26 Taula: Helburu zehatzen eta betekizunen trazabilitate matrizea. (Jarraitua)

	HZ1	HZ2	HZ3	HZ4
BF-2	X			
BF-3	X			
BF-4	X			
BF-5	X			
BF-6	X			
BF-7	X			
BF-8	X			
BF-9	X			
BF-10	X			
BF-11	X			
BF-12	X			
BF-13	X			
BF-14	X			
BF-15	X			
BF-16	X			
BF-17	X			
BF-18	X		X	
BF-19	X			
BF-20	X	X	X	X

KAPITULUA 5

Arkitektura eta diseinua

Kapitulu honetan sortutako softwarearen arkitekturaren eta diseinuaren atal nagusiak azaltzen dira. Lehenik eta behin, softwareak jarraitzen duen arkitektura orokorra eta osagaien arteko komunikazioa azaltzen da. Jarraian, aplikazioko osagaiek beharrezko datuak gordetzeko erabili duten datu-basearen diseinua erakusten da eta aplikazioak erabiltzen dituen ariketek behar duten informazioan sakontzen da. Azkenik, backendaren eta MCP zerbitzariaren arteko komunikazioa nola egin den azaltzen da, baita ere, MCP zerbitzariak eskainitako tresnak eta erabiltzeko bete beharrezko eskemak ere.

5.1 Aplikazioaren arkitektura orokorra

Diseinatu den softwarearen arkitekturak hiru osagai nagusi ditu:

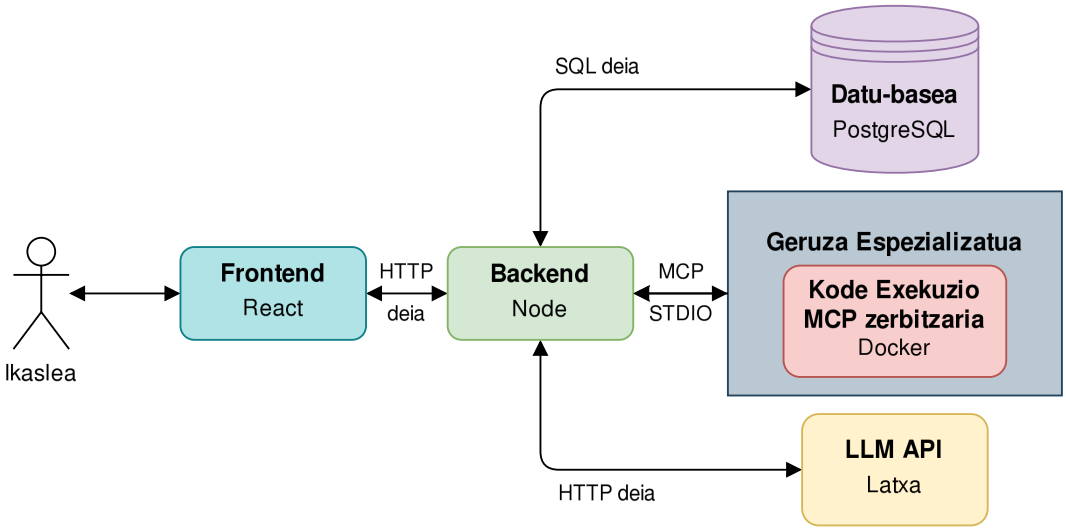
Bezero geruza
: Arkitekturako bezero nagusia da. Erabiltzaileak erabiliko duen frontend aplikazioak osatzen du.

Zerbitzari geruza nagusia
: Arkitekturako zerbitzaria nagusia da. Backend aplikazioak osatzen du eta sistemako orkestratzaile gisa jokatzen du.

Geruza espezializatua
: MCP zerbitzariek osatzen duten geruza espezializatua da. Gure kasuan, kode exekuzio MCP zerbitzaria izango du. Geruza espezializatuko MCP zerbitzariak bakarrik zerbitzari geruza nagusiarekin komunikatuko dira.

Hiru geruza hauek hiru entitateekin komunikatuko dira: ikaslea, LLMa eta datu-basea. Sistemaren elkarrekintza fluxu orokorra 5.1 Irudian ikusi daiteke.

Orokorrean, lan-fluxua honakoa izango da. Ikasleak frontendak eskaintzen duen Graphical User Interface-a (GUI) erabiltzen du ekintza zehatz bat egiteko. Frontendak ikasleak egindako ekintza prozesatzen du eta ikasleak eskatutako datuak eskuratzeko, sortzeko edo eguneratzeko backendarekin komunikatzen da. Backendak eskaera bat jasotzean zein



5.1 Irudia: Aplikazioaren arkitektura orokorra.

motakoa den ikusiko du eta erantzun bat sortuko du: datu-baseko datuak behar badira datu-basearekin konektatuko da, geruza espezializatuko MCP zerbitzari baten emaitza behar badu MCP zerbitzariari deituko du, eta LLMaren erantzuna behar badu LLMari deituko dio.

Arkitektura honek aplikazioaren erantzukizunak banatzen ditu mantengarritasuna eta eskalagarritasuna bermatzeko. Etorkizunean, osagai berri bat gehitu behar denean, egin beharreko aldaketak ez lukete jada existitzen diren osagaien funtzionalitateak apurtu.

5.2 Arkitekturaren iterazioak

Aplikazioaren diseinuan bilakaera iteratiboan oinarritutako arkitektura bat definitu da. Prozesu horretan bi fase edo iterazio nagusi aurreikusi dira, sistemaren eskalagarritasuna eta epe luzeko bideragarritasuna bermatzeko. GrAL honen barruan, sistemaren muina eta lehenengo iterazio osoa garatu da. Bigarren iterazioa, arkitekturaren etorkizuneko hobekuntza eta optimizazio gisa planteatzen da.

5.2.1 Lehenengo iterazioa: oinarrizko arkitektura

Lehenengo fase honetan, sistemaren oinarrizko arkitektura nagusia inplementatu da osorik: frontenda, datu-basea, LLM ereduarekiko komunikazio-geruza eta kodea exekutatzeko MCP zerbitzaria.

Hurbilpen honen ezaugarri nagusia backendaren kudeaketa zentralizatua da, eta horixe da bigarren iterazioarekiko diferentzia nagusia. Batetik, frontendak ikasleen eta ariketen datuak eskatzen dizkionean backendari, honek konexio zuzena egiten du datu-basearekin beharrezko informazioa berreskuratzeko. Bestetik, backendak LLMari elkarrizketa-fluxuak bidaltzen dizkionean, ikaslearen profilari buruzko datuekin testuinguru espezifiko bat eraikitzen du, eta informazio hori guztia elkarrizketarekin batera helarazten dio ereduari.

5.2.2 Bigarren iterazioa: eskalagarritasuna

Behin oinarrizko aplikazioa martxan egonda, bigarren iterazioak jada funtzionatzen duen sistema hori hartzen du oinarritzat, hainbat optimizazio aplikatzeko. Aldaketa hauen helburu nagusia ariketen eta ikasleen informazioaren eskuraketa backendetik kanporatzea da, LLM ereduak une bakoitzean behar eta nahi duen informazio espezifikoa soilik eskuratu dezan.

Horretarako, geruza espezializatuan bi MCP zerbitzari gehiago sortzea aurreikusten da (zerbitzari bat domeinu bakoitzarentzat): ariketen MCP zerbitzaria eta aurrerapenaren MCP zerbitzaria. Zerbitzari espezializatu bakoitzak datu-basearekin konexio independentea izango du, eta hainbat tresna eskainiko ditu datuak kudeatzeko:

- Ariketen MCP zerbitzaria: Ariketen informazio orokorra eta ariketa bakoitzari lotutako proba-kasuak (testak) berreskuratzeko erabiliko da.
- Aurrerapenaren MCP zerbitzaria: Ikaslearen informazioa zein egindako ebazpen eta saiakeren datuak eskuratu, sortu eta eguneratzeko tresnak eskainiko ditu.

Bi zerbitzari hauek integratuta, frontendak backendari ariketaren edo ikaslearen informazioa eskatzen dionean, backendak ez du datu-basea zuzenean atzituko; aitzitik, dagokien MCP zerbitzariei eskatuko die informazioa.

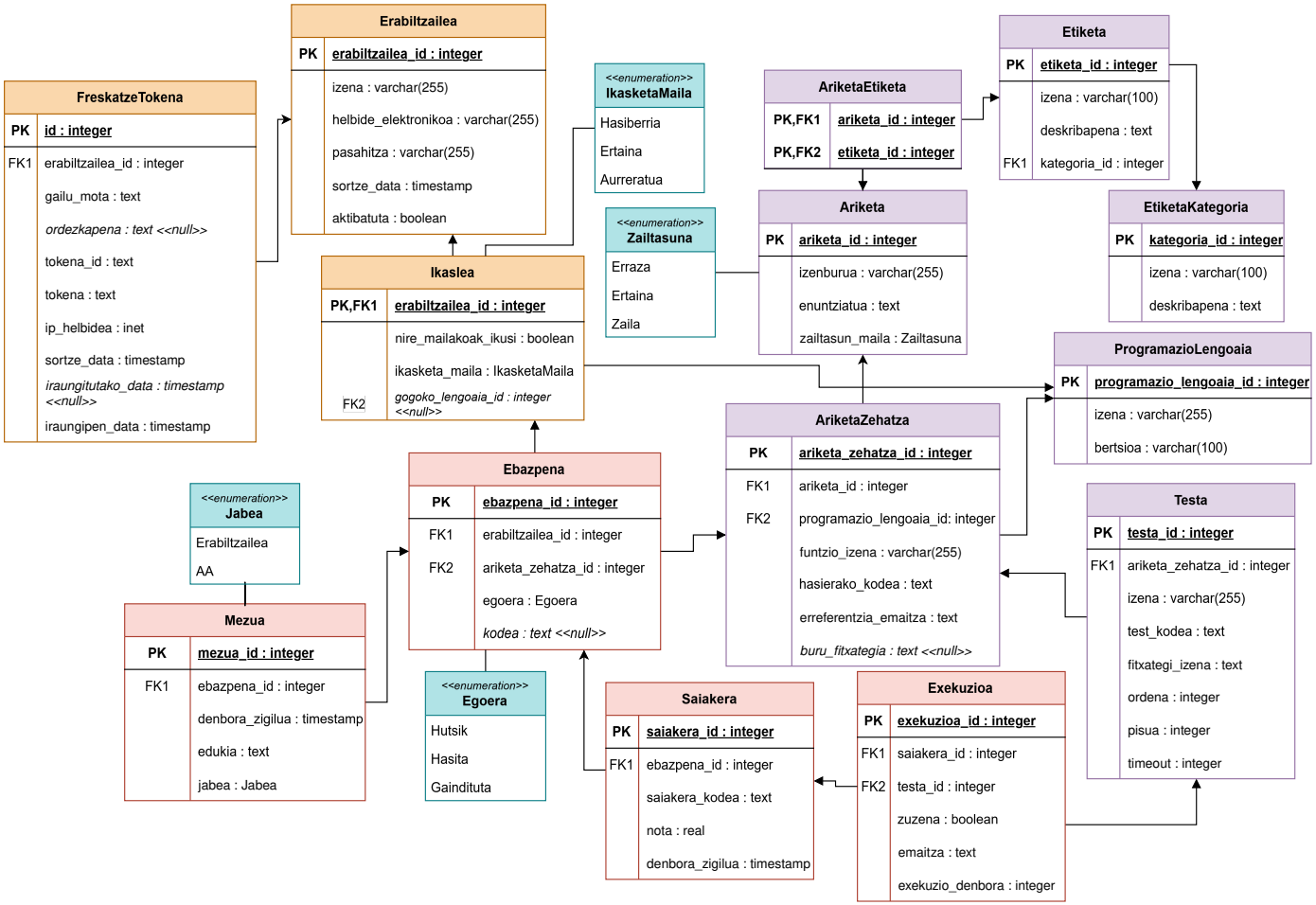
Planteamendu honek abantaila handia dakar LLMarekiko komunikazioan. Testuinguru estatiko bat une oro eraiki eta bidali beharrean, LLMari MCP tresnak (tools) emango zaizkio, informazioa behar duenean bere kabuz eskuratu dezan. Horrela, elkarrizketa bakoitzean bidali beharreko testuinguruaren tamaina nabarmen murriztuko da, eta backendak testuinguru-orkestratzaile gisa jardungo du soilik.

5.3 Datu-basearen diseinua

Aplikazioak kudeatu beharko dituen datuak gordetzeko PostgreSQLen erabiltzeko datubase erlazional bat diseinatu da, ikusi 5.2 Irudia. Irudian ikusten den bezala, taulak hiru taldetan banatu daitezke.

- **Kautotze** taulak, kolore horiarekin: `FreskatzeTokena`, `Erabiltzailea` eta `Ikaslea`.
- **Aurrerapen** taulak, kolore gorriarekin: `Ebazpena`, `Saiakera`, `Exekuzioa` eta `Mezua`.
- **Ariketa** taulak, kolore morearekin: `Ariketa`, `AriketaEtiketa`, `Etiketa`, `Etiketa Kategoria`, `AriketaZehatza`, `ProgramazioLengoaia` eta `Testa`.

Sortutako datu-basea baliagarria da aplikazioarentzat, bai lehenengo zein bigarren iterazioko hurbilpenerako, egituran inolako aldaketarik egin gabe.



5.2 Irudia: Proiektuaren datu-basearen eredu erlazionalaren diagrama UMLz.

5.3.1 Gorde beharreko ariketen informazioa

Datu-basean gorde behar diren datuen zati garrantzitsu bat plataformak erabiliko dituen ariketak dira. Informazio datu-multzo honetan hainbat datu mota gorde behar dira:

- **Ariketarekin zuzenean lotuta dagoen informazioa**: izenburua eta enuntziatua.
- **Ariketarekin zeharka lotuta dagoen informazioa**: etiketaren izena, etiketaren deskribapena, kategoriaren izena eta kategoriaren deskribapena.
- **Ariketaren ebazpen posible batekin lotuta dagoen informazioa**: programazio-lengoaiaren izena, programazio-lengoaiaren bertsioa, ariketaren ebazpenaren funtzioaren izena, ebazpenaren hasierako kodea, ebazpen kodearen erreferentzia emaitza bat eta, beharrezkoa bada, buru fitxategiaren kodea.
- **Ariketaren ebazpena zuzena dela egiaztatzen duten testen informazioa**: test multzoaren izena, test multzoaren fitxategi izena, test multzoaren exekuzio kodea, test multzoen exekuzio ordena, ikaslearen ebaluaziorako test multzoaren pisu erlatiboa eta exekuzio denbora maximoa.

3 kapituluan azaldu den bezala, ariketen sorkuntza proiektu honetatik kanpo utzi da. Plataforma erabili eta probatu ahal izateko, ariketa multzo bat hartu da Leetcodetik eskuragarri dauden metadatuekin. Zehazki, honako datuak eskuratu dira:

- Ariketaren izenburua, enuntziatua eta etiketen izenak.
- Programazio-lengoaien ariketaren hasierako eta erreferentzia kodea.

Gainontzeko metadatuak sortu egin behar izan dira, eta, proben kasuan, gutxieneko proba unitarioak idatzi dira.

5.4 Backendaren endpointak

Backenda REST API bat da. Atal honetan API honek eskainiko dituen endpointen definizioak zerrendatuko dira, domeinuaren arabera banatuta. Endpoint bakoitzarentzako honako informazioa adierazi da: ibilbidea, HTTP metodo mota, eskaeran bidali beharreko datuak eta erabiltzailea kautotuta egon behar den ala ez. Aipatu behar da erabiltzailea kautotuta badago, backendak erabiltzailearen identifikatzailea `req.user` objektuan izango duela.

REST APIaren endpoint nagusiak 5.1 Taulan ikusi daitezke, eta endpoint guztien definizio zehatzak B Eranskinan ikusi daitezke.

5.5 MCP komunikazioa

2.3.2 azpiatalean azaldu den bezala MCPek bi garraio mekanismo inplementatzen ditu. Gure kasuan, MCP zerbitzariak eta backenda gailu berdinean exekutatuko direnez STDIO garraio mota erabiliko da.

Protokoloa ez da hutsetik inplementatu behar izango, baizik eta MCPren TypeScripterako Software Development Kit (SDK) ofiziala erabiliko da: `@modelcontextprotocol/sdk`.

5.1 Taula: Backendak izango dituen endpoint nagusiak.

Domeinua	Ibilbidea	Metodoa	Kautotzea
Saiakerak	/api/attempts	`Get`	Bai
Saiakerak	/api/attempts/:saiakera_id	`Get`	Bai
Saiakerak	/api/attempts/solution	`Post`	Bai
Kautotzea	/api/auth/register	`Post`	Ez
Kautotzea	/api/auth/login	`Post`	Ez
Kautotzea	/api/auth/refresh	`Post`	Ez
Txata	/api/chat	`Get`	Bai
Txata	/api/chat/send	`Post`	Bai
Aurrerapena	/api/dashboard	`Get`	Bai
Ariketak	/api/exercises	`Get`	Bai
Ariketak	/api/exercises/:ariketa_id	`Get`	Bai
Erabiltzailea	/api/users/profile	`Get`	Bai
Erabiltzailea	/api/users/account	`Put`	Bai
Erabiltzailea	/api/users/password	`Put`	Bai
Erabiltzailea	/api/users/education	`Put`	Bai
Erabiltzailea	/api/users/account	`Delete`	Bai

5.6 Kode exekuzio MCP zerbitzariaren tresnak eta funtzionamendu adibidea

2.3 atalean MCP AAen eta informazio iturrien arteko komunikazio segurua eta estandarra egiteko diseinatu dela azaldu da. Lan honetan MCP zerbitzari bat inplementatu da. Diseinatutako kode exekuzio MCP zerbitzariak hiru tresna nagusi eskaintzen dizkio sistemari, plataformak onartzen duen programazio-lengoaia bakoitzeko bat: `run_c_exercise`, `run_java_exercise` eta `run_python_exercise`.

5.6.1 Eskema komuna eta funtzionamendu-fluxua

Tresna guztiek sarrera-egitura ia berdina dute —ikaslearen kodea eta exekutatu beharreko testak— eta irteera-eskema estandarra itzultzen dute JSON formatuan. Tresnen sarrera-egituren desberdintasun bakarra `attempt` parametroaren balioa da, 5.1 Kodean adierazten den moduan, programazio-lengoaia batzuek ikaslearen kodeaz gain buruko kodea ere behar dute. 5.2 Kodean irteera-eskema eta 5.3 Kodean testu bihurtutako `text` parametroaren eskema aurkezten da.

```
type inputSchema = {
    attempt: string | {
        header?: string;
        source: string;
    };
    tests: Array<{
```

```
        code: string;
        fileName: string;
        name: string;
        order: number;
        testId: number;
        timeout: number;
        weight: number;
    }>;
};
```

5.1 Kodea: Kode exekuzio MCP zerbitzarien tresnen sarrera eskema orokorra.

```
type ToolResult = {
    content: Array<{
        text: string;
        type: "text";
    }>;
    isError: boolean;
};
```

5.2 Kodea: Kode exekuzio MCP zerbitzariaren tresnen erantzunaren eskema.

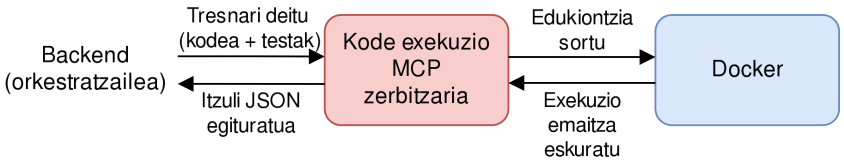
```
type ExecutionResult = {
    duration: number;
    error: null | string;
    preRun: null | {
        duration: number;
        exitcode: null | number;
        stderr: string;
        stdout: string;
        success: boolean;
    };
    testResults?: Array<{
        duration: number;
        exitCode: null | number;
        name: string;
        order: number;
        phase: null | string;
        status: TestStatus;
        stderr: string;
        stdout: string;
        testId: number;
        weight: number;
    }>;
};
```

5.3 Kodea: Kode exekuzio MCP zerbitzariaren tresnen erantzunaren `text` parametroaren eskema.

Ondorengo 5.3 Irudiak MCPk nola jokatzen duen erakusten du.

5.6.2 Adibide praktikoa: Python ariketa baten ebaluazioa

Jarraian, MCPren jokaera garbiago ikusteko, ariketa multzoko “Bi gehitu” ariketa Python programazio-lengoaian oinarritutako adibide praktiko bat erakusten da.



5.3 Irudia: MCPren jokaera aplikazioan.

1. Lehenik eta behin, backendak `run_python_exercise` deitzen du ikaslearen iturburu-kodearekin (`attempt.source`) eta proba-kasuen multzoarekin (`tests`). Proba multzo bakoitzak fitxategiaren izena (`fitxategi_izena`), proba multzoaren izena (`izena`), exekuzio ordena (`ordena`), nota kalkulatzeko pisu erlatiboa (`pisua`), proba multzoaren identifikatzailea (`testa_id`), proba-kasuen kodea (`testa_kodea`) eta exekuzio muga-denbora (`timeout`) atributuak ditu.

```
mcp.executeCode('Python', {
    attempt: {
        header: null,
        source: 'class Solution:\n' +
          '    def twoSum(self, nums: list[int], target: int) -> list[int]:\n' +
          '        hashmap = {}\n' +
          '        for i in range(len(nums)):\n' +
          '            complement = target - nums[i]\n' +
          '            if complement in hashmap:\n' +
          '                return [i, hashmap[complement]]\n' +
          '            hashmap[nums[i]] = i\n' +
          '        return []'
    },
    tests: [
        {
            fitxategi_izena: 'basicCase.py',
            izena: 'Oinarrizko kasua',
            ordena: 1,
            pisua: 1,
            testa_id: 1,
            testa_kodea: 'from src.attempt import Solution\n' +
              '\n' +
              'solution = Solution()\n' +
              '\n' +
              'def test_1():\n' +
              '    result = solution.twoSum([2, 7, 11, 15], 9)\n' +
              '    assert sorted(result) == [0, 1]\n' +
              '\n' +
              'def test_2():\n' +
              '    result = solution.twoSum([3, 2, 4], 6)\n' +
              '    assert sorted(result) == [1, 2]\n' +
              '\n' +
              'def test_3():\n' +
              '    result = solution.twoSum([3, 3], 6)\n' +
              '    assert sorted(result) == [0, 1]\n',
            timeout: 2000
        }
    ]
})
```

2. Bigarrenik, kode exekuzio MCP zerbitzariak deia jasotzen du, Docker ingurunean

test multzo bakoitza indibidualki exekutatzen du eta test guztien emaitza itzultzen du. Emaitzak proba unitario guztien exekuzio denbora (`duration`), exekuzioan errorea gertatu den (`error`), programazio-lengoaiaren konpilazioaren emaitza behar badu (`preRun`) eta proba-kasuen multzoaren emaitzak (`testResults`) atributuak ditu. Proba-kasu multzoaren emaitzak exekuzio denbora (`duration`), irteera-kodea (`exitCode`), proba-kasuaren izena (`name`), exekuzio ordena (`order`), exekuzio fasea (`phase`), exekuzioaren emaitzaren egoera (`status`), exekuzioaren emaitzaren irteera eta errore kanalak (`stdout` eta `stderr`), proba multzoaren identifikatzailea (`testId`) eta nota kalkulatzeko pisu erlatiboa (`pisua`) atributuak ditu.

```
{
    duration: 2040,
    error: null,
    preRun: null,
    testResults: [
        {
            duration: 2077,
            exitCode: 0,
            name: 'Oinarrizko kasua',
            order: 1,
            phase: 'run',
            status: 'passed',
            stderr: '',
            stdout: '============================= test session starts ==============================\n' +
              'platform linux -- Python 3.14.5, pytest-9.0.3, pluggy-1.6.0\n' +
              'rootdir: /opt/code-execution\n' +
              'collected 3 items\n' +
              '\n' +
              'tests/test_basicCase.py ...                                             [100%]\n' +
              '\n' +
              '============================== 3 passed in 0.02s ===============================\n',
            testId: 1,
            weight: 1
        }
    ]
}
```

3. Azkenik, kode exekuzio MCP zerbitzariak exekuzioaren emaitza JSON egituratuan eta estandarizatuan biltzen du (`content[0].text`) eta backendari itzultzen dio.

```
{
    isError: false,
    content: [
        {
            text: JSON.stringify({
                duration: 2040,
                error: null,
                preRun: null,
                testResults: [
                    {
                        duration: 2077,
                        exitCode: 0,
                        name: 'Oinarrizko kasua',
                        order: 1,
```

```
                        phase: 'run',
                        status: 'passed',
                        stderr: '',
                        stdout: '============================= test session starts ==============================\n' +
                          'platform linux -- Python 3.14.5, pytest-9.0.3, pluggy-1.6.0\n' +
                          'rootdir: /opt/code-execution\n' +
                          'collected 3 items\n' +
                          '\n' +
                          'tests/test_basicCase.py ...                                [100%]\n' +
                          '\n' +
                          '============================== 3 passed in 0.02s ===============================\n',
                        testId: 1,
                        weight: 1
                    }
                ]
            }, null, 2),
            type: "text"
        }
    ]
}
```

5.7 Feedback pedagogikoaren diseinua eta LLMaren testuingurua

Atal honetan, LLMak feedback pertsonalizatua sortzeko bidaltzen zaion informazio egitura eta promptaren diseinu estrategikoa zehazten dira. Sistemaren helburua ez denez ikasleari erantzun zuzena ematea, LLMari bidalitako testuingurua eta jarraibideak zorrotz definitu eta kontrolatu behar dira. LLMaren APIari egiten den eskaera batean ondoko informazioa bidaliko da: system prompta, elkarrizketako azken mezuak, ikaslearen mezu berria, ikaslearen iturburu-kodea eta MCP tresnak.

5.7.1 Eskaera eskema

Zehazki, APIari egingo den eskaerak honako itxura du:

- `max_tokens`: erantzunak erabili ditzakeen token kopuru maximoa.
- `messages`: system promptak, LLMaren eta ikaslearen elkarrizketaren azken mezuek eta ikaslearen uneko kodeak osatutako mezu zerrenda.
- `model`: LLM ereduaren izena.
- `temperature`: LLMaren irteera ausazkoagoa bilakatzeko balioa.
- `tools`: LLMak erabili ditzakeen MCP tresnen zerrenda.

5.7.2 Feedbackaren sorkuntza-fluxua

Ikasleak txatean mezu bat idaztean, LLMaren feedbacka sortzeko hurrengo fluxua exekutatzen da:

1. Ikasleak txatean mezua idazten du eta bidali egiten du.
2. Backendak ikaslearen mezua jasotzen du.
3. Backendak datu-basetik ikaslearen azken saiakerak, uneko kodea eta ikasketa maila eskuratzen ditu.
4. Backendak datu-basetik ariketaren enuntziatua, izenburua, programazio-lengoaia eta erreferentzia emaitza eskuratzen ditu.
5. Backendak datu-basetik eskuratutako datuekin, uneko kodea erabili gabe, system prompta eraikitzen du.
6. Backendak datu-basetik ikaslearen elkarrizketak eskuratzen ditu.
7. Backendak ikaslearen uneko kodea elkarrizketara atxikitzen du.
8. Backendak LLMari bidaliko den elkarrizketaren luzera mugatzen du.
9. Backendak aurretik bildutako informazioarekin LLMaren APIari eskaera bidaltzen dio.
10. LLMaren APIk backendari erantzuna bidaltzen dio.
11. Backendak LLMaren erantzunak tresna dei eskaera duen aztertzen du. Hala bada, backendak tresna deitzen du, tresnaren erantzuna jasotzen du, LLMari birbidaltzen dio eta LLMaren erantzuna jasotzen du.
12. Backendak ikaslearen mezua berria eta LLMaren erantzuna datu-basean gordetzen ditu.
13. Backendak frontendari LLMaren erantzuna bidaltzen dio.
14. Frontendak erantzuna ikuskarazten du.

Adibide gisa, ikasleak mezu bat bidaltzean sortzen den testuingurua erakusten da.

```
[
    {
        "content": "SYSTEM-PROMPT\n\n"
            + "## ARIKETAREN ENUNTZIATUA: Bi gehitu\n\n"
            + "Programazio ariketa hau Python 3.14 programazio lengoaian ebatzi behar du ikasleak. Ariketa honen enuntziatua honako hau da:\n\n"
            + "ARIKETAREN-ENUNTZIATUA\n\n\n"
            + "## ARIKETAREN ERREFERENTZIA EMAITZA\n\n"
            + "```Python\nclass Solution:\n    def twoSum(self, nums: list[int], target: int) -> list[int]:\n        hashmap = {}\n        for i in range(len(nums)):\n            complement = target - nums[i]\n            if complement in hashmap:\n                return [i, hashmap[complement]]\n            hashmap[nums[i]] = i\n        return []\n\n```\n\n"
            + "## IKASLEAREN AZKENEKO SAIKERAK\n\n"
            + "- Data: 2026-06-12T10:20:14.331Z, Nota: 10\n",
        "role": "system"
    },
    {
```

```
        "content": "Kaixo, zein zara?\n\n"
            + "Ikaslearen uneko kodea:\n"
            + "```Python\nclass Solution:\n    def twoSum(self, nums: list[int],
            ↪  target: int) -> list[int]:\n        hashmap = {}\n        for i in
            ↪  range(len(nums)):\n            complement = target - nums[i]\n
            ↪  if complement in hashmap:\n                return [i,
            ↪  hashmap[complement]]\n            hashmap[nums[i]] = i\n
            ↪  return []\n```",
        "role": "user"
    }
]
```

Eta, jarraian, LLM ereduak sortutako eta itzulitako mezua erakusten da.

```
Kaixo! Irakaslea naiz. Zertan lagundu zaitzaket? Arazoren bat duzu ariketarekin? Edo
agian azalpenen bat behar duzu? Esan zer behar duzun!

Zure kodea aztertzen ari naiz... Itxura ona du! Baina, nola dakizu `nums` zerrenda ez
dela hutsik egongo? Eta zer gertatzen da `target` zenbakia ezin bada lortu `nums`
zerrendako bi zenbakiren baturarekin?
```

Laburbilduz, kapitulu honetan sortutako softwarearen arkitekturaren eta diseinuaren erabaki nagusiak azaldu dira: aplikazioaren arkitektura orokorra, garapen iterazioak, datu-basearen diseinua, backendak eskaintzen dituen endpointak, MCP protokoloaren erabilera entitateen komunikaziorako, kode exekuzio MCP zerbitzariak eskaintzen dituen tresnak (tools) eta LLM ereduari bidalitako testuingurua. Hurrengo kapituluan, diseinu erabaki hauek kontuan hartuta, aplikazioa inplementatzeko hartutako erabakiak eta tresnak azaltzen dira.

KAPITULUA 6

Inplementazioa

Kapitulu honetan aplikazioaren osagai desberdinen inplementazioan hartutako erabaki nagusiak azaltzen dira: datu-basea sortzeko erabilitako tresnak, backendaren egitura eta segurtasuna eta middleware konfigurazioa, kode exekuzio MCP zerbitzariaren kodearen exekuzioa edukiontzietan, eta frontendaren egitura.

Garatu den web-plataformaren iturburu-kodea publikoki eskuragarri dago `https://github.com/Josu-A/gral` estekan.

6.1 Datu-basearen inplementazioa

Behin datu-eredu erlazionala diseinatua (ikusi 5.3 atala) PostgreSQL sistema hautatu da datu-basea inplementatzeko: entitateen arteko erlazio konplexuak adierazteko, kode irekiko eta lokalean exekutatu daitekeen RDBMS egoki bat [31]. Inplementazioan, datuen osotasuna bermatzeko Prisma Object–Relational Mapper (ORM) tresna —zerbitzarien eta datu-basearen artean kokatzen den geruza— integratu da [32] Node.js eta TypeScript erabiliz.

6.1.1 Mantengarritasuna

Datu-basearen bilakaera eta mantengarritasuna modu egokian kudeatzeko, Prisma ORMak 3 abantaila eskaintzen ditu:

- Datu-basearen eskema erabiliko den Database Management System-arekiko (DBMS) agnostikoa da; hots, DBMS aldatzeko aukera ematen du eskemaren sintaxia aldatu gabe.
- Zehaztutako eskematik mota-segurua duen bezero bat sortzen du. Bezero hau erabiliz errore gehienak konpilazio-garaian detektatzen dira, eta kodean entitateen eta haien propietateen osaketa automatikoa ahalbidetzen du.
- Migrazioak erabiltzen ditu datu-basearen eguneraketak kudeatzeko, datu-basearen egoera fidagarria izateko eta ingurune ezberdinetan sinkronizatuta mantentzeko.

Prismaren funtzionamendua hobeto ulertzeko, 5.2 Irudian diseinatutako datu-basearen entitate-erlazio diagramaren `Erabiltzailea` entitatearen inplementazioa 6.1 Kodean erakusten da adibide gisa.

```
model Erabiltzailea {
    erabiltzailea_id     Int      @id @default(autoincrement())
    izena                String   @db.VarChar(255)
    helbide_elektronikoa String   @unique @db.VarChar(255)
    pasahitza            String   @db.VarChar(255)
    sortze_data          DateTime @default(now())
    aktibatuta           Boolean  @default(false)

    ikaslea Ikaslea?

    freskatze_tokenak FreskatzeTokena[]

    @@index([helbide_elektronikoa])
    @@map("erabiltzailea")
}
```

6.1 Kodea: Prisma erabiliz `Erabiltzailea` taularen inplementazioa.

6.1.2 Hasierako datuen karga

Datu-basea sortzerakoan eta eskema ezartzean, datu-baseko taulak hutsik daude. GrAL honetan sortutako aplikazioarekin lan egin ahal izateko hasierako datu batzuk gehitu behar dira, bai garapen ingurunean erabiltzeko, bai ekoizpen ingurunean erabiltzeko. Jarraian, ingurune bakoitzak kargatzen dituen datuak eta integritate logikoa bermatzeko kargatzeko ordena zerrendatzen dira.

- Ekoizpen ingurunea: programazio-lengoaiak, ariketen kategoriak, ariketen etiketak, LeetCode plataformatik jasotako ariketak eta metadatuak, eta ariketen proba unitarioak.
- Garapen ingurunea: ekoizpen inguruneko datuak orden berdinean kargatzen dira, eta, amaitzeko, ikasle-probak egiteko gezurreko ikasleen erabiltzaileak.

Datu-basea datuz betetzeko *seed script* bat erabiliko da. Kargatu nahi diren datuak JSON objektuetan gordeko dira TypeScript fitxategietan eta Prismak kargatuko duen script baten bidez datu-basea osatuko da.

6.2 Backendaren inplementazioa

Backend zerbitzaria inplementatzeko Express frameworka [33] erabili da Node.js exekuzio-denbora ingurunean [34]. Garapenaren mantengarritasuna bermatzeko eta JavaScripten datu-mota dinamikoek sor ditzaketen erroreak saihesteko, kode osoa TypeScript programazio-lengoaian garatu da: aldagai estatikoak erabiltzen dituen eta JavaScriptera konpilatzen duen programazioa-lengoaia [35].

Frontendak hainbat datu mota bidali ditzake: *query* gisa, *parameter* gisa, eskaeraren gorputzean eta abar. Datu hauek backendak espero dituen baldintzak bete ditzaten eta

segurtasun kontuengatik, bezeroaren eskaera guztietan bidaltzen diren datuak baliozkotu egin dira. Horretarako Zod liburutegia erabili da [36]. Zod erabiliz baliozkotutako eskemak berrerabili daitezke eta TypeScripten datu motak automatikoki sortu daitezke, eskema berdina bi toki desberdinetan mantendu behar ez izateko. Liburutegiaren erabileraren adibide gisa erabiltzaileak saioa hasteko bidali behar duen eskema ikusi daiteke 6.2 Kodean. Bertan, helbide elektronikoa eta pasahitza behar duen `LoginSchema` eskema definitzen da eta TypeScripten erabiltzeko datu-mota `infer` erabiliz automatikoki sortzen da.

```
import { z } from "zod";

const LoginSchema = z.object({
    helbide_elektronikoa: z.email("Helbide elektroniko ez da baliozkoa"),
    pasahitza: z.string().min(1, "Pasahitza beharrezkoa da"),
});

type ILogin = z.infer<typeof LoginSchema>;
```

6.2 Kodea: Zod erabiliz erabiltzaileak saioa hasteko bidali behar dituen datuen eskemaren inplementazioa.

6.2.1 Entitateen arteko komunikazioa

Backendak sistemako beste entitateekin komunikatzeko hiru interfaze nagusi integratu dira.

Datu-basearen konexioa
: Datu-basean Create, Read, Update and Delete (CRUD) eragiketak egin ahal izateko Prismak sortzen digun `db` objektua erabili da. Honek sistemaren domeinuko entitateen gainean eragiketak exekutatzea ahalbidetzen du.

MCP zerbitzariarekin lotura
: Backendak MCP bezero gisa jokatzen du. 5.5 atalean azaldu dugu MCP zerbitzariek `@modelcontextprotocol/sdk` erabiliko dutela zerbitzaria sortzeko. Era berean, backendak MCP bezeroa sortzeko `@modelcontextprotocol/sdk` erabiliko du: `Client` objektua bezeroa sortzeko eta `StdioClientTransport` garraio mekanismoa zehazteko.

LLM integrazioa
: Ikasleari euskarazko laguntza pertsonalizatua eskaintzeko Euskal Herriko Unibertsitateko (EHU) Hitz zentroak garatutako Latxa 70Bko hizkuntza-eredua integratu da. Latxaren zerbitzariak OpenAIren APIarekin bateragarria den interfazea eskaintzen duenez, komunikazioa OpenAIren liburutegi ofiziala erabiliz bideratu da [37].

6.2.2 Segurtasuna eta middleware konfigurazioa

Backenda eta datu-basea aplikazioaren puntu kritikoak dira. Hori dela eta, garrantzitsua da datu-basea eta, batez ere, zerbitzaria, ohiko erasoetatik babestea. Horretarako, Expressekin ondo lan egiten duten hiru *middleware*[1] integratu dira.

[1] Middleware bat funtzio bat da. Zerbitzariak eskaera bat jasotzean funtzio bati pasatzen dio, aztertu egiten du eta prozesatu ondoren zerrendako hurrengo funtziora pasatzen du eskaera. Baina, eskaera okerra dela ondorioztatzen bada, baztertu egiten da eta ez da hurrengo funtziora pasatzen.

Cors [38]
Kanpoko eskaerak kontrolatzeko mekanismoa —ingelesez, Cross-Origin Resource Sharing (CORS)— aplikatzeko erabili da. Garrantzitsua da backendak gure frontendaren *cross-origin* eskaerak bakarrik onar ditzan eta beste guztiak baztertu ditzan. Kontuan hartu behar da CORS nabigatzaileen mekanismoa dela. Beraz, ez du eraginik izango `curl` edo antzeko programak erabiltzen badira.

Helmet [39]
HTTP erantzunen goiburuak modu seguruan ezartzen dituen tresna da. Cross Site Scripting (XSS) eta antzeko erasoak ekiditeko ezinbestekoa da.

Express-rate-limit [40]
Bezero zehatz batek denbora tarte batean egin ditzakeen eskaerak mugatzen ditu. Denial of Service (DOS) eta "indar gordinaren" —ingelesez, *brute-force*— bidezko erasoak saihesteko aplikatu da, bereziki kautotze endpoint sentikorretan.

6.2.3 Kautotzea

Web-plataformako eremu babestuak —aurrerapena, ariketen zerrenda, lan-ingurunea eta profila— seguruak izateko, RFC 7519 estandarra jarraitzen duen JWT mekanismoa inplementatu da [41]. Saioa hastean, backendak bi token sortzen eta bidaltzen dizkio frontendari rola eta segurtasuna bermatzeko:

Access tokena
Iraupen motzeko tokena da. Tokena sortzeko erabiltzailearen helbide elektronikoa eta identifikatzailea kodifikatu eta sinatu egiten dira sekretu bat erabiliz. Ondoren, access tokena frontendari bidaltzen zaio eta frontendak tokena memorian gordetzen du. Hemendik aurrera, frontendak backendari kautotzea behar duen eskaera bat bidaltzen dionean HTTP *Authorization* burukoan access tokena atxikiko du, eta backendak eskaera jasotzean tokena baliozkoa dela egiaztatuko du. Access tokenei 15 minutuko iraupena ezarri zaie.

Refresh tokena
Iraupen luzeko tokena da. Tokena sortzeko erabiltzailearen helbide elektronikoa eta tokenaren identifikatzailea, beste sekretu bat erabiliz, kodifikatu eta sinatu egiten dira. Jarraian, refresh tokenaren hasha datu-basean gordetzen da. Ondoren, frontendari refresh tokena `httpOnly`, `sameSite` eta `secure` motako cookie batean bidaltzen zaio. Hemendik aurrera, backendak frontendari access tokena baliogabea dela jakinarazten badio, erabiltzaileak saioa berriz hasi gabe frontendak backendari access token eta refresh token berriak eskatuko dizkio. Refresh tokenei 14 eguneko iraupena ezarri zaie.

Segurtasuna guztiz bermatzeko, Refresh Token Rotation (RTR) mekanismoa inplementatu da. Aktore gaizto batek erabiltzaile baten refresh tokena eskuratzen badu saioa hasi beharrik gabe erabiltzailearen kontua erabili dezake. Hau saihesteko, refresh tokena berrerabiltzen denean refresh tokenaren jabeak dituen baliozko refresh token guztiak baliogabetzeko mekanismoa inplementatu da.

6.2.4 Geruza banaketa: CSR patroia

Proiektuaren hedagarritasuna eta kodearen isolamendua bermatzeko, domeinu bakoitzaren barne-logika Controller, Service and Repository (CSR) arkitektura-patroia jarraituz egituratu da:

Controller (Kontrolatzailea)
: Backendaren eta frontendaren arteko komunikazioa kudeatzen du: frontendaren eskaera jaso, eskaera datuak baliozkotu, service (zerbitzua) geruzari prozesatu beharreko datuak igaro eta service (zerbitzua) geruzatik eskuratzen duen erantzuna frontendari itzuli.

Service (Zerbitzua)
: Endpointaren negozio logika exekutatzen du. Hemen kudeatzen dira frontendak controllerari (kontrolatzaileari) egindako eskaera gauzatzeko kalkuluak eta logika.

Repository (Biltegia)
: Service (zerbitzua) geruzak informazioa irakurtzea, gordetzea, eguneratzea edo ezabatzea behar duenean, geruza honek datu-basearen gainean CRUD eragiketak exekutatzen ditu eta emaitzak itzultzen dizkio.

6.3 Kode exekuzio MCP zerbitzariaren inplementazioa

5.5 atalean aipatu den bezala, kode exekuzio MCP zerbitzaria Node.js, TypeScript eta SDK ofiziala erabiliz inplementatu da.

SDK ofiziala erabiliz, zerbitzariak hiru tresna (`run_c_exercise`, `run_java_exercise` eta `run_python_exercise`) eskaintzen dizkio backendari. Eskaera bat STDIO garraio-kanaletik jasotzean, zerbitzariak exekuzio-mota identifikatu, sarrera-datuak balioztatu eta dagokion programazio-lengoiako ingurune isolatura bideratzen du prozesua.

6.3.1 Docker irudien sorrera

Ikasleen kodea kode exekuzio MCP zerbitzariaren ordenagailuan exekutatu liteke, baina honek segurtasun arazoak —begizta infinituak, fitxategi sisteman aldaketak edo erasoak— sortuko lituzke. Hori dela eta, exekuzio-ingurune guztiz isolatuko Docker irudiak sortu dira [42]; irudi bat zerbitzariak onartzen duen programazio-lengoaia bakoitzeko. Docker irudi guztiek ahalik eta baimen gutxien dituen `runner` erabiltzaile bat dute, segurtasun arazoak ekiditeko ez du inongo *root* baimenik eta ikaslearen kodea exekutatzeko erabiltzen da.

6.3.2 Kodearen exekuzioa

Sortutako Docker irudiak edukiontziak sortzeko erabili dira. Edukiontzi hauek ikaslearen kodea konpilatzeko, programazio-lengoaiak konpilazio prozesua behar badu, eta proba multzoak exekutatzeko erabili dira.

Gainera, proba multzo bakoitza exekutatzeko behar den denbora kalkulatzeko eta isolamendua mantentzeko, proba multzo bakoitzeko edukiontzi berri bat sortu da. Isolamendu honek ikaslearen kode konpilatua partekatzeko beharra sortzen du; bestela, proba multzo

bakoitzak exekutatu behar denean lehenik ikaslearen kodea konpilatu beharko da. Hau ekiditeko, tresnaren (tool) dei bat egitean hiru behin-behineko karpeta sortzen dira eta sortuko diren edukiontziei *mount* egiten zaie: iturburu-kodea eta proba unitarioak partekatzeko irakurtzeko soilik diren karpetak, eta exekuzioan zehar sortutako artefaktuak partekatzeko irakurketa-idazketa karpeta. Proba guztiak exekutatzen amaitzerakoan edo errorea gertatutakoan, bai Docker edukiontziak, bai behin-behineko karpetak ezabatzen dira.

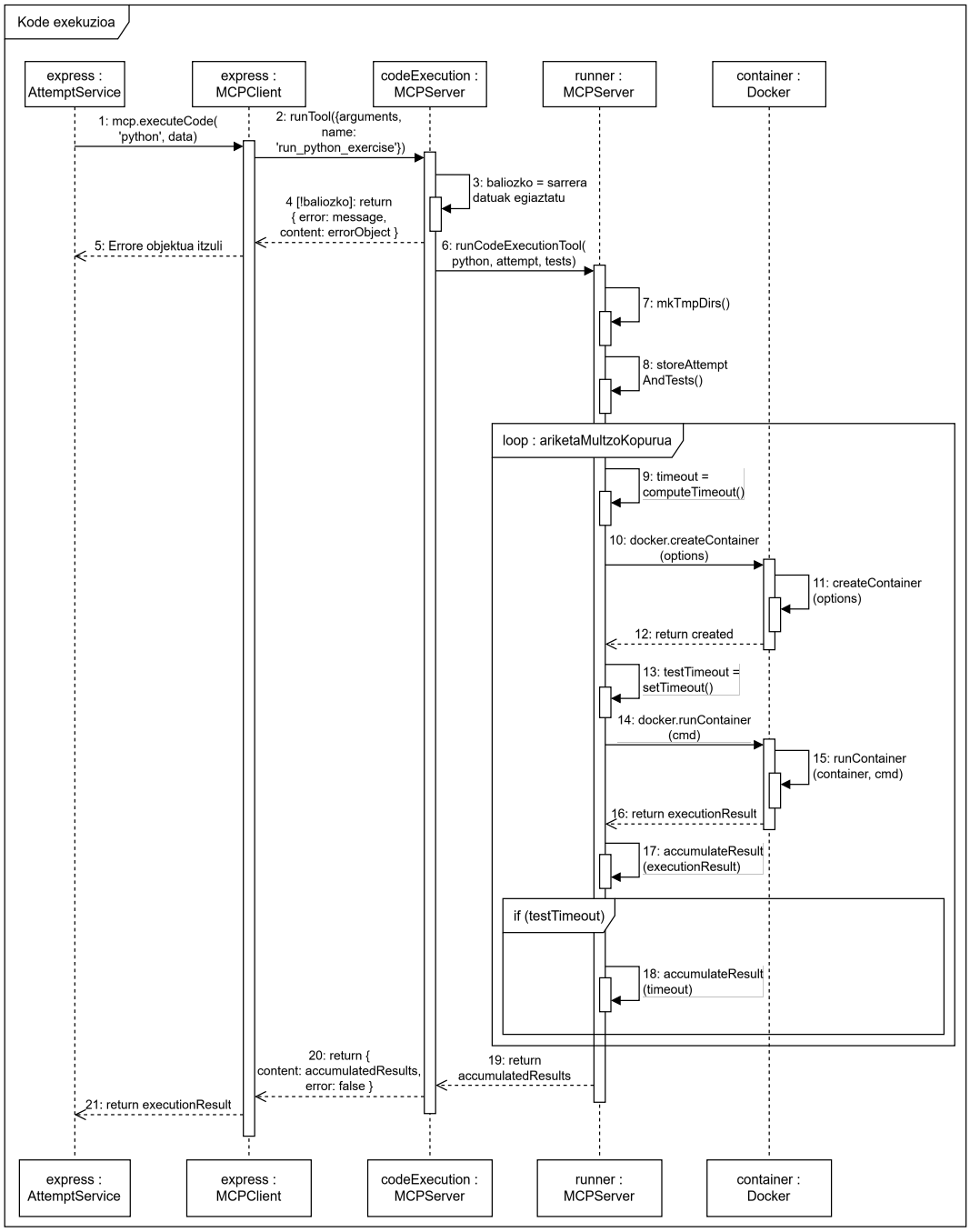
Adibide praktiko gisa `run_python_exercise` tresnaren deiaren exekuzio-fluxua 6.1 Irudian ikusi daiteke. Jarraian, exekuzioaren fluxu nagusia azaltzen da:

1. Backendak MCP bezeroa erabiltzen du MCP zerbitzariaren tresna (tool) deitzeko.
2. Kode exekuzio MCP zerbitzariak sarrera parametroak baliotzen ditu.
3. Kode exekuzio MCP zerbitzariak ikaslearen iturburu-kodea eta proba unitarioak behin-behineko karpetetan gordetzen ditu.
4. Proba unitario bakoitza indibidualki exekutatzen da:
   a) Exekuzioaren muga-denbora kalkulatzen da.
   b) Docker edukiontzia sortzen da.
   c) Proba unitarioak Docker edukiontzian exekutatzen dira.
      - Exekuzio emaitza gorde egiten da.
      - Exekuzio muga-denbora gainditu bada emaitza oker orokorra gordetzen da.
5. Kode exekuzio MCP zerbitzariak MCP bezeroari exekuzio emaitzak itzultzen dizkio.
6. MCP bezeroak backendari emaitzak itzultzen dizkio.

6.3.3 Segurtasun kontrola

Ikasleak bidali duen iturburu-kodea exekutatzean sistema blokeatu ez dadin, aurreko atalean azaldu denaz gain, edukiontzi bakoitzari ematen zaizkion baliabideak eta proba multzo bakoitzaren exekuzio denbora mugatu da.

- Exekutatzen den proba multzo bakoitzak gaindituz gero tresnaren (tool) exekuzio osoa gelditzen duen *timeout* indibiduala du.
- Tresnen exekuzioak gehienezko 5 minutu iraungo dute.
- Edukiontziei sistemaren baliabideak mugatu zaizkie:
  - Memoria konpilazio fasean eta exekuzio fasean 1 GiB eta 512 MiB dira hurrenez hurren.
  - Gehienez 0.5 nukleoko prozesamendu-ahalmena du.
  - Gehienez 128 prozesu sor ditzake.
  - Linuxen gaitasun —ingelesez, *capability*— guztiak baztertzen dira.



6.1 Irudia: `run_python_exercise` tresnaren deiaren exekuzio-fluxuaren sekuentzia-diagrama.

– Pribilegio berriak eskuratzea mugatzen du.

– Bakarrik `/tmp` eta `/opt/code-execution/out` ibilbideetan idatzi daiteke.

– Internetekin konektatzea debekatzen da.

6.4 Frontendaren inplementazioa

Frontenda inplementatzeko React frameworka erabili da SPA bat sortzeko [43]. React Document Object Model-eko (DOM) elementuen egoera *(state)* gogoratzen duen User Interface-ak (UI) eraikitzeko liburutegia da. React liburutegiari React Router liburutegia gehitzean, nabigatzeko ibilbideak erabiltzen dituen webgune konbentzional bat simulatu daiteke eta SPA egitura mantendu.

Reactek frontendari egitura ematen dion bitartean, UIaren itxura diseinatzeko Tailwind CSS erabili da. Liburutegi honek aurredefinitutako utilitate-klaseak eskaintzen ditu eta horri esker, ez da CSS estiloko klaserik zerotik hasi [44]. Tailwind web-orrien garapena azkartzeko erabili da, DOMeko elementu bakoitzaren estiloak elementuan bertan adierazteko.

Frontendaren orri gehienen inplementazioan Reactek eskaintzen dituen *hook*-ak erabili dira[2]. Hook asko erabili dira eta ez du interes handirik erabilitako hook guztiak zehazki adieraztea; beraz, hook mota nagusi bakoitza zertarako erabili den azaltzen da jarraian.

`useState`
: Aldagaien egoera kudeatzeko. Adibidez, `isLoading`-ek orrialdea kargatu den adierazten du eta `setIsLoading` `isLoading` aldagaia eguneratzeko erabiltzen da.

```
const [isLoading, setIsLoading] = useState<boolean>(false);
```

`useEffect`
: DOMean aldaketak gertatzen direnean kodea exekutatzeko. Adibidez, `isSending`, `loadingError` edo `isLoading` aldagaiaren balioa aldatzen denean edo orrialdearen lehenengo errenderizazioan, lehenengo argumentuko funtzio anonimoa exekutatzen da.

```
useEffect(() => {
    if (!isSending && !loadingError && !isLoading) {
        textareaRef.current?.focus();
    }
}, [isSending, loadingError, isLoading]);
```

`useMemo`
: Kalkulu garestien emaitzak memorian gordetzeko eta bakarrik dependentziak aldatzen direnean birkonputatzeko. Adibidez, aurrerapen dashboardean hautatutako saiakeren orrialdekatzeko tartea bakarrik hautatutako zenbakia edo saiakera zerrenda aldatzen direnean birkalkulatzeko, eta ez errenderizazio guztietan.

```
const paginatedAttempts = useMemo(() => {
    const startIndex = (currentPage - 1) * MAX_ATTEMPTS_PER_PAGE;
    return (
        dashboardData?.lastAttempts.slice(
            startIndex,
            startIndex + MAX_ATTEMPTS_PER_PAGE,
        ) ?? []
    );
}, [currentPage, dashboardData]);
```

`useCallback`
: Funtzioen erreferentzia memorian gordetzeko eta bakarrik dependentziak aldatzen

[2] React hook-ak Reacten barne bizi-zikloa eta egoera kudeatzeko funtzioak dira.

direnean birkonputatzeko. Pantaila berriro errenderizatzean React osagaien barruan definitutako funtzioak birdefinitu egiten dira, eta funtzio horietako bat *useEffect*-en dependentzia bada errenderizazio guztietan *useEffect*-a berriro exekutatuko da. Adibidez, kautotze prozesuan access token berria memorian gordetzeko `setToken` funtzioa erabiltzen da. Funtzioa ez badugu `useCallback` batean biltzen jarraian agertzen den `useEffect`-en barruko funtzio anonimoa errenderizazio guztietan exekutatuko da.

```
const setToken = useCallback((newToken: null | string) => {
    setAccessToken(newToken);
    setToken_(newToken);
}, []);

useEffect(() => {
    const initializeAuth = async () => {
        try {
            if (!getAccessToken()) {
                await refreshAccessToken();
                setToken(getAccessToken());
            }
        } catch {
            setToken(null);
            console.error("Error refreshing access token");
        } finally {
            setIsLoading(false);
        }
    };
    initializeAuth();
}, [setToken]);
```

useRef

Balioak errenderizazio bizi-ziklotik kanpo gordetzeko: DOMeko elementuei erreferentzia errenderizazioen artean eskuragarri izateko, edo errenderizazioen artean aldagaien balioak sinkronoki eguneratzeko, asinkronoki beharrean. Adibidez, *dropdown* elementu baten menuaren kanpoaldean klik egiterakoan, dropdown elementua ixteko erabiltzen da.

```
const menuRef = useRef<HTMLDivElement>(null);

useEffect(() => {
    if (!isOpen) {
        return;
    }
    const handleMouseDown = (event: MouseEvent): void => {
        const target = event.target as Node;
        if (
            menuRef.current &&
            !menuRef.current.contains(target)
        ) {
            setIsOpen(false);
        }
    };
    document.addEventListener("mousedown", handleMouseDown);
    return () => {
        document.removeEventListener("mousedown", handleMouseDown);
    };
}, [isOpen]);
```

6.4.1 Kautotze kudeaketa

Frontenda backendarekin komunikatzeko Axios liburutegia erabili da [45]. Backendari eskaera bat bidaltzean, ez da *fetch* bat erabiltzen eskaera egiteko, baizik eta aldez aurretik sortutako Axios instantzia bat erabiltzen da. Nahiz eta instantzia honek abstrakzio maila bat gehitu, kodea ez errepikatzea ekartzen du. Abstrakzio maila hau ez bada erabiltzen, eskaera bakoitzak kautotze informazioa eta HTTP burukoak txertatu beharko lituzke. Horrez gain, kautotzeak akats egiten duenean, eskaera bakoitzak access token berria eskuratzeko prozesu guztia idatzi beharko luke. Axios erabiliz, kautotze kudeaketaren ardura kanporatu eta prozesu osoa zentralizatu egiten da.

6.4.2 Webgunearen orrialdeak

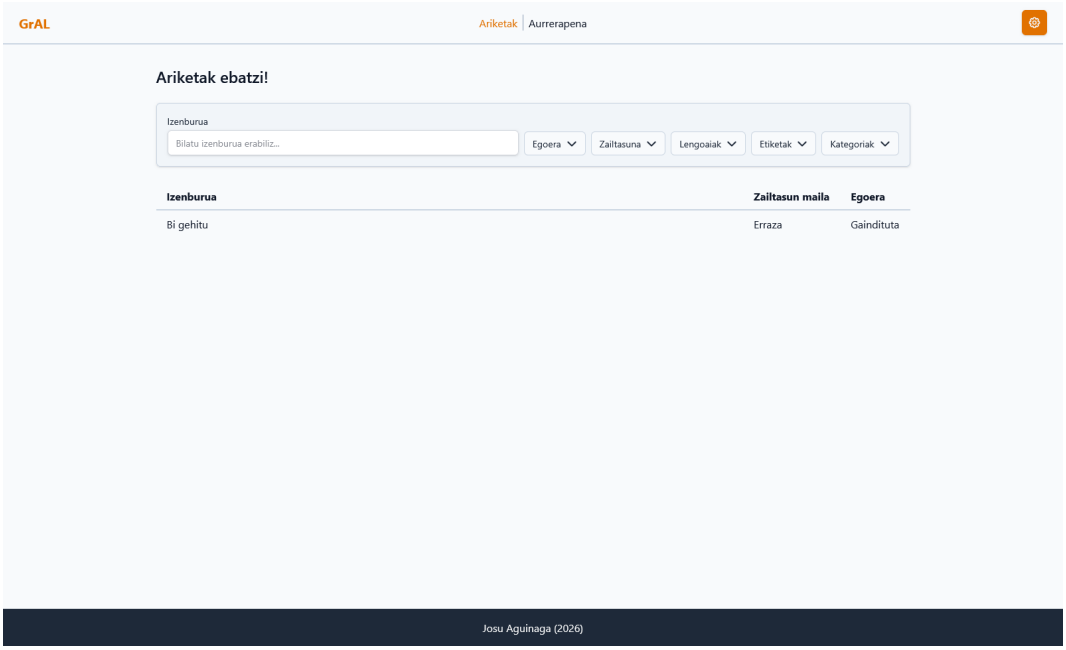
Kautotze eraginkorra inplementatu ondoren, frontendeko orrialdeak sortu dira. Webguneak lau orrialde nagusi ditu: ariketen zerrenda, lan-ingurunea, aurrerapena eta ikaslearen profila.

Ariketen zerrendaren orrialdea
Ikasleari aplikazioan gorde diren ariketen informazioa zerrenda batean erakusten dio. Ikasleak ariketa batean klik egiterakoan lan-ingurunean ariketa irekiko du. Orrialdeak bi osagai nagusi ditu: ariketak bilatzeko iragazkien barra eta ariketen informazioa erakusteko zerrenda. Orrialdearen interfazea 6.2 Irudian ikusi daiteke.

Lan-ingurunearen orrialdea
Ikasleak gehien erabiliko duen orrialdea da. Bertan, ariketa zehatz bat ikusi eta lana egiteko aukera izango du. Lan-inguruneak 5 osagai nagusi izango ditu.

- Ariketaren enuntziatua eta metadatuak dituen fitxa.
- Ikaslearen kodearen exekuzioak sortzen duen emaitzaren berrelikadura jasotzeko fitxa.
- Ikasleak iraganean ariketa zehatz horretan egin dituen saiakeren zerrendaren fitxa. Saiakera batean klik egitean, saiakera horri dagokion iturburu-kodea kargatuko da kode editorean.
- Ikasleak ariketaren soluzioa idazteko kode editorea. Zehazki, VSCode-k erabiltzen duen Monaco kode editorea erabili da [46]. Kode editore bat zerotik inplementatzeak denbora asko behar du. Monaco erabiliz, ordea, industrian probatuta dagoen eta ondo funtzionatzen duen editore bat dugu. Konfigurazio gutxi batzuk eginda hainbat funtzionalitate gehigarri dituen editorea dugu: hautatutako programazio-lengoaiaren arabera sintaxia nabarmendu eta aurretik idatzitako testuan oinarritutako kode iradokizunak eskaini. Kontuan hartu behar da Language Server Protocol-rik (LSP) ez dela exekutatuko, lanean ari den ikasle bakoitzarentzat instantzia bat sortu beharko litzatekeelako eta baliabide asko kontsumituko lituzkeelako. Beraz, LSPek ematen dituzten funtzionalitateak ez dira erabilgarri egongo; besteak beste, kode-iradokizun aurreratuak eta definizioen ikusketa.
- Ikasleak LLMarekin komunikatzeko erabiliko duen laguntza txata.

Orrialdearen interfazea 6.3 Irudian ikusi daiteke.



6.2 Irudia: Frontendaren ariketak orrialdeko GUIa.

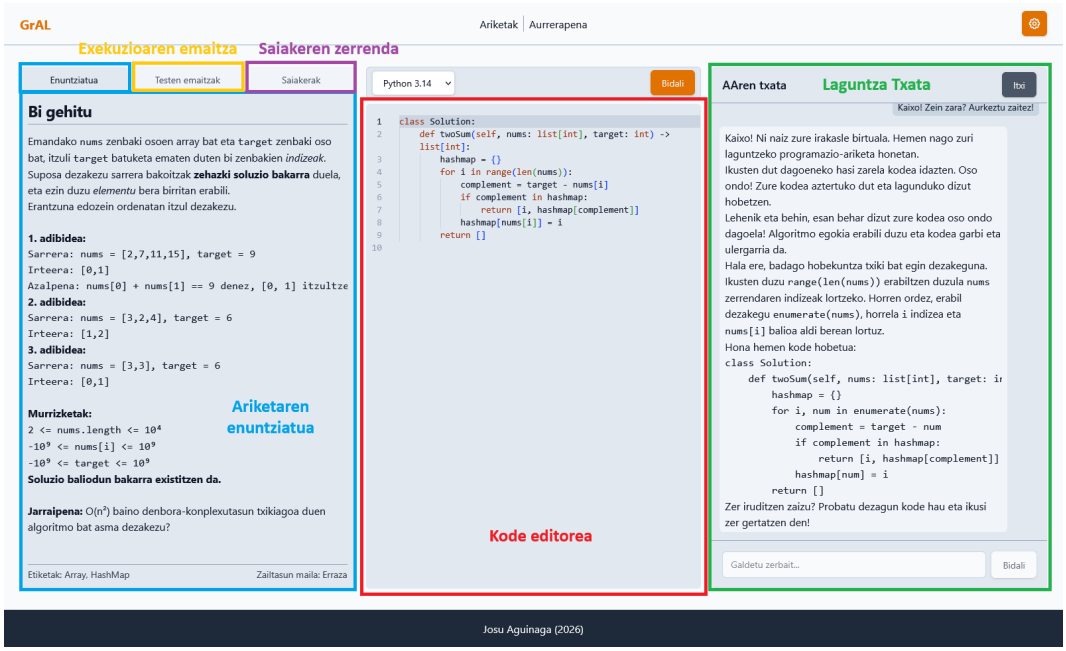
Ikaslearen aurrerapenaren orrialdea
Ikasleak iraganean egindako ariketen aurrerapena erakusten duen orrialdea da. Orrialdeak hiru osagai nagusi ditu: zailtasun maila bakoitzeko ikasleak ondo egin dituen ariketa kopurua, ikaslearen egungo ikaskuntza maila eta ikasleak iraganean egindako saiakeren zerrenda bat. Ikasleak saiakera batean klik egitean lan-ingurunea irekiko da eta bertan saiakeraren iturburu-kodea kargatuko da. Orrialdearen interfazea 6.4 Irudian ikusi daiteke.

Ikaslearen profilaren orrialdea
Ikaslearen eta ikaslearen kontuaren informazioa kudeatzeko orrialdea da. Orrialdean ikaslearen informazioa kudeatzeko hainbat formulario erakusten dira.

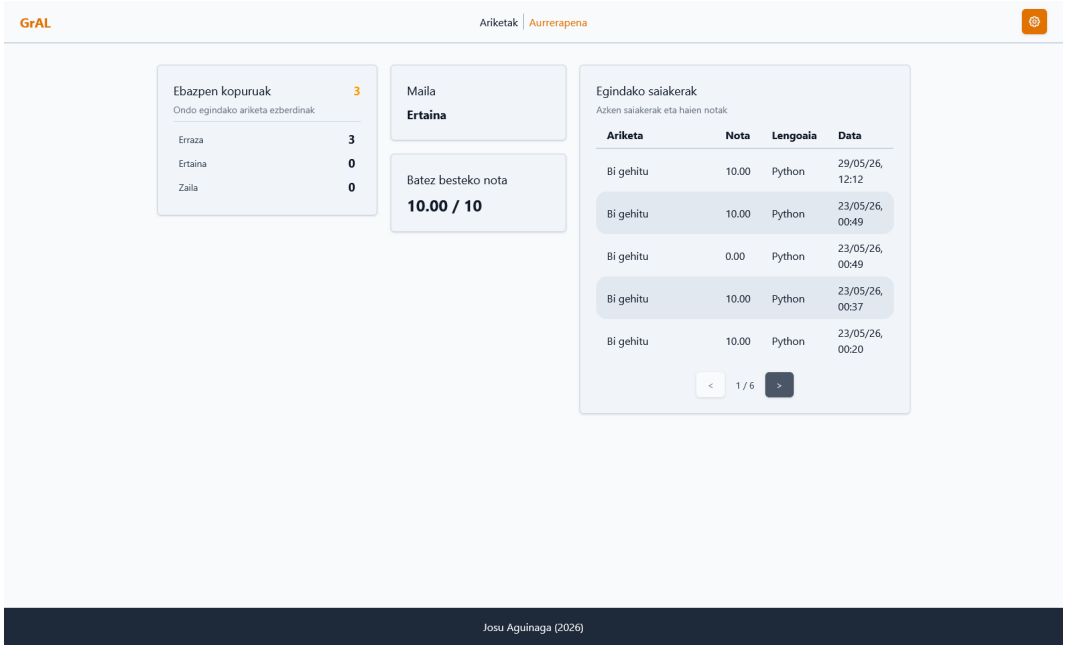
- Ikasleak datu hauek aldatzeko aukera du: erabiltzaile izena, pasahitza, ikasketa maila, ikaslearen ikasketa mailari bakarrik dagozkion ariketak ikustea nahi duen eta ikaslearen gogoko programazio-lengoaia.
- Gainera, ikasleak edozein momentuan eta itxaron behar izan gabe sistematik informazio pertsonala ezabatzeko aukera du. Zehazki, LLMarekin izandako elkarrizketak ezabatu ditzake, ariketen saiakeren historia ezabatu dezake, edo, zuzenean, kontua ezabatu dezake.

Orrialdearen interfazea 6.5 Irudian ikusi daiteke.

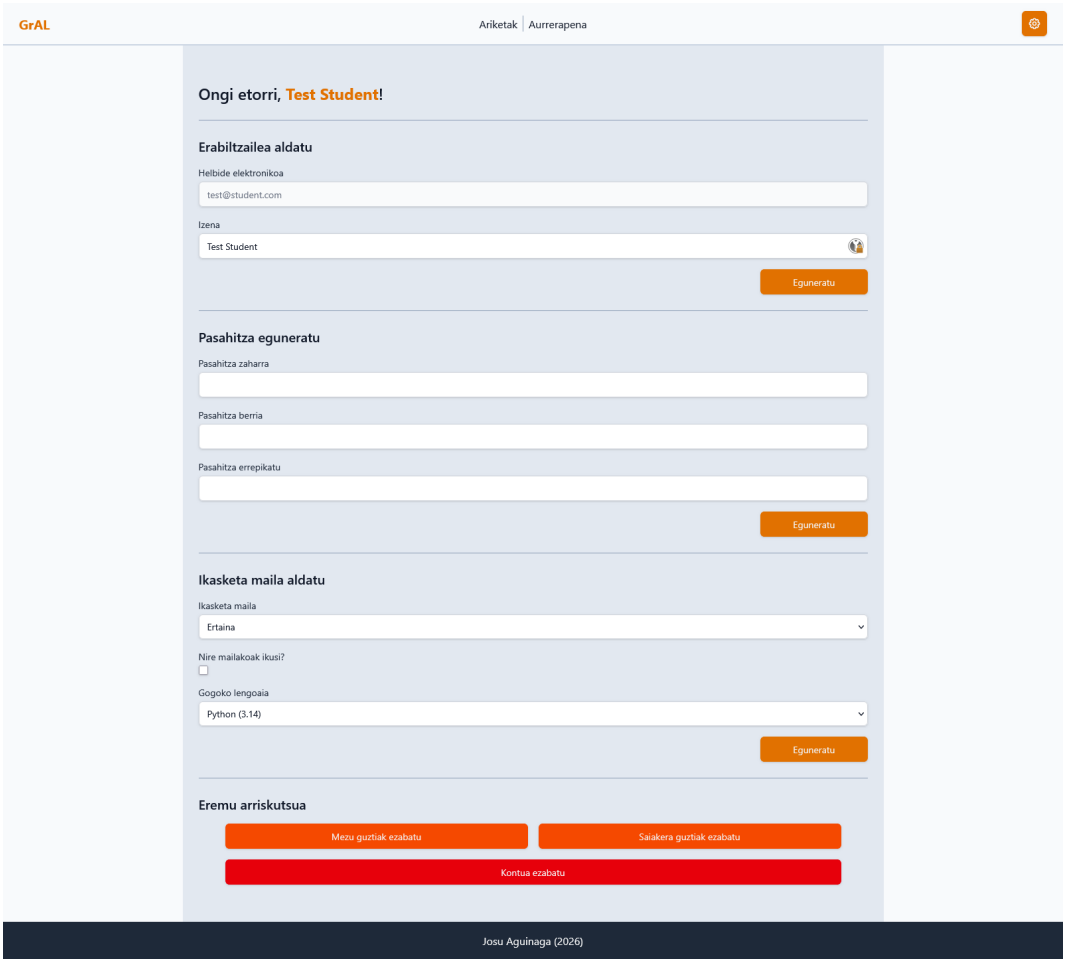
Laburbilduz, kapitulu honetan softwarearen inplementazioaren erabaki nagusiak azaldu dira: datu-basea mantengarria izateko nola inplementatu eta hasierako datuak nola kargatu, CSR patroia jarraituz backendaren inplementazioa eta hartutako segurtasun neurriak, kodea modu seguruan exekutatzeko MCP zerbitzariaren inplementazioa eta Dockeren erabilera, eta frontenda inplementatzeko Reacten erabilpena.



6.3 Irudia: Frontendaren lan-ingurune orrialdeko GUIa.



6.4 Irudia: Frontendaren ikaslearen aurrerapen orrialdeko GUIa.



6.5 Irudia: Frontendaren profila orrialdeko GUIa.

KAPITULUA 7

Jarraipen eta kontrola

Kapitulu honetan proiektuaren garapen egokia mantentzeko egin diren kudeaketa lanak adierazten dira. Lehenengo, proiektuaren irismenaren kudeaketa eta eguneraketak aipatzen dira. Ondoren, proiektuaren ataza ezberdinak egiteko erabili den denbora eta izandako desbiderapenak alderatzen dira. Azkenik, proiektuaren garapenaren zehar gertatu diren arriskuak eta haien eraginak zerrendatzen dira.

7.1 Irismenaren kudeaketa

GrAL honen zati garrantzitsu bat teknologia berrien azterketa eta erabilera izan da. Ondorioz, hasiera batean proiektuaren planifikazioan (3 kapitulua) definitu zen irismena ez zen guztiz zehatza. Proiektu honetan egin den web-plataforma garatzeko atazak egin ahala —batez ere, betekizunen bilketa egindakoan— proiektuaren tamaina handitu egin da, eta GrALaren garapen denbora onargarria mantentzeko proiektuaren irismena mugatzea erabaki da. Hala ere, baztertutako funtzionalitateak 8 kapituluan hobekuntza gisa egiteko utzi dira. Jarraian, proiektuaren irismenaren eguneraketak azaltzen dira:

Irismena mugatu
: **I.Pr** lan-paketeak barneratzen dituen atazak egin gabe utzi dira.

Betekizun-funtzionalak mugatu
: Backend zerbitzariaren inplementazioa gehiegi ez luzatzeko eta estimatutako denboraldien barnean mantentzeko, **BF-1** eta **BF-2** betekizun funtzionalek zehazten duten erabiltzaileen egiaztapena, eta **BF-4** betekizun funtzionala (pasahitza berreskuratu) baztertu egin dira. Hiru funtzionalitate hauek posta elektronikoen bidalketa dute amankomunean.

7.2 Garapen denboren kudeaketa

Jarraian aurkezten den 7.1 Taulan proiektuaren atazak garatzeko erabili diren denbora errealak eta desbiderapenak adierazten dira. Orokorrean, desbiderapen nagusiak bost lan-atazetan gertatu dira: IB.MCP, Di.BB, I.F, I.B eta LA.M.

7.1 Taula: Proiektuko lan-atazen garapen denbora errealak eta desbiderapenak.

Lan-paketea	Estimatutako denbora (h)	Eskainitako denbora (h)	Desbiderapena (h)
Informazio Bilaketa (IB) adarra	40	44	+4
IB.AE	10	12	+2
IB.MCP	15	22	+7
IB.PostgreSQL	5	2	-3
IB.React	10	8	-2
Diseinua (Di) adarra	38	40	+2
Di.BB	15	20	+5
Di.A	10	10	0
Di.DBD	8	8	0
Di.UU	5	2	-3
Inplementazioa (I) adarra	150	150	0
I.IK	5	3	-2
I.F	45	55	+10
I.B	55	70	+15
I.MCP	20	16	-4
I.DB	10	6	-4
I.Pr	15	0	-15
Kudeaketa (K) adarra	30	27	-3
K.P	15	15	0
K.JK	15	12	-3
Lan Akademikoa (LA) adarra	70	115	+35
LA.M	60	95	+35
LA.De	10	10	0
Guztira	328	376	+48

MCP protokoloa eta inguratzen dituen teknologiak (IB.MCP) barneratzen dituzten lan-atazak estimatutakoa baino % 46 denbora gehiago hartu du.

Di.BB, I.F eta I.B lan-atazek izandako desbiderapenak ere esanguratsuak izan dira. Aurreko atalean azaldu den bezala, eraiki den web-plataformak funtzionalitate asko ditu eta planifikazioa egiterakoan ez ziren guztiak aurreikusi. Honek, bai diseinuan, bai inplementazioan, estimatutako lan-karga handitu du eta atazak egiteko denbora gehiago behar izan da. Zehazki, % 33, % 22 eta % 27 denbora gehiago hartu dute hurrenez hurren.

Memoriaren garapenak, berrikusketak eta zuzenketak estimatutako baino % 58 denbora gehiago behar izan du.

Bestaldetik, lan-ataza hauek espero baino errazagoak izan dira eta estimatutako denbora erdian egin dira: IB.PostgreSQL, Di.UU eta I.DB.

Guztira, GrAL honen garapenean 376 ordu erabili dira, estimatutakoa baino % 14 ordu gehiago.

7.3 Arriskuen kudeaketa

Atal honetan GrALaren garapenean zehar gertatu diren arriskuak zerrendatzen dira eta bakoitzak izandako eragina adierazten da.

A2. Teknologia berrien ikaskuntza

MCP protokoloaren espezifikazioaren irakurketak eta ulermenak, inplementazioen adibideen bilaketak, eskuragarri dauden SDKen azterketak eta memorian idatzitako protokoloaren azalpenak estimatutako baino denbora gehiago behar izan dute.

Eragina

Web-plataformaren arkitektura eta diseinu atazak pare bat egun atzeratu dira.

Irtenbidea

Denbora berreskuratzeko, garapen mugarrien aurretik utzitako denbora-tartea erabili da.

A4. Gaixotasuna

Proiektuaren betekizunen bilketa egitean osasun arazoak izan dira.

Eragina

Osasun arazoengatik atazen garapena aste bat atzeratu da.

Irtenbidea

Galdutako denbora berreskuratzeko garapen mugarrien aurretik utzitako denbora-tartea erabili da.

KAPITULUA 8

Ondorioak

Kapitulu honetan proiektuan lortu diren emaitzak eta ondorioak aztertzen dira. Horrez gain, GrAL honen garapenaren zehar ikasi dena azaltzen da. Amaitzeko, proiektua zabaltzeko etorkizunean egin daitezkeen hobekuntzak adierazten dira.

8.1 Emaitzak eta ondorioak

GrAL honetan ikasleek programazio ariketak egiteko web-plataforma oso bat diseinatu eta eraiki da. Inplementatuko web-plataformak hiru geruzatan oinarritutako bezero-zerbitzari arkitektura modularra jarraitzen du, eraginkortasuna eta eskalagarritasuna bermatuz. Diseinatutako arkitektura honek erantzukizunen banaketa totala ematen du.

Frontendak erabiltzailearekin elkarreragiten du, eta backendaren APIa erabiliz erabiltzailearen eta ariketen datuak eskuratzen ditu, eguneraketa lanak egiten ditu eta ikasleari feedback pedagogikoa eskaintzen dio.

Backend zerbitzariak testuinguru orkestratzaile gisa jarduten du. Batetik, ariketen, ikasleen eta saiakeren datuak bermatzeko datu-basearen instantziarekin komunikatzen da. Bestetik, ikaslearen ikasketa mailara egokitutako feedback pedagogikoa sortzeko LLMaren APIarekin komunikatzen da.

Ikasleen kodea exekutatzeko backenda kode exekuzio MCP zerbitzariarekin komunikatzen da. Docker ingurunea erabiliz kode exekuzioaren isolamendua lortu da, segurtasuna bermatuz eta exekuzio denborak mugatuz.

Orokorrean, plangintzan definitutako eskakizun gehienak arrakastaz bete dira eta eraikitako softwarea GitHuben eskuragarri jarri da[1]. Hala ere, GrALa garatzeko ofizialki 300 ordu aurreikusita daude, eta denbora esparru horretatik gehiegi ez urruntzeko jarraian zerrendatzen diren atazak ezin izan dira burutu:

- **I.Pr** lan-paketea garatzea baztertu egin da. Honek ez du esan nahi plataformaren funtzionamendu egokia ez dela egiaztatu, baizik eta proba unitarioak ez direla esplizituki idatzi.

[1] Sortu den aplikazioaren iturburu-kodea hemen aurkitu daiteke: https://github.com/Josu-A/gral

- Ez da inplementatu backendak posta elektronikoak bidaltzeko sistema. Beraz, **BF-4** betekizun funtzionala, eta **BF-1** eta **BF-2** betekizun funtzionalen erabiltzaile kontuen egiaztapena ez dira inplementatu. Hots, erregistratzean erabiltzaileek ez dute egiaztapen prozesurik jarraitu behar.

8.2 Ikasitakoa

Proiektu honen garapenak ezagutza berrien ekarpen pertsonala izan du. Zehazki, proiektuan zehar honakoa ikasi da:

Ezagutza teknikoa
: Datu-basea diseinatu eta inplementatzeko PostgreSQL eta Prisma ikasi dira. Frontenda garatzeko React eta Tailwind ikasi dira. Backenda garatzeko JWT kautotze tokenen erabilera, segurtasuna bermatzeko ekintzak, TypeScript programazio-lengoaia, Zod balidazio liburutegia, LLMaren APIaren deien inplementazioa eta MCP bezeroen sorrera ikasi dira. Kode exekuzio MCP zerbitzaria inplementatzeko MCP zerbitzarien sorrera eta kodea isolamenduan exekutatzeko Docker edukiontzien erabilera ikasi dira.

Kudeaketa
: Hainbat interesdun dituen eta iraupen luzea duen proiektua kudeatzen ikasi da: plangintza egitea eta eguneratzea, arriskuen analisia, betekizunen bilketa, interesdunen arteko komunikazio eraginkorra eta bileren kudeaketa.

Trebetasuna
: Aurrekoaz gain, lana bakarka egin eta kudeatzeko diziplina eskuratu da, etorkizunerako oso erabilgarria izango dena.

8.3 Hobekuntzak eta hurrengo urratsak

Amaitzeko, etorkizunean proiektua hobetzeko egin daitezkeen proposamen batzuk adierazten dira, 8.1 atalean azaldutako egin gabeko funtzionalitateak barne.

Pasahitza berreskuratu *(BF-4)*
: Erabiltzaileak pasahitza ahaztu badu pasahitza berria sortzeko aukera izango du.

Erabiltzailea egiaztatu *(BF-1 eta BF-2)*
: Erabiltzailea erregistratzerakoan helbide elektronikoa erabiliz kontua egiaztatu beharko du.

Testuinguru MCP zerbitzariak *(2. iterazioa)*
: LLMaren APIari momentuan behar duen testuingurua bakarrik bidaltzeko ikaslearen aurrerapena eta ariketen informazioa MCP zerbitzariak sortuko dira.

Ariketak eta proba unitarioak gehitu
: Backend zerbitzariaren endpoint nagusien funtzionamendu egokia bermatzeko proba unitarioak gehituko dira.

RFC 9449 espezifikazioa inplementatu
Demonstrating Proof of Possession (DPoP) mekanismoa inplementatuz access eta refresh tokenak gako publiko eta pribatuekin babesten dira. Honela, tokenak erabiltzeko eskaera bakoitza gako pribatuarekin sinatu behar denez, aktore gaizto batek tokenak lapurtuko balitu ezin izango ditu erabili gako pribatua gabe [47].

LLMaren API gehiago gehitu eta automatikoki aldatu
Backendak, testuinguru orkestratzaile gisa, LLMen API anitzen konexioa kudeatuko du, eta unean erabiltzen dagoena erortzen bada automatikoki ordezkatuko du.

LSPak gehitu
Monaco kode editoreak programazio lengoaien egituren eta liburutegien kode iradokizun aurreratuak eskaintzeko, erabiltzaile bakoitzarentzat hautatu daitekeen programazio-lengoaiaren LSP instantzia bat exekutatu beharko da.

Frontendan iragazitako ariketen eskaerak *cache*an gorde
Erabiltzaileak aurreko unean zehaztutako iragazi-multzo bat berriz aukeratzen badu, cachean gordetako ariketa zerrenda erakutsiko zaio, backendarekin komunikatu gabe.

Datu-basearen egoera cachean gorde
Redis erabiliz datu-baseko taulen kopia bat memorian gordeko da, eta datu-basearen atzipenak murrizteko datuak bertatik eskuratuko dira.

Mugikorreko aplikazioa sortu
Web-plataforma mugikorretan aplikazio gisa erabiltzeko React Native erabiliz inplementatuko da.

Gai iluna gehitu
Web-plataformak erabiltzailearen hobespenak jarraituz GUIaren gaia ezarriko du: argia edo iluna. Horrez gain, erabiltzaileak gaia aldatzeko aukera izango du.

i18n (internalizazioa) eta l10n (lokalizazioa)
Web-plataforma hizkuntza ezberdinetan eskuragarri egongo da.

ARIA
Laguntza-teknologikoak behar dituzten pertsonen esperientzia hobetzeko frontendeko HTML elementuek *aria* atributuak izango dituzte.

Legediak eta araudiak
Etorkizunean web-plataforma produkzioan jartzen bada, sistemak legeak eta arautegiak zorrotz bete beharko ditu. Adibidez, plataforma edozein adieneko ikasleek erabiltzeko aukera duenez, Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) [48] eta General Data Protection Regulation (GDPR) [49] legediak zorrotz bete beharko dira.

ERANSKINA A

Sekuentzia-diagramak

Eranskin honetan sistemaren erabilpen-kasuen sekuentzia diagramak erakusten dira.

A.1 BF-1: Erregistratu

Ikusi A.1 Irudia erregistratu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.2 BF-2: Saioa hasi

Ikusi A.2 Irudia saioa hasi erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.3 BF-3: Saioa itxi

Ikusi A.3 Irudia saioa itxi erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.4 BF-5: Profila ikusi

Ikusi A.4 Irudia profila ikusi erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.5 BF-6: Datu pertsonalak editatu

Ikusi A.5 Irudia datu pertsonalak editatu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.6 BF-7: Pasahitza aldatu

Ikusi A.6 Irudia pasahitza aldatu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.7 BF-8: Ikasketa maila aldatu

Ikusi A.7 Irudia ikasketa maila aldatu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.8 BF-9: Kontua ezabatu

Ikusi A.8 Irudia kontua ezabatu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.9 BF-10: Saiakerak ezabatu

Ikusi A.9 Irudia saiakerak ezabatu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.10 BF-11: Mezuak ezabatu

Ikusi A.10 Irudia mezuak ezabatu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.11 BF-12: Aurrerapena begiratu

Ikusi A.11 Irudia aurrerapena begiratu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.12 BF-13: Ariketak zerrendatu

Ikusi A.12 Irudia ariketak zerrendatu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.13 BF-14: Ariketak bilatu

Ikusi A.13 Irudia ariketak bilatu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.14 BF-15: Ariketa aukeratu

Ikusi A.14 Irudia ariketa aukeratu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.15 BF-16: Ariketa lengoaia aldatu

Ikusi A.15 Irudia ariketa lengoaia aldatu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.16 BF-17: Saiakerak ikusi

Ikusi A.16 Irudia saiakerak ikusi erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.17 BF-18: Saiakera bidali

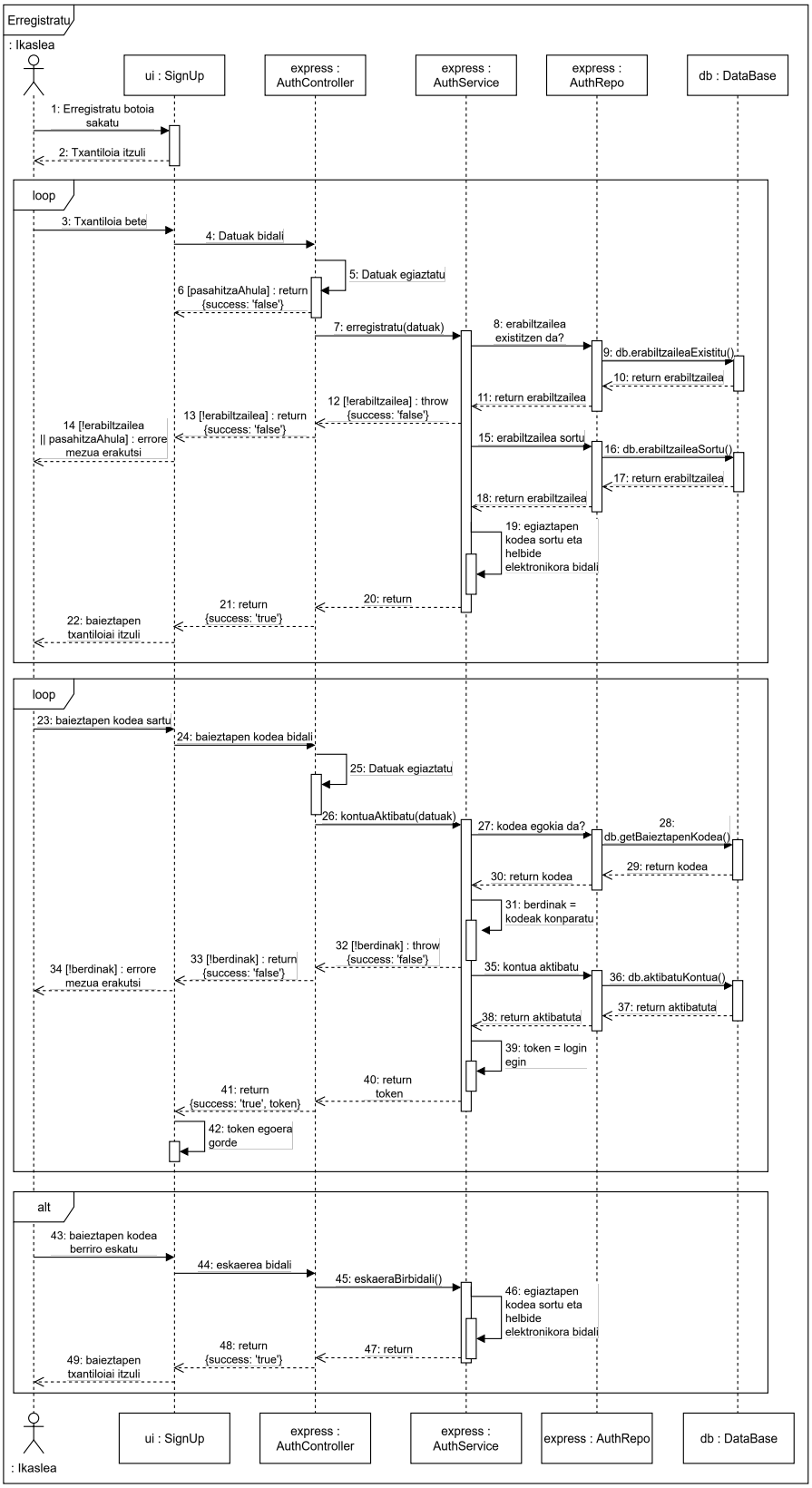
Ikusi A.17 Irudia saiakera bidali erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.18 BF-19: Saiakera ireki

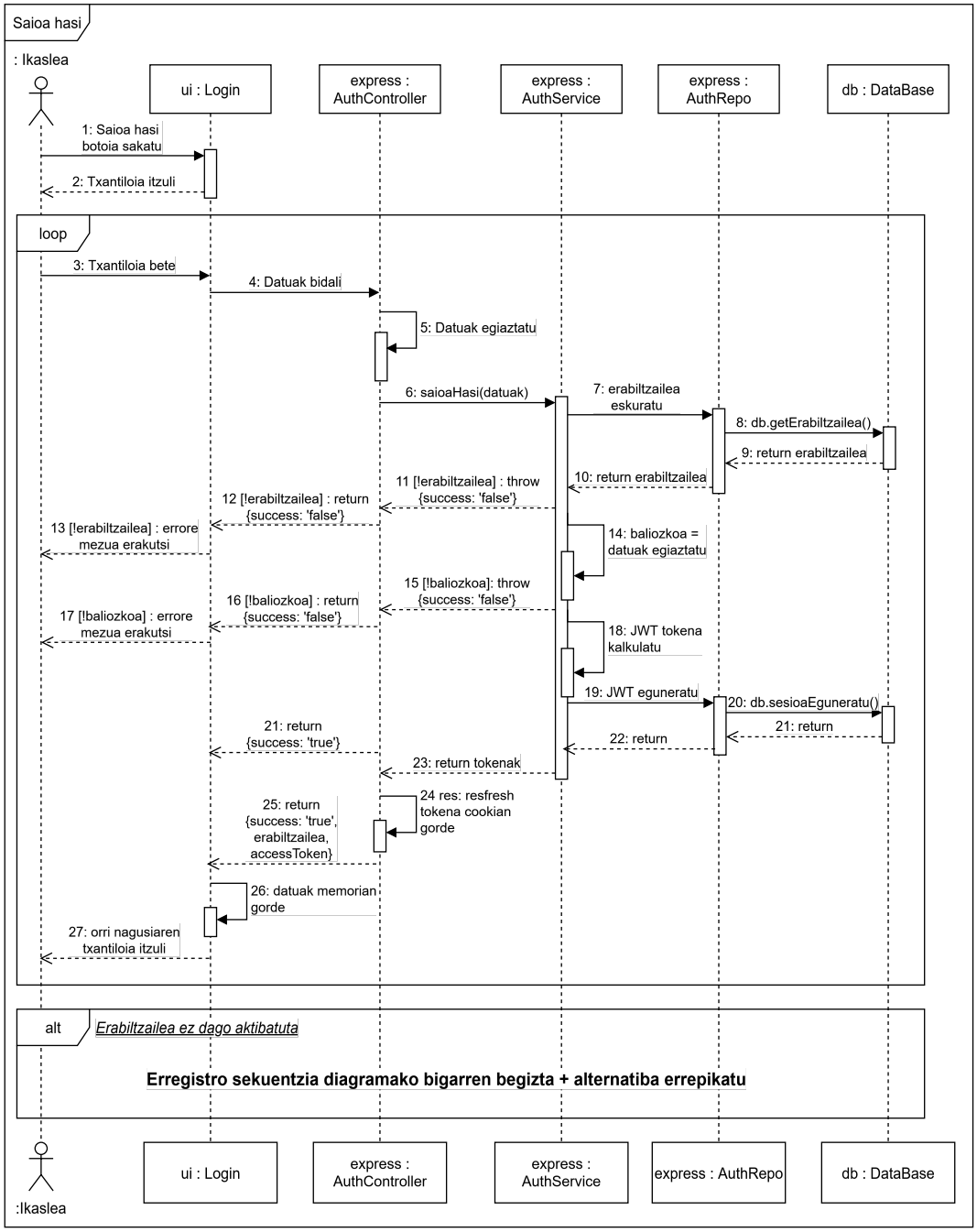
Ikusi A.18 Irudia saiakera ireki erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.

A.19 BF-20: Laguntza eskatu

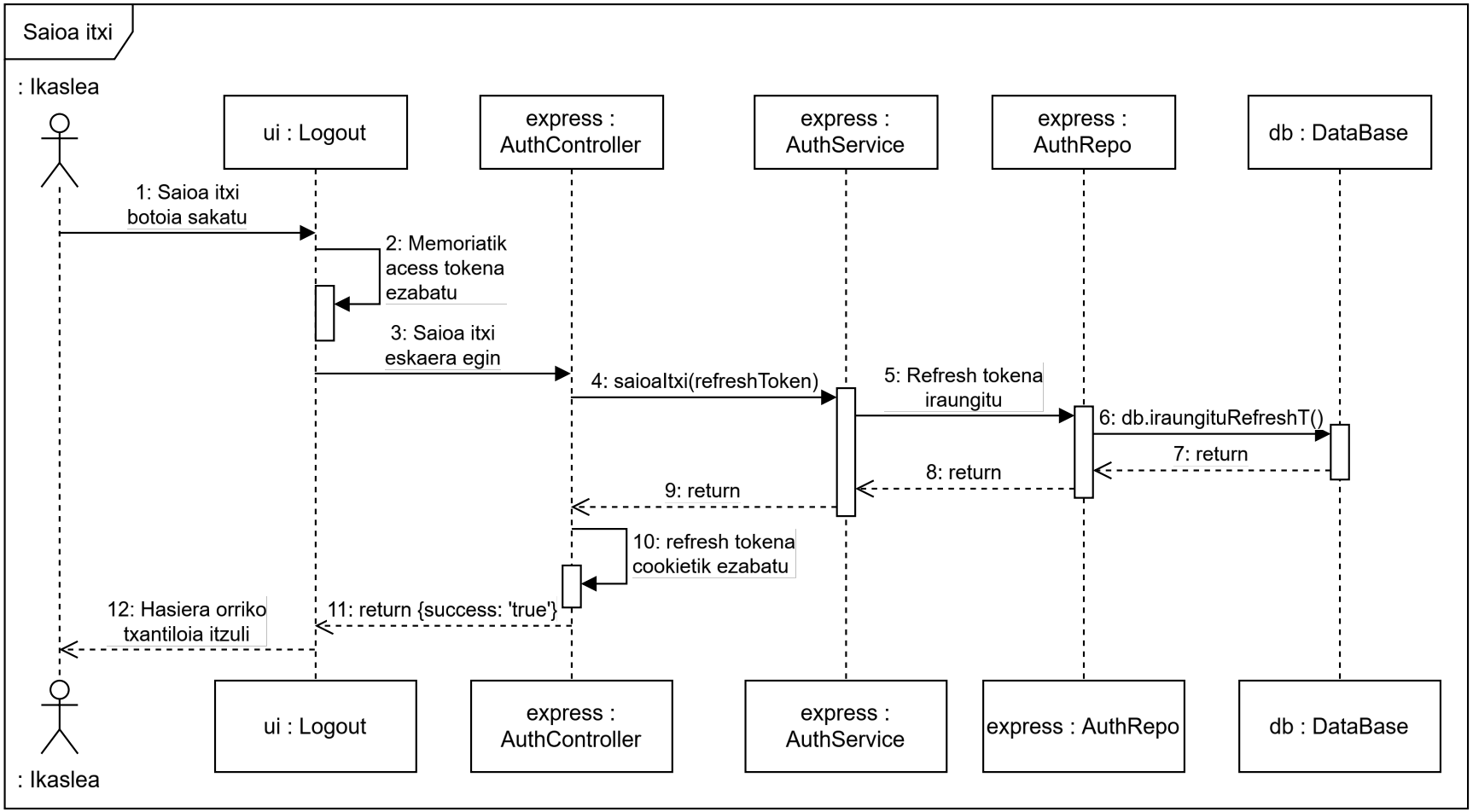
Ikusi A.19 Irudia laguntza eskatu erabilpen-kasuari dagokion sekuentzia diagrama aztertzeko.



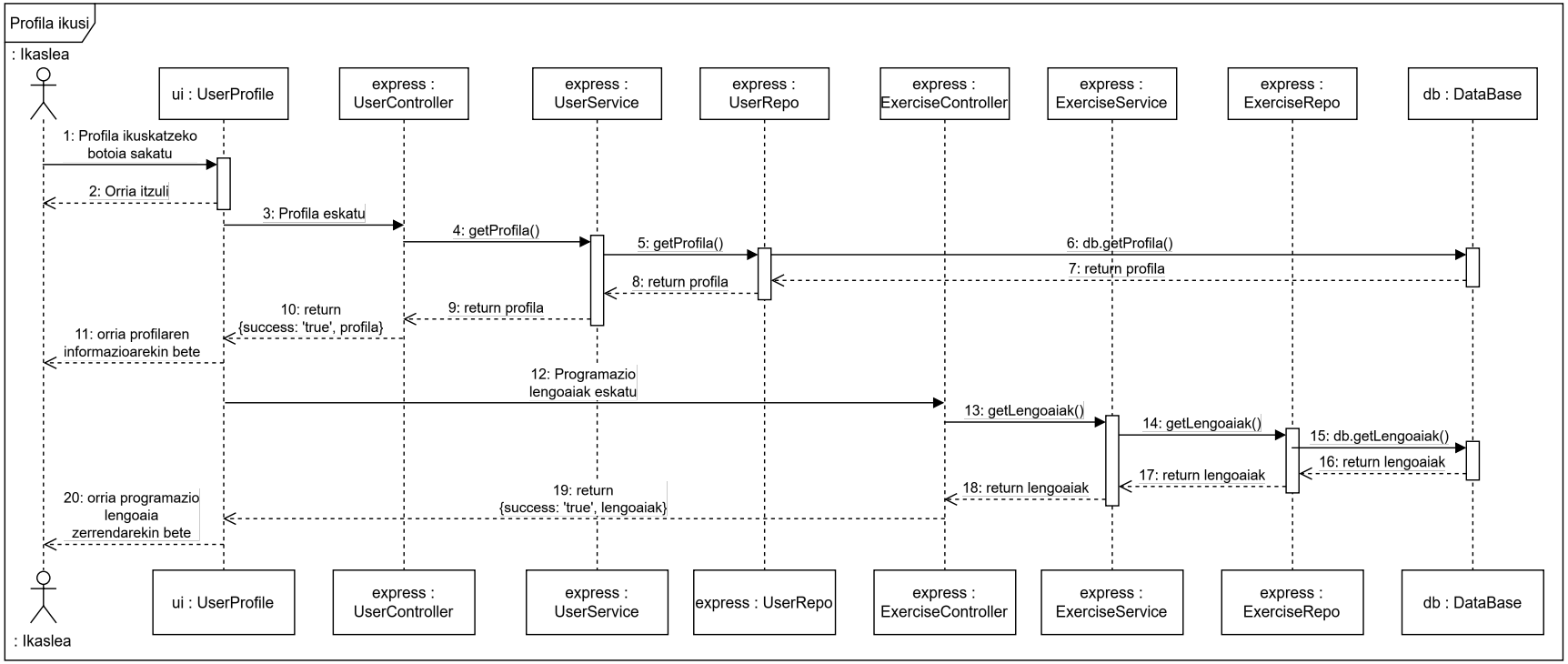
A.1 Irudia: BF-1 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.1 Taula.



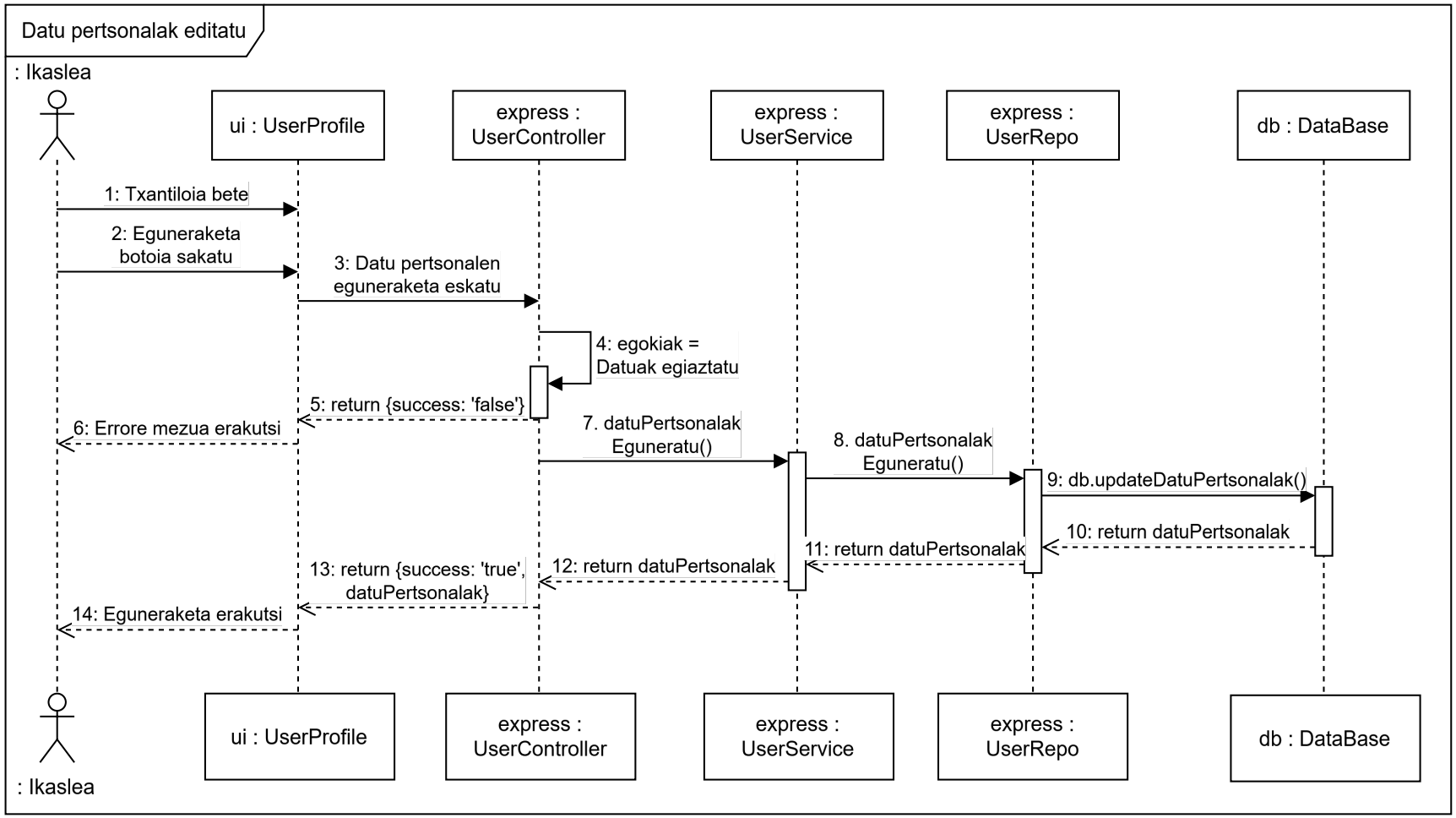
A.2 Irudia: BF-2 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.2 Taula.



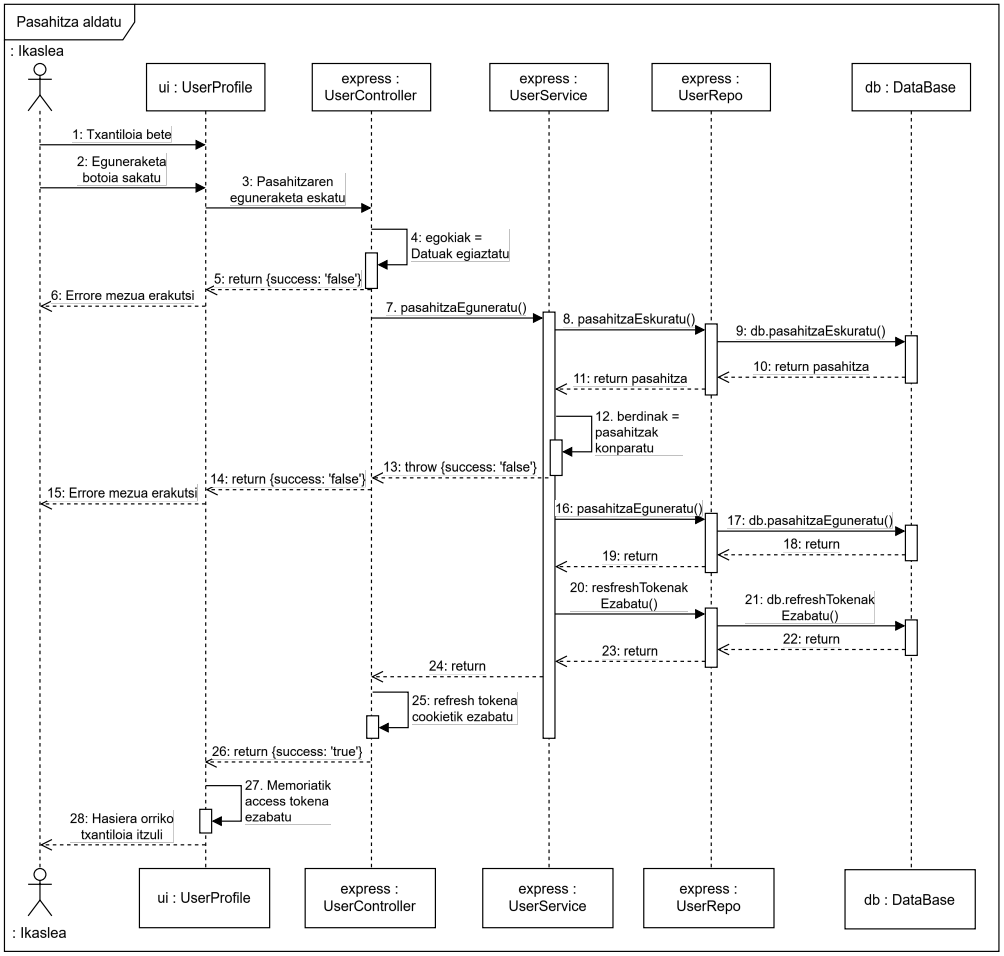
A.3 Irudia: BF-3 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.3 Taula.



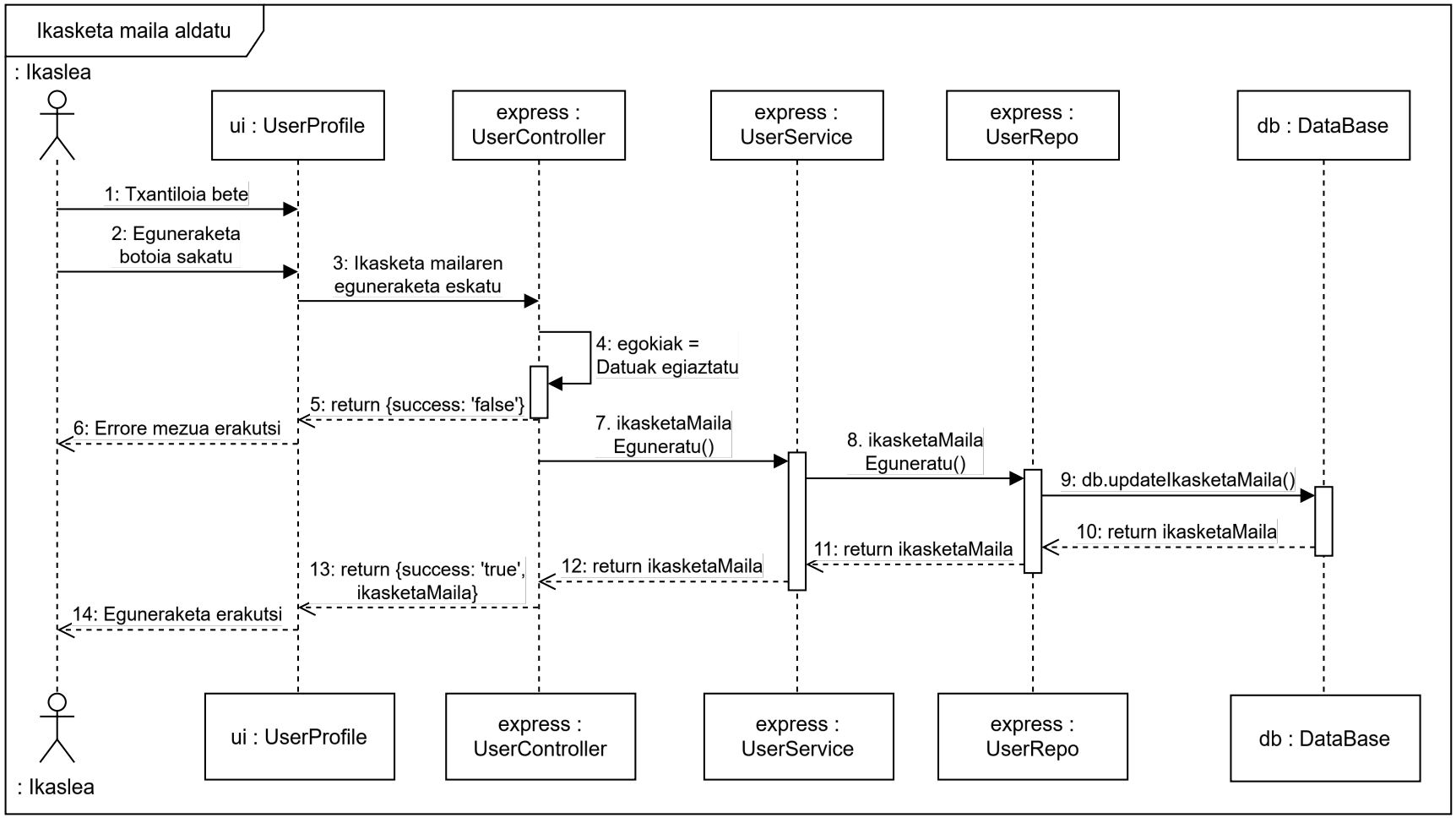
A.4 Irudia: BF-5 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.5 Taula.



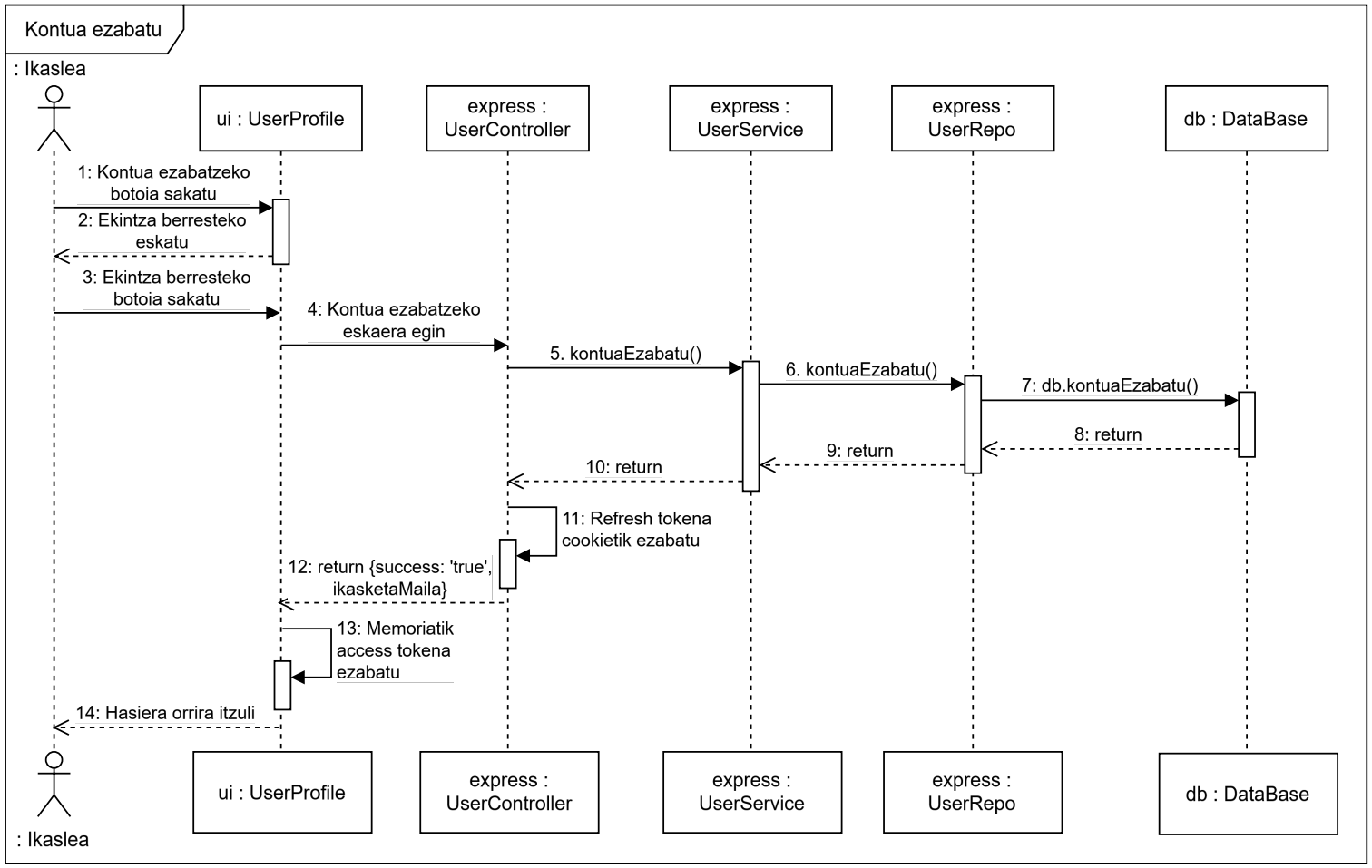
A.5 Irudia: BF-6 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.6 Taula.



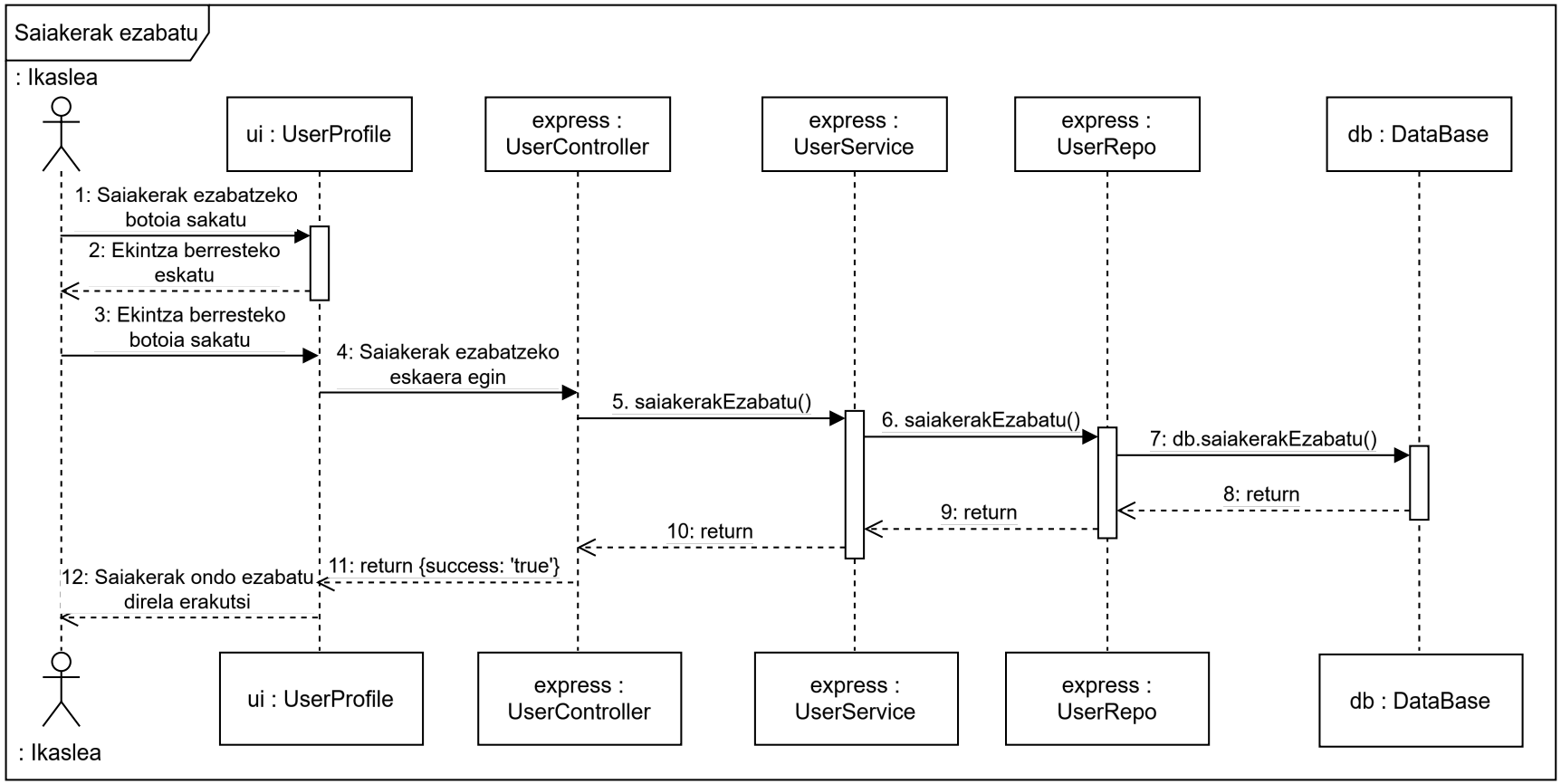
A.6 Irudia: BF-7 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.7 Taula.



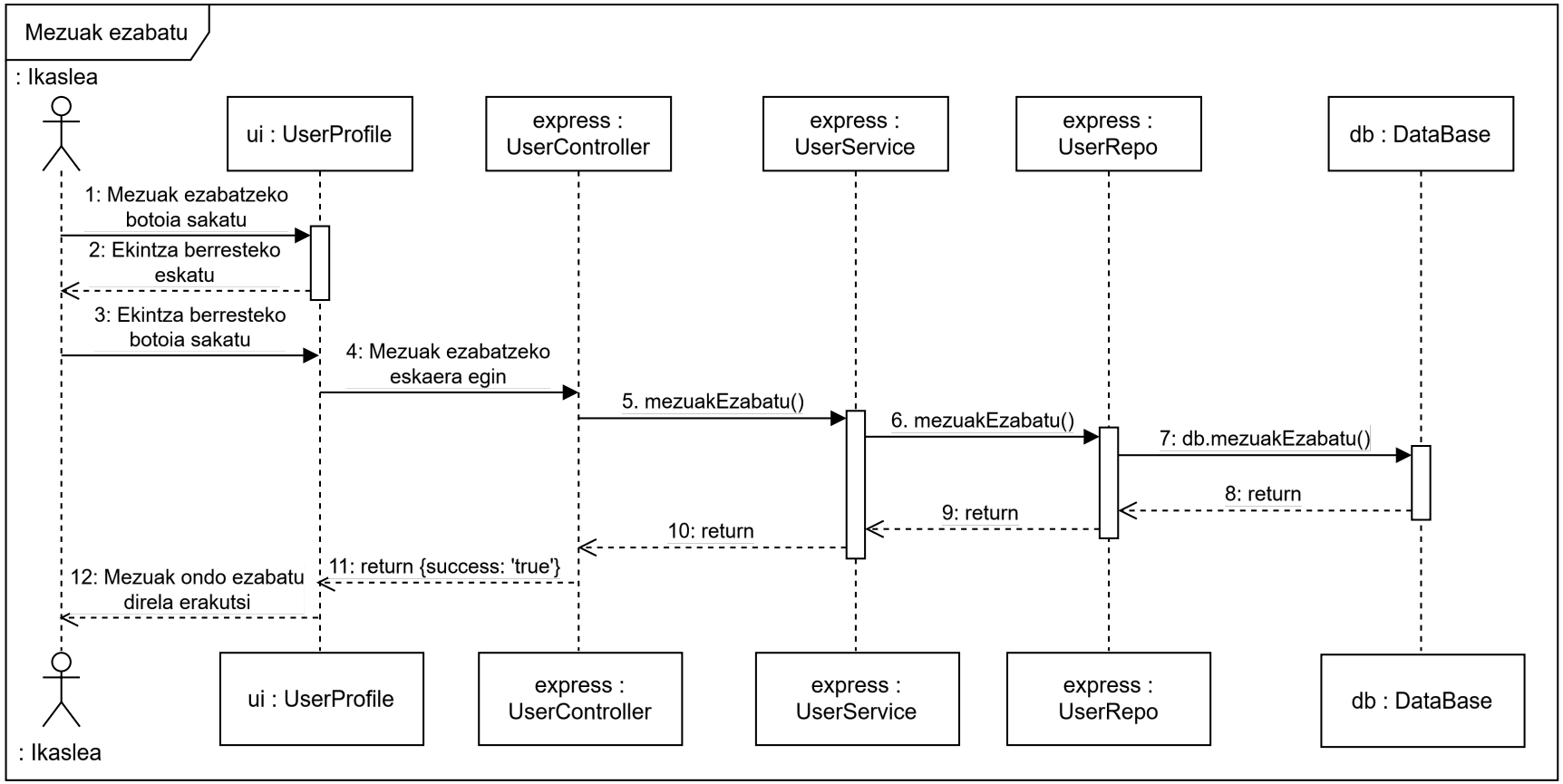
A.7 Irudia: BF-8 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.8 Taula.



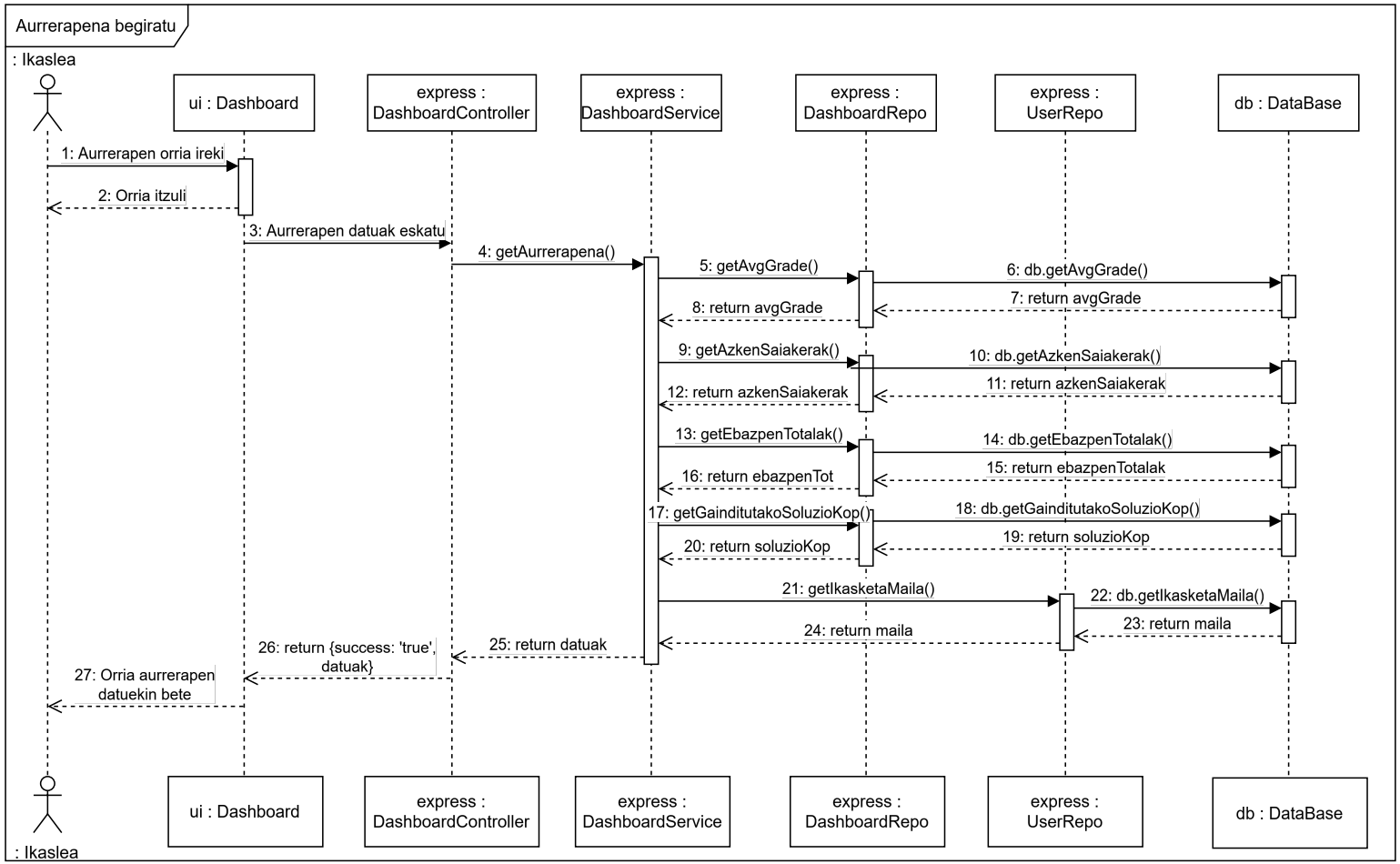
A.8 Irudia: BF-9 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.9 Taula.



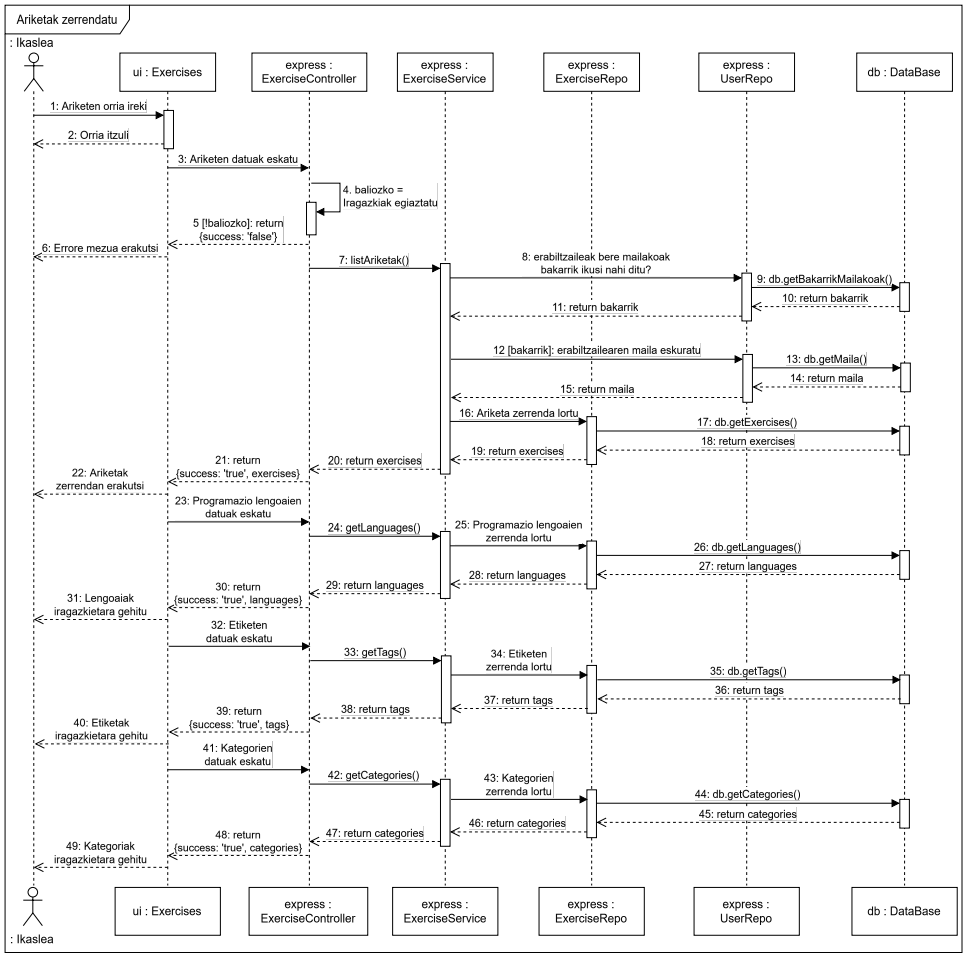
A.9 Irudia: BF-10 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.10 Taula.



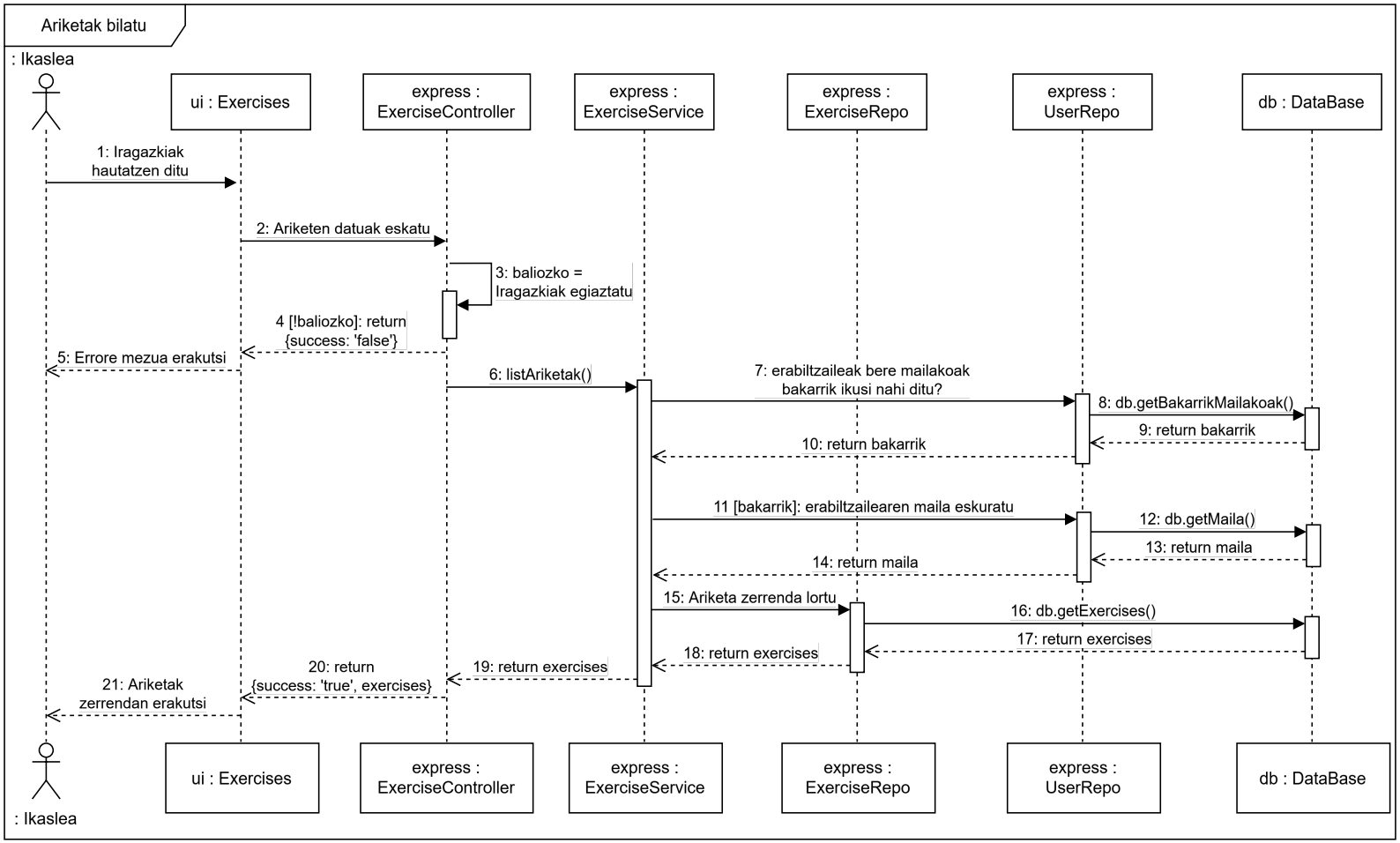
A.10 Irudia: BF-11 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.11 Taula.



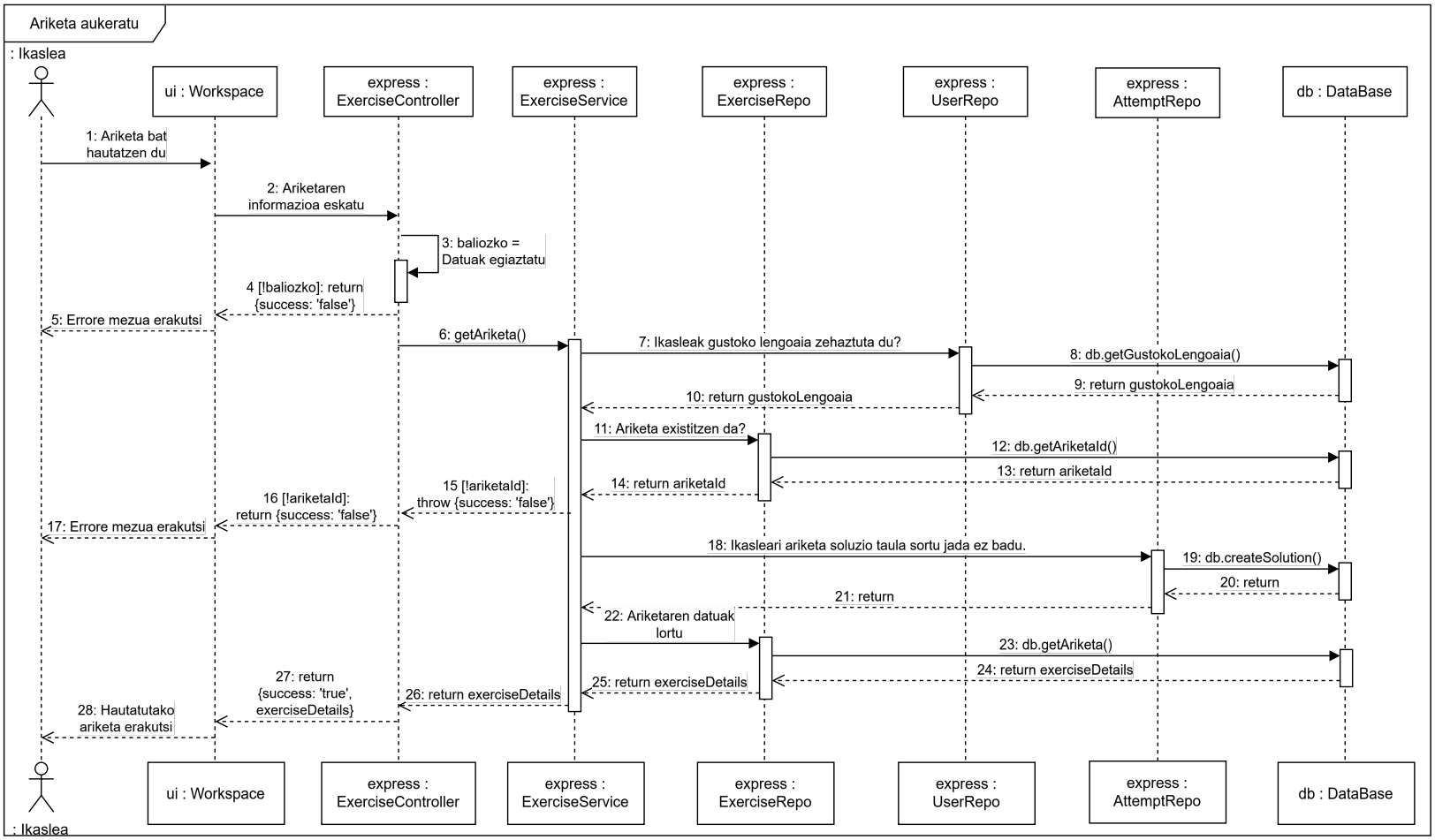
A.11 Irudia: BF-12 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.12 Taula.



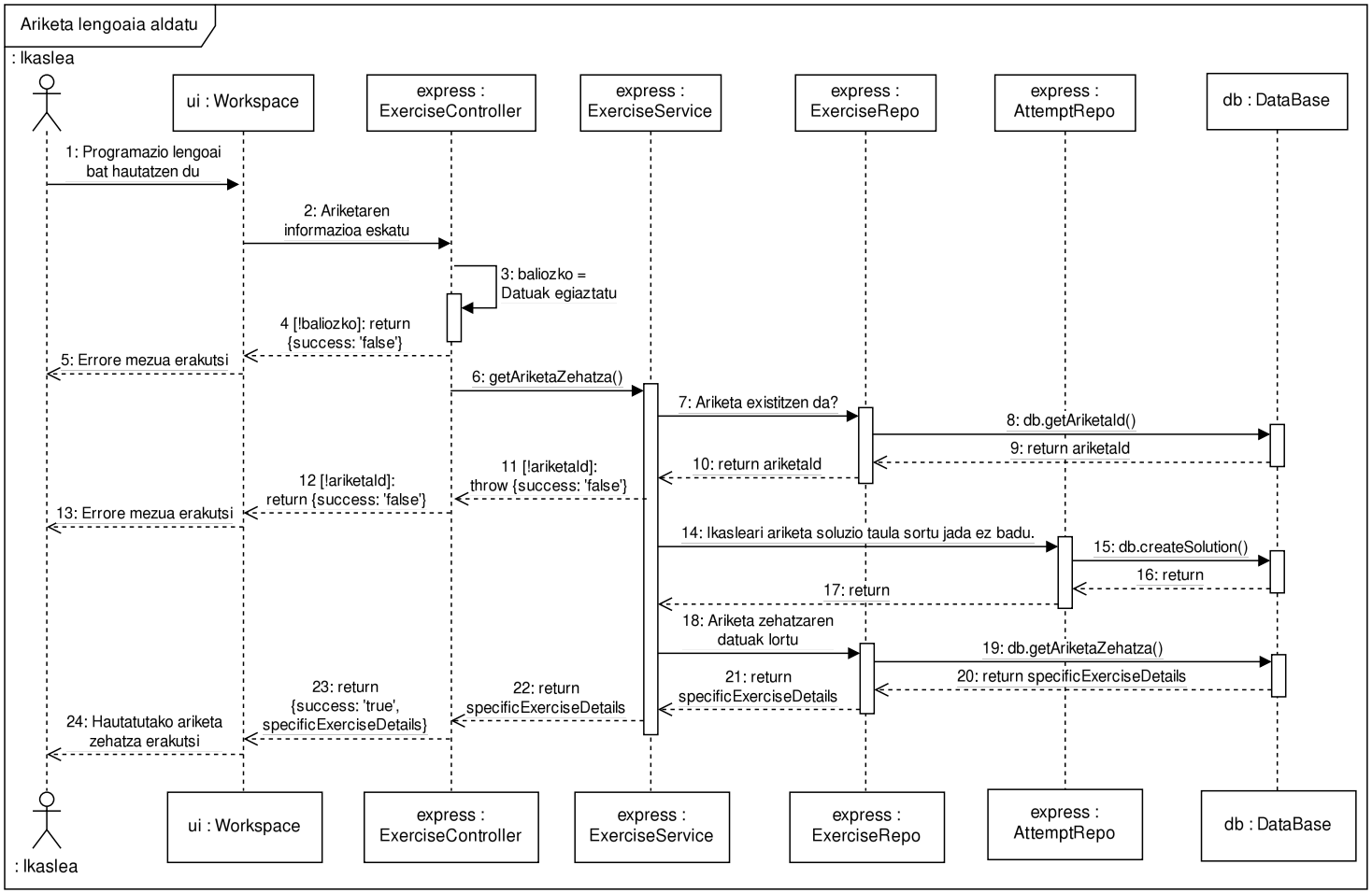
A.12 Irudia: BF-13 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.13 Taula.



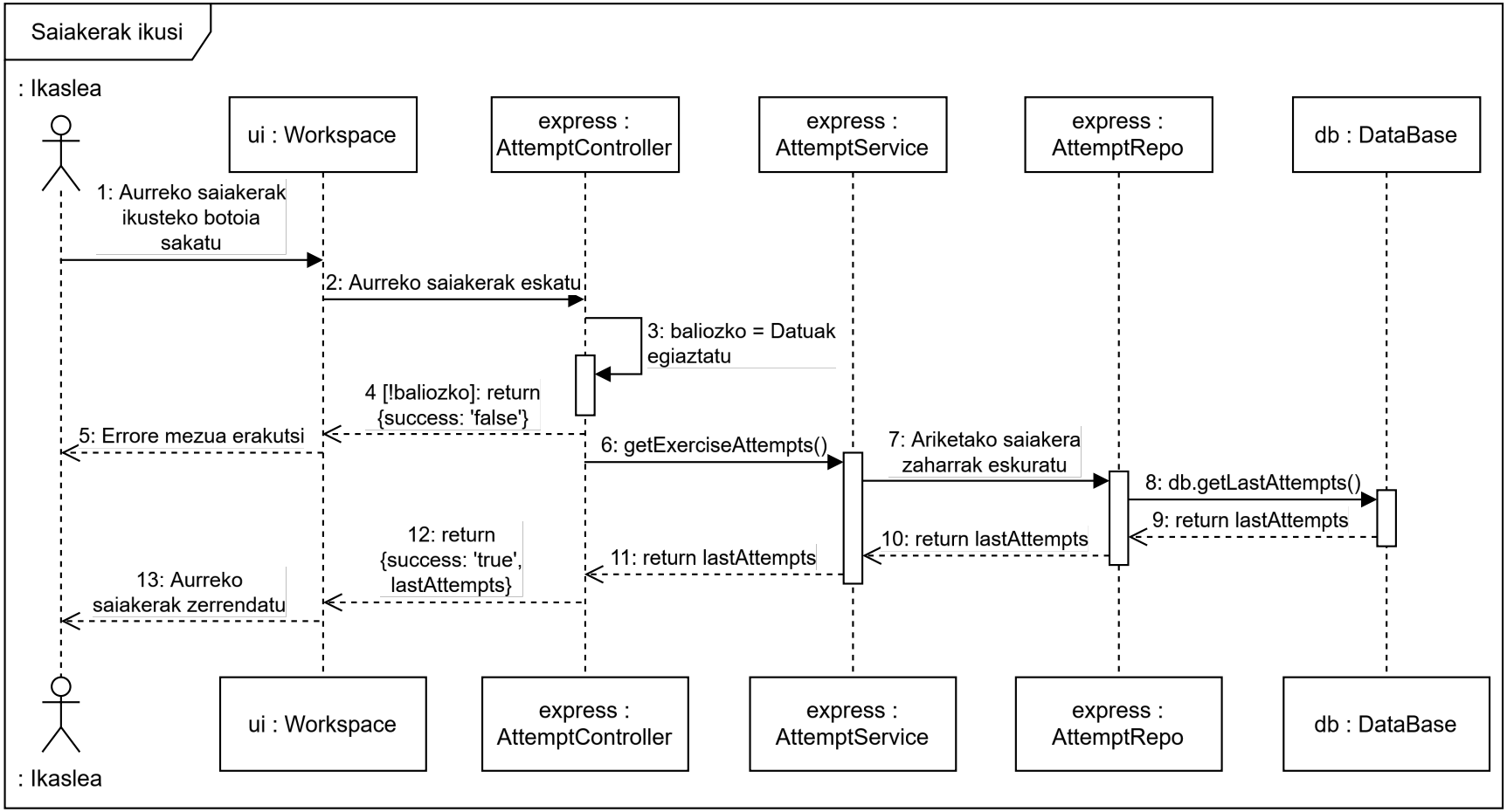
A.13 Irudia: BF-14 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.14 Taula.



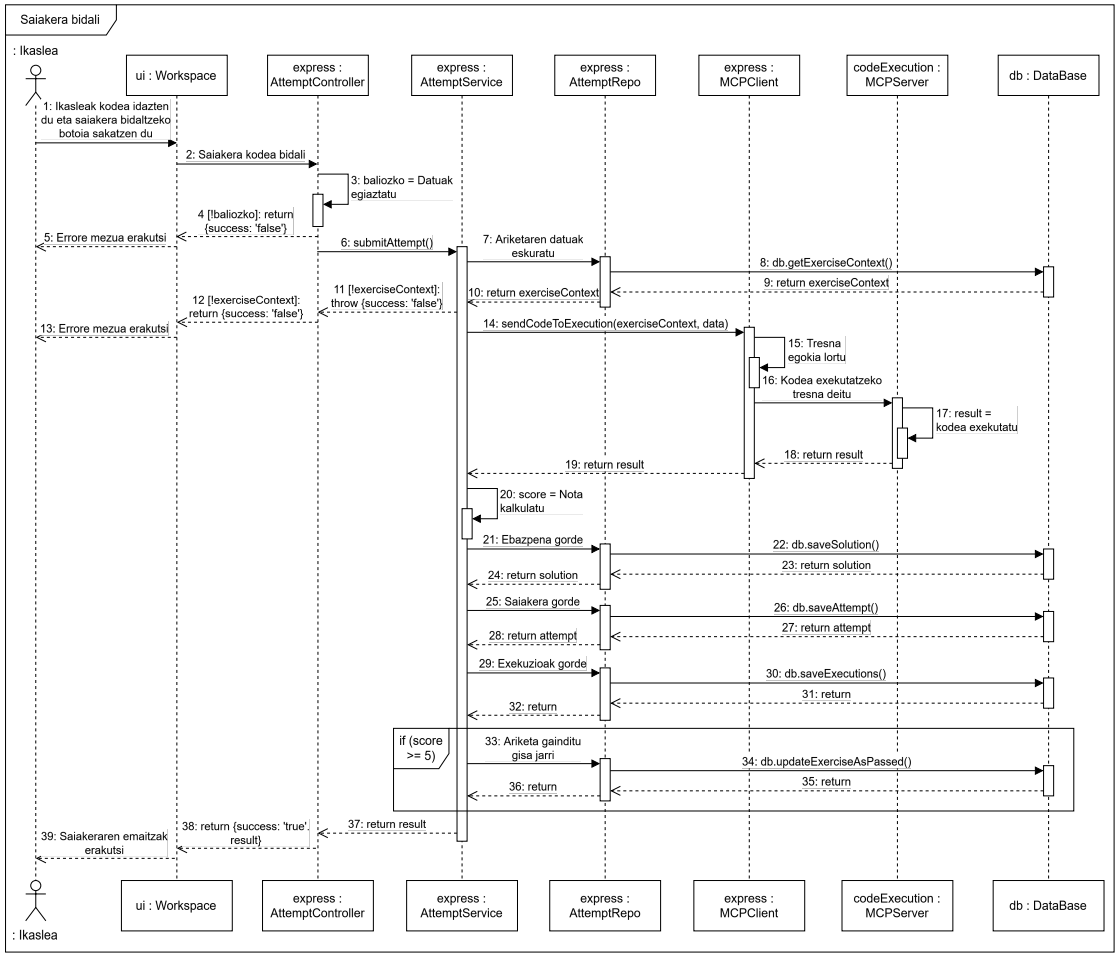
A.14 Irudia: BF-15 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.15 Taula.



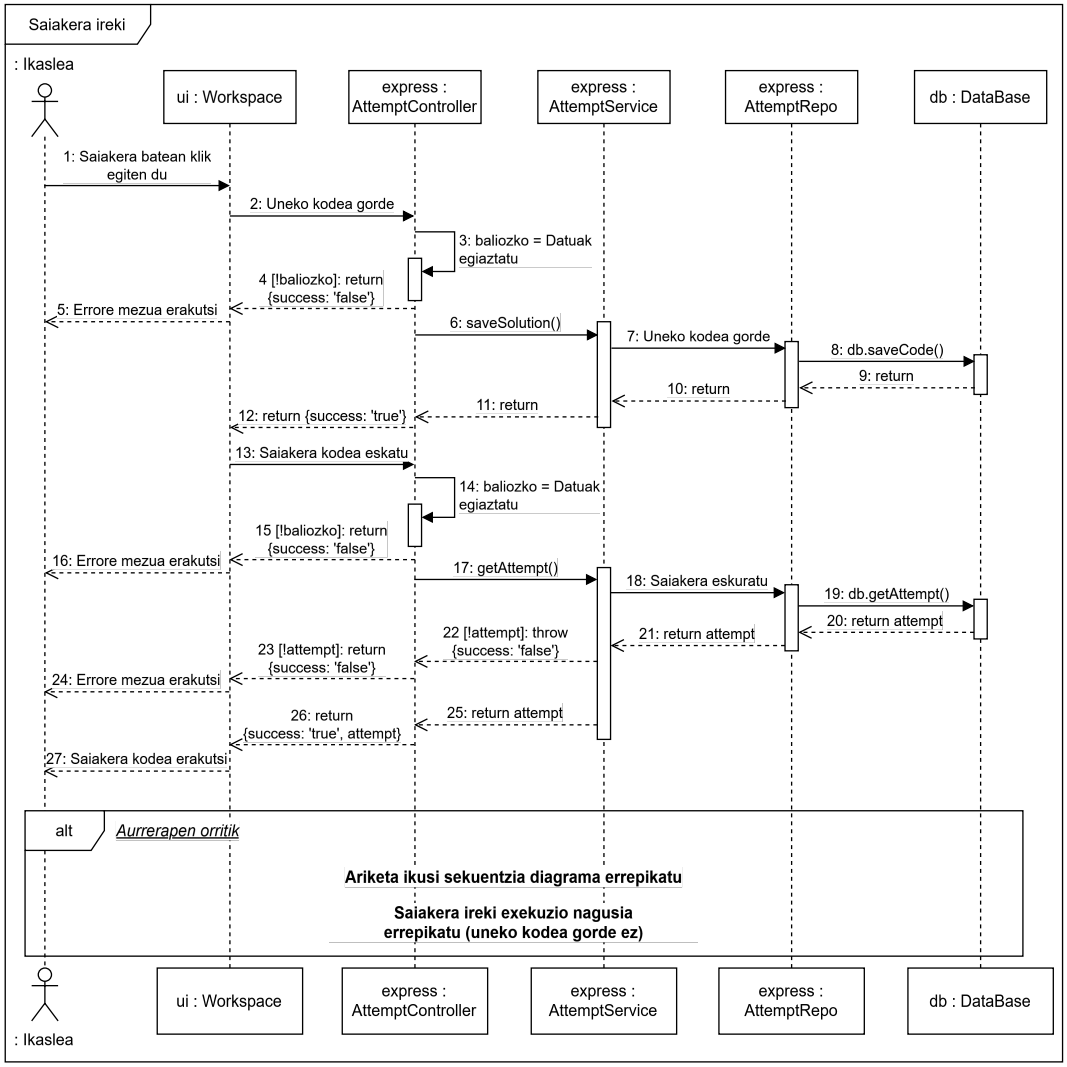
A.15 Irudia: BF-16 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.16 Taula.



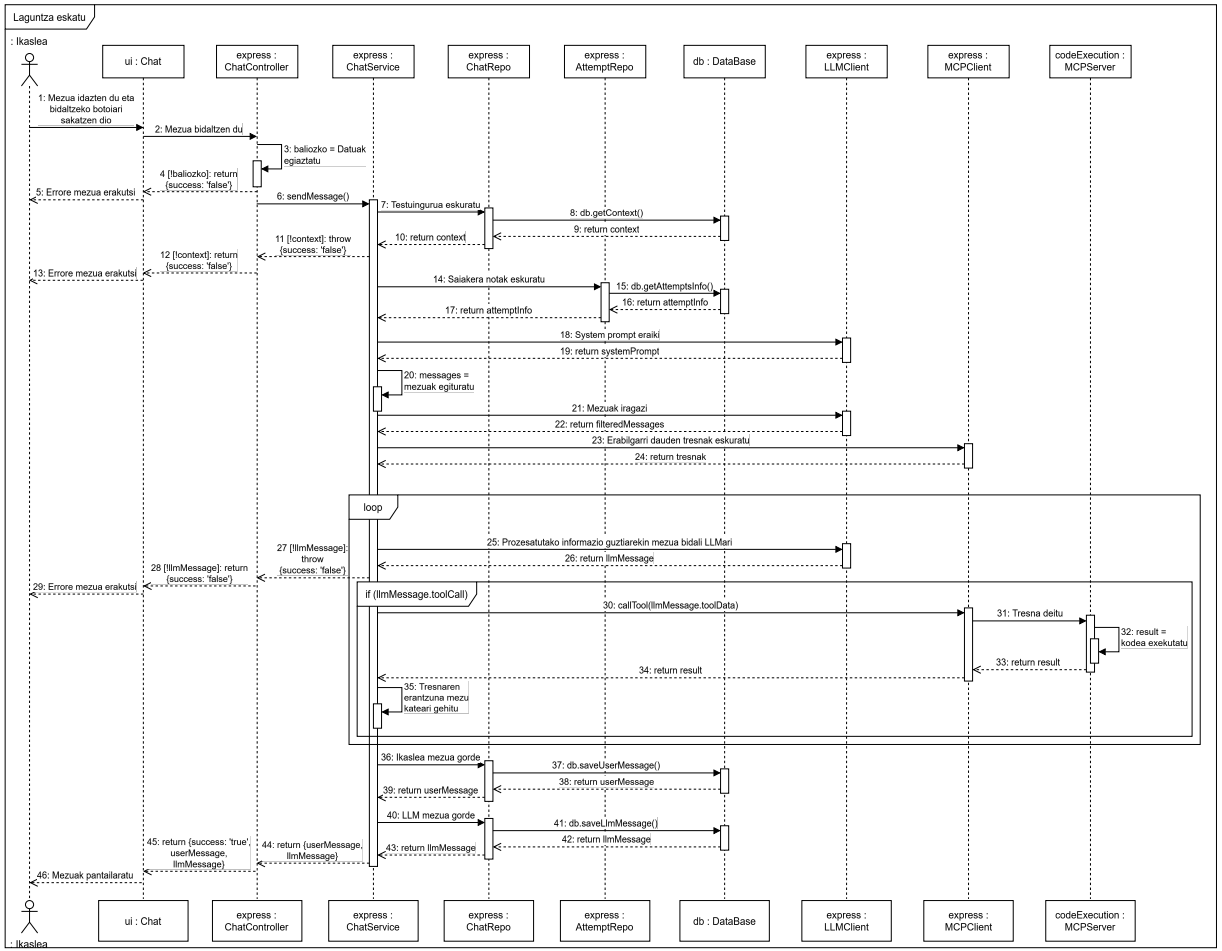
A.16 Irudia: BF-17 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.17 Taula.



A.17 Irudia: BF-18 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.18 Taula.



A.18 Irudia: BF-19 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.19 Taula.



A.19 Irudia: BF-20 betekizun funtzionalari dagokion sekuentzia diagrama, ikusi 4.20 Taula.

ERANSKINA B

Backendaren endpointak

Eranskin honetan backendaren REST API endpoint guztiak erakusten dira.

Saiakerak
Domeinu honetako ibilbide guztiak `/api/attempts`-ekin hasiko dira.

/api/attempts
Saiakerak zerrendatu.

- Metodoa: `Get`
- Eskema: `{query: {ariketaId}}`
- Kautotuta: Bai

/api/attempts/:saiakera_id
Saiakera eskuratu.

- Metodoa: `Get`
- Eskema: `{params: {saiakera_id}}`
- Kautotuta: Bai

/api/attempts/save
Ebazpena gorde.

- Metodoa: `Post`
- Eskema: `{body: {ariketa_zehatza_id, kodea}}`
- Kautotuta: Bai

/api/attempts/solution
Saiakera bidali.

- Metodoa: `Post`
- Eskema: `{body: {ariketa_zehatza_id, kodea}}`
- Kautotuta: Bai

Kautotzea
Domeinu honetako ibilbide guztiak `/api/auth`-ekin hasiko dira.

/api/auth/register
Erabiltzailea erregistratu.

- Metodoa: `Post`
- Eskema: `{body: {helbide_elektronikoa, ikasketa_maila, izena, pasahitza, pasahitza_errepikatu}}`
- Kautotuta: Ez

/api/auth/verify
Kontua egiaztatu.

- Metodoa: `Post`
- Eskema: `{body: {helbide_elektronikoa, tokena}}`
- Kautotuta: Ez

/api/auth/login
Saioa hasi.

- Metodoa: `Post`
- Eskema: `{body: {helbide_elektronikoa, pasahitza}}`
- Kautotuta: Ez

/api/auth/logout
Saioa itxi.

- Metodoa: `Post`
- Eskema: Ez
- Kautotuta: Ez

/api/auth/refresh
Tokena freskatu.

- Metodoa: `Post`
- Eskema: Ez
- Kautotuta: Ez

/api/auth/request-restore
Pasahitza berrezartzeko eskaera egin.

- Metodoa: `Post`
- Eskema: `{body: {helbide_elektronikoa}}`
- Kautotuta: Ez

/api/auth/restore
Pasahitza berrezarri.

- Metodoa: `Post`
- Eskema: `{body: {pasahitza, pasahitza_errepikatu, token}}`
- Kautotuta: Ez

Txata
Domeinu honetako ibilbide guztiak `/api/chat`-ekin hasiko dira.

/api/chat
Mezuak eskuratu.

- Metodoa: `Get`
- Eskema: `{query: {ebazpena_id}}`
- Kautotuta: Bai

/api/chat/send
Mezua bidali.

- Metodoa: `Post`
- Eskema: `{body: {content, ebazpena_id}}`
- Kautotuta: Bai

Aurrerapena
Domeinu honetako ibilbide guztiak `/api/dashboard`-ekin hasiko dira.

/api/dashboard/
Aurrerapen informazioa eskuratu.

- Metodoa: `Get`
- Eskema: `{query: {attempts_to_fetch}}`
- Kautotuta: Bai

Ariketak
Domeinu honetako ibilbide guztiak `/api/exercises`-ekin hasiko dira.

/api/exercises
Ariketen zerrenda eskuratu.

- Metodoa: `Get`
- Eskema: `{query: {egoerak, etiketa_ids, etiketa_ids_mode, etiketa_kategoria_ids, etiketa_kategoria_ids_mode, programazio_lengoaia_ids, programazio_lengoaia_ids_mode, titulua, zailtasunak}}`
- Kautotuta: Bai

/api/exercises/programming-languages
Programazio-lengoaien zerrenda eskuratu.

- Metodoa: `Get`
- Eskema: Ez
- Kautotuta: Bai

/api/exercises/tags
Etiketen zerrenda eskuratu.

- Metodoa: `Get`
- Eskema: Ez
- Kautotuta: Bai

/api/exercises/categories
Kategorien zerrenda eskuratu.

- Metodoa: `Get`
- Eskema: Ez

- Kautotuta: Bai

/api/exercises/:ariketa_id
Ariketa baten informazioa eskuratu.

- Metodoa: `Get`
- Eskema: `{query: {programazio_lengoaia_id}, params: {ariketa_id}}`
- Kautotuta: Bai

/api/exercises/:ariketa_id/language
Programazio-lengoaia zehatz baten ariketaren informazioa eskuratu.

- Metodoa: `Get`
- Eskema: `{query: {programazio_lengoaia_id}, params: {ariketa_id}}`
- Kautotuta: Bai

Erabiltzailea
Domeinu honetako ibilbide guztiak `/api/users`-ekin hasiko dira.

/api/users/profile
Erabiltzailearen profilaren informazio eskuratu.

- Metodoa: `Get`
- Eskema: Ez
- Kautotuta: Bai

/api/users/account
Datu pertsonalak eguneratu.

- Metodoa: `Put`
- Eskema: `{body: {izena}}`
- Kautotuta: Bai

/api/users/password
Pasahitza eguneratu.

- Metodoa: `Put`
- Eskema: `{body: {pasahitza_berria, pasahitza_errepikatu, pasahitza_zaharra}}`
- Kautotuta: Bai

/api/users/education
Ikasketa maila eguneratu.

- Metodoa: `Put`
- Eskema: `{body: {gogoko_lengoaia, ikasketa_maila, nire_mailakoak_ikusi}}`
- Kautotuta: Bai

/api/users/account
Erabiltzailea ezabatu.

- Metodoa: `Delete`

- Eskema: `{query: {}}`
- Kautotuta: Bai

/api/users/attempts
Erabiltzailearen saiakerak ezabatu.

- Metodoa: `Delete`
- Eskema: Ez
- Kautotuta: Bai

/api/users/messages
Erabiltzailearen elkarrizketak ezabatu.

- Metodoa: `Delete`
- Eskema: Ez
- Kautotuta: Bai

Health
Domeinu honetako ibilbide guztiak `/api/health`-ekin hasiko dira.

/api/health
Zerbitzariaren osasun ikuskapena egin.

- Metodoa: `Get`
- Eskema: Ez
- Kautotuta: Ez

ERANSKINA C

Erabilitako tresnak eta teknologiak

Proiektua garatzeko teknologia ugari erabili dira. Hauen artean protokoloak, *framework*ak, programazio-lengoaiak, exekuzio-denbora inguruneak, liburutegiak, web-aplikazioak eta sistema-aplikazioak daude. Jarraian, lan honetan erabilitako tresnak zerrendatu dira.

C.1 Programazio-lengoaiak, exekuzio-tresnak eta datu-baseak

C Maila baxuko programazio-lengoaia konpilatu [50].

CSS Webguneei estiloak emateko lengoaia [51].

Docker
Aplikazioak edukiontzi isolatuetan exekutatzeko tresna [42].

GCC
C (eta beste hainbeste) programazio-lengoaia konpilatzeko erabiltzen den konpilatzaile-bilduma (GNU Compiler Collection (GCC)) [52].

HTML
Webguneak eraikitzeko markaketa-lengoaia [53].

Java Object-Oriented Programming-an (OOP) oinarritutako programazio-lengoaia, konpilatu eta gero interpretatu egiten dena [54].

JavaScript
Web-nabigatzaileen webguneei interaktibitatea emateko erabiltzen den programazio-lengoaia dinamikoa [55].

JDK Java programazio-lengoaiarekin lan egiteko garapen-tresnen bilduma [56].

JUnit
Java programazio-lengoaiarentzat proba-kasuak idazteko frameworka [57].

LaTeX
Dokumentu teknikoak idazteko testuen konposiziorako sistema [58].

Node.js
JavaScript exekutatzeko exekuzio-denbora ingurunea [34].

PostgreSQL
Kode irekiko RDBMSa [31].

PyTest
Python programazio-lengoaiarentzat proba-kasuak idazteko frameworka [59].

Python
Sintaxia sinplea duen programazio-lengoaia dinamiko eta interpretatua [60].

SQL Datu-base erlazionalekin komunikatzeko erabiltzen den lengoaia [61].

TypeScript
Aldagai estatikoak erabiltzen dituen eta JavaScriptera konpilatzen duen programazioa-lengoaia [35].

C.2 Liburutegiak

Garapenean erabilitako liburutegiak **NPM**ren bitartez deskargatu dira: Node.js-erako pakete-kudeatzailea [62].

Axios
HTTP eskaerak egiteko bezeroa [45].

Bcryptjs
Zifratze-tresnak eskaintzen duen liburutegia [63].

Concurrently
Komandoak aldi berean exekutatzen eta kudeatzen ahalbidetzen duen tresna [64].

Cookie-parser
HTTP cookieak objektu batean gordetzeko middlewarea [65].

Cors
Kanpoko eskaerak kontrolatzeko mekanismoa aplikatzeko middlewarea [38].

Cross-env
Plataforma ezberdinetan ingurune-aldagaiak ezartzeko tresna [66].

Dockerode
Dockeren APIa Node.js-etik erabiltzeko liburutegia [67].

Dotenv
`.env` fitxategietatik ingurune-aldagaiak kargatzeko tresna [68].

Eslint
TypeScript eta JavaScript kodean akatsak aurkitzeko *linter*ra [69].

Express
Node.js-en APIak eraikitzeko frameworka [33].

Express-rate-limit
Express aplikazioak jasotako bezeroen eskaerak mugatzeko middlewarea [40].

Helmet
Express aplikazioa babesteko HTTP goiburuak ezartzeko middlewarea [39].

Jsonwebtoken
JWTak sortzeko eta egiaztatzeko liburutegia [70].

Lint-staged
Staged egoeran dauden Git fitxategiak linterrarekin automatikoki aztertzeko tresna [71].

Modelcontextprocotol/sdk
MCP inplementatzen duen TypeScripterako SDKa [72].

Monaco-editor/react
VSCoderen kode irekiko kode-editorea [46].

Node-cron
Atazak automatikoki exekutatzeko *cron* tresna [73].

Pg Node.js-eko PostgreSQL bezeroa [74].

Prettier
Iturburu-kodea automatikoki formateatzeko tresna [75].

Prisma
TypeScripterako ORMa [32].

React
Elementuen uneko egoera gogoratzen duen interfazeak eraikitzeko liburutegia [43].

React-dom
React osagaiak DOMean atxikitzeko liburutegia [76].

React-markdown
Markdown edukia HTML bihurtzeko liburutegia [77].

React-router-dom
Reacten nabigazio ibilbideak eraikitzeko liburutegia [78].

Remark-gfm
Markdown ikuskatzeko liburutegiak "GitHub Flavored Markdown" erabiltzeko plugina [79].

Simple-git-hooks
Git *commit* egindakoan *script*ak automatikoki exekutatzeko tresna [80].

Sonner
Jakinarazpenak ikuskatzeko Reacteko osagaia [81].

TailwindCSS
Utilitate-klaseetan oinarritutako CSS liburutegia [44].

Tar-fs
Tar fitxategi paketatuak kudeatzeko liburutegia [82].

TSUP
Typescript paketatzailea (*bundler*) [83].

TSX TypeScript fitxategiak konpilatu gabe zuzenean exekutatzeko tresna [84].

Vite Reactekin erabili daitekeen eraikuntza-komandoa eta garapen-zerbitzaria [85].

Winston
Node.js-erako *logger*a [86].

Winston-daily-rotate-file
Loggerraren fitxategiak egunero aldatzeko tresna [87].

Zod Eskemak balidatzeko liburutegia [36].

C.3 Garapen-tresnak

Draw.io
Diagramak marrazteko eta JavaScripten idatzitako web-editorea [88].

Neovim
Terminaleko kode-editore arina eta hedagarria [89].

Online Gantt
Gantt diagramak egiteko web-aplikazioa [90].

Overleaf
LaTeX dokumentuak online eta modu kolaboratiboan idazteko web-plataforma [91].

VSCode
Microsoftek garatutako kode-editore grafikoa eta hedagarria [92].

C.4 Kudeaketa-tresnak

Git Proiektuen bertsio kontrola egiteko sistema azkarra eta kode irekikoa [93].

GitHub
Git biltegiak igotzeko eta partekatzeko web-plataforma [94].

Microsoft Teams
Taldekideen arteko komunikazioa ahalbidetzen duen aplikazioa [95].

OneDrive
Microsoften hodeiko fitxategi-biltegiratze zerbitzua [96].

Thunderbird
: Mozillaren kode irekiko posta-elektroniko bezeroa [97].

ERANSKINA D

AA sortzailearen erabilera deklarazioa

Erabilitako tresna/k: Copilot, Claude eta Gemini.

Egindako erabilera (dagokiona markatu):

- ☐ Testu-sorkuntza
- ☐ Birformulazioa
- ☒ Itzulpena / zuzenketa
- ☐ Egituraren iradokizuna
- ☐ Laguntza metodologikoa
- ☒ Kodetzeko / programatzeko laguntza
- ☒ Beste batzuk

Egindako erabileraren deskribapena:
Iturburu-kodea idazterakoan Copiloten *inline suggestions* erabili da. Kontzeptu berriak ikasteko Claude erabili da. Memorian erabiltzen diren hitz teknikoen itzulpenarekin laguntzeko Gemini erabili da.

Autoretza konpromisoa:
Sortutako eduki guztia pertsonalki berrikusi, berregin eta balioztatu dudala bermatzen dut.

ERANSKINA E

Bileren aktak

Eranskin honetan proiektua garatzeko egin diren bileren aktak azaltzen dira: egin dena eta egin beharrekoa. Bilera guztietan GrALaren zuzendariak eta garatzaileak parte hartu dute.

E.1 Bilera akta (2026-02-23)

Data: 2026-02-23

Egin dena:

- Lanaren aurkezpena

Egitekoa:

- Artearen egoera: Hezkuntza plataformak eta programazio ikaskuntza tresnak aztertu.
- Ikasi zer den Model Context Protocol, nola funtzionatzen duen eta nola integratzen den LLMekin.
- Eskakizunen definizioa: plataformak zer egin behar duen eta nola jokatu behar duen zehaztea
- Hasierako arkitekturaren diseinua: goi-mailako arkitektura-diagrama bat marraztea (frontend-a, backend, MCPa, LLMa, datu-basea)
- Erabiltzaile interfazearen diseinua: hasierako pantailak sortu: editorea, txata, aurrerapen-panela (progression-dashboard)
- Programazio ariketen hautaketa eta diseinua: sortu lehen ? ariketak deskribapenekin, probekin (testak) eta metadatuekin

E.2 Bilera akta (2026-02-27)

Data: 2026-02-27

Egin dena:

- Irismena definitua eta adostua

Egitekoa:

- MCP
- Ariketen diseinua: metadatuak, ze programazio lengoaia etab. Non gorde? Datubasean, beste nonbaiten?
- Betekizun funtzionalak eta ezin funtzionalak.
- Itziar: galdetu GrALaren ordena eta esperokoa. Zehaztasun maila.

E.3 Bilera akta (2026-03-06)

Data: 2026-03-06

Egin dena:

- MCP

Egitekoa:

- Garapen Jasangarrirako Helburuak
- MCP
- Ariketen diseinua
- Itziar: irakurri atazak eta definizio guztia.
- Itziar: GrAL adib: erabiltzaileak, segurtasuna etab.
- Itziar: AAren inguruko web-orria.

E.4 Bilera akta (2026-03-13)

Data: 2026-03-13

Egin dena:

- GHJ
- MCP
- Ariketa diseinua

- DB diseinua

Egitekoa:

- MCP exekuziorako: zer egin ikasleak idatzi duena, egokia izan arren, konputazionalki kostu handikoa bada? ERABAKITZEKO/BEGIRATZEKO
- Ariketa adibide bat sortu – Itziar – JSON moduan.
- MCPrekin jarraitu

E.5 Bilera akta (2026-03-27)

Data: 2026-03-27

Egin dena:

- LLMak azaldu aurrekarietan
- MCP azaldu aurrekarietan
- MCP bezero/zerbitzariak nola eraiki ikusi
- Biblatex konpondu
- Betekizunak (memoria)

Egitekoa:

- DB eguneratu: Sesioa -> JWT
- Betekizun ez funtzionalak
- Sekuentzia diagramak (kodea eta gero)
- Frontend-a hasi? (Git, React, etc.)

E.6 Bilera akta (2026-04-10)

Data: 2026-04-10

Egin dena:

- LLM, MCP eta betekizunak – memorian
- Aztertuta MCP bezero/zerbitzaria nola eraiki

Egitekoa:

- Inplementatzen hasi: datu-basea eta sesio-kontrola. Hasierako pantaila?
- ariketak jaso eta bistaratu?

E.7 Bilera akta (2026-04-17)

Data: 2026-04-17

Egin dena:

- Postgresql instalatu (linux)
- Monorepoa konfiguratu
- Git konfigurazioa
- Git hookeak gehitu (workflowak beharrezkoak... testak, formatter...)
- Typescript konfiguratu + inportazio aliasak
- Linterrak zehaztu
- Loggerra konfiguratu
- Env balidazioa, zod
- Express zerbitzaria
  - Cors
  - Http burukoak, Helmet
  - Rate limiterrak
  - Body parserrak
  - Errore kudeaketa
  - "graceful shutdown" (http eskaera aktiboen blokeoa)
  - JWT: access tokenen middlewarea inplementatu (token sorketa ez)
  - Domeinuetan oinarritutako Model View Controller arkitektura zehaztu
    - Controller funtzioen signature
    - Ibilbideak
    - Routerrak (betekizun funtzionalak + healthcheck)
- Datu-basea:
  - Hasiera batean PG hutsa (konplexutasuna mantentzeko, ts tipoak eskuz sortu behar, eguneraketa zailak, migrazio zailak, ...)
  - ORM bat erabili lana errazteko: Prisma
  - Konfigurazioa
  - Datu-basearen eskemaren sorrera

Egitekoa:

- Memoriako EED eguneratu
- Zerbitzaria: Negozio logika + iraunkortasun geruza ?
- Datu-basea: seed adibideak
- Galdetzeko: repo lizentzia

E.8 Bilera akta (2026-04-24)

Data: 2026-04-24

Egin dena:

- Negozio logika
- Iraunkortasun geruza
- Datu-basea eguneratu (buruko fitxategiak gehitu (c), ikaslea eguneratu, ...)
- Seeding modularra
- Seed adibide batzuk
- React konfiguratu eta monorepoarekin integratu
- SPA -> Routing, token handler
- Refresh token zahar bat berrerabiltzerakoan saio guztiak ezeztatu (security)
- Landing page, login page, register page, settings page, layouts, components, styling
- React -> double render + auth, arazoak konpondu
- Programazio lengoaiak, etiketak, kategoriak, lortzeko rutak gehitu.
- Ariketen eskuraketa filtratua (backend, espero funtzionatzen)

Egitekoa:

- Memoriako:
  - EED eguneratu (etorkizunera begira)
  - Aipamenak gehitu: test kasuak sortu, enuntziatuen itzulpena LLM bidez.
- Zerbitzaria: Negozio logika + iraunkortasun geruza ?
- Datu-basea: seed adibideak
- FrontEnd: Monaco... etab.

E.9 Bilera akta (2026-04-30)

Data: 2026-04-30

Egin dena:

- Ikasleen lan ingurunea (falta txata, ariketa mcp bidez eskuratu)
- Ariketak iragazi
- Code execution MCP egiten.

Egitekoa:

- Monaco – React egin
- Ariketaren enuntziatu bat erakutsi (datu-basean dagoena), ikasleak inplementatu egiten du eta test-ak pasatzea "eskatzen" du. MCP docker.
- Saiakerak inplementatu - AURRERAGO
- Negozio-logika

E.10 Bilera akta (2026-05-13)

Data: 2026-05-13

Egin dena:

- Code execution martxan!

Akronimoa

AA Adimen Artifiziala iii, 1–3, 5, 6, 8–10, 36, 42, 43, 52

AAIF Agentic AI Foundation 5

ADDI Archivo Digital para la Docencia y la Investigación 19

API Application Programming Interface 7, 12, 13, 15, 17, 18, 23, 43, 51, 56, 57, 61, 77–79, 103, 110, 111

CORS Cross-Origin Resource Sharing 62

CRUD Create, Read, Update and Delete 61, 63

CSR Controller, Service and Repository 63, 69

DBMS Database Management System 59

DOM Document Object Model 66, 67, 111

DOS Denial of Service 62

DPoP Demonstrating Proof of Possession 79

EHU Euskal Herriko Unibertsitatea 61

GCC GNU Compiler Collection 109

GDPR General Data Protection Regulation 79

GJH Garapen Jasangarrirako Helburuak iii, iv

GrAL Gradu Amaierako Lana i, 1, 2, 7, 10, 13, 15, 19, 20, 22, 48, 60, 73, 75, 77, 117

GUI Graphical User Interface 47, 79

HTTP Hypertext Transfer Protocol 7, 51, 62, 68, 110, 111

JSON JavaScript Object Notation 6, 52, 55, 60

JSON-RPC JSON RPC 5

JWT JSON Web Token 29, 30, 62, 78, 111

LDE Lanaren Deskonposizio Egitura 14

LH Lehen Hezkuntza iv

LLM Hizkuntza-eredu Handia i, iii, 1–8, 11–13, 15, 17, 18, 23, 47–49, 56–58, 61, 68, 69, 77–79

LOPDGDD Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales 79

LSP Language Server Protocol 68, 79

MCP Model Context Protocol i, iii, 1–7, 10–18, 23, 41, 47–49, 51–56, 58, 59, 61, 63, 64, 69, 74, 75, 77, 78, 111

NBE Nazio Batuen Erakundea iii

NLP Natural Language Processing 4

OOP Object-Oriented Programming 109

ORM Object–Relational Mapper 59, 111

RDBMS Relational Database Management System 12, 59, 110

REST Representational State Transfer 12, 15, 17, 51, 103

RoPE Rotary Position Encoding 4

RPC Remote Procedure Call 6

RTR Refresh Token Rotation 62

SDK Software Development Kit 51, 63, 75, 111

SPA Single Page Application 12, 14, 16, 66

SSE Server-Sent Events 7

STDIO Standard Input/Output 6, 51, 63

UI User Interface 66

URI Uniform Resource Identifier 7

XSS Cross Site Scripting 62

Bibliografia

[1] United Nations. *The 17 Goals*. Department of Economic eta Social Affairs, Sustainable Development. 2015. Eskuragarri hemen: https://sdgs.un.org/goals (azkenekoz ikusia 2026-03-10). Ikusi iii orrialdea.

[2] OpenAI. *Introducing GPT-5.4*. 2026ko martxoaren 5a. Eskuragarri hemen: https://openai.com/index/introducing-gpt-5-4/ (azkenekoz ikusia 2026-03-15). Ikusi 3 orrialdea.

[3] Anthropic. *Claude Opus 4.6*. 2026ko otsailaren 5a. Eskuragarri hemen: https://www.anthropic.com/news/claude-opus-4-6 (azkenekoz ikusia 2026-03-15). Ikusi 3 orrialdea.

[4] Google. *Gemini 3.1 Pro: A smarter model for your most complex tasks*. 2026ko otsailaren 19a. Eskuragarri hemen: https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/ (azkenekoz ikusia 2026-03-15). Ikusi 3 orrialdea.

[5] Junchao Wu et. al. «A survey on llm-generated text detection: Necessity, methods, and future directions». Non: *Computational Linguistics* 51.1 (2025), orrk. 275-338. Ikusi 3 orrialdea.

[6] Yadagiri Annepaka eta Partha Pakray. «Large language models: a survey of their development, capabilities, and applications». Non: *Knowledge and Information Systems* 67.3 (2025), orrk. 2967-3022. Ikusi 4 orrialdea.

[7] Cole Stryker eta Jim Holdsworth. *What is NLP (natural language processing)?* 2024. Eskuragarri hemen: https://www.ibm.com/think/topics/natural-language-processing (azkenekoz ikusia 2026-03-31). Ikusi 4 orrialdea.

[8] Ashish Vaswani et. al. «Attention is all you need». Non: *Advances in neural information processing systems* 30 (2017). Ikusi 4 orrialdea.

[9] Bonan Min et. al. «Recent advances in natural language processing via large pre-trained language models: A survey». Non: *ACM Computing Surveys* 56.2 (2023), orrk. 1-40. Ikusi 4 orrialdea.

[10] Cole Stryker. *What are large language models (LLMs)?* 2021. Eskuragarri hemen: https://www.ibm.com/think/topics/large-language-models (azkenekoz ikusia 2026-03-31). Ikusi 4, 5 orrialdeak.

[11] Jianlin Su et. al. «Roformer: Enhanced transformer with rotary position embedding». Non: *Neurocomputing* 568 (2024), orr. 127063. Ikusi 4 orrialdea.

[12] Dave Bergmann eta Cole Stryker. *What is an attention mechanism?* 2024. Eskuragarri hemen: https://www.ibm.com/think/topics/attention-mechanism (azkenekoz ikusia 2026-04-02). Ikusi 4 orrialdea.

[13] Anthropic. *Donating the Model Context Protocol and establishing the Agentic AI Foundation*. 2025eko abenduaren 9a. Eskuragarri hemen: https://www.anthropic.com/news/donating-the-model-context-protocol-and-establishing-of-the-agentic-ai-foundation (azkenekoz ikusia 2026-04-03). Ikusi 5 orrialdea.

[14] Anthropic. *Introducing the Model Context Protocol*. 2024ko azaroaren 25a. Eskuragarri hemen: https://www.anthropic.com/news/model-context-protocol (azkenekoz ikusia 2026-04-03). Ikusi 5 orrialdea.

[15] Anthropic. *Introduction – Model Context Protocol*. 2025. Eskuragarri hemen: https://modelcontextprotocol.io/ (azkenekoz ikusia 2026-03-13). Ikusi 5, 6 orrialdeak.

[16] Yan Amal Abdillah et. al. «Design and Development of Interactive Moodle Using Design Thinking to Support Online Learning». Non: *Elinvo (Electronics, Informatics, and Vocational Education)* 9.2 (2002), orrk. 208-219. Ikusi 8 orrialdea.

[17] Martin Dougiamas. «Moodle: A Virtual Learning Environment for the Rest of Us». Non: *TESL-EJ* 08.02 (2004ko iraila). Eskuragarri hemen: https://tesl-ej.org/wordpress/issues/volume8/ej30/ej30m2/ (azkenekoz ikusia 2026-03-24). Ikusi 8 orrialdea.

[18] Moodle. *Statistics*. 2026ko martxoaren 24a. Eskuragarri hemen: https://stats.moodle.org/ (azkenekoz ikusia 2026-03-24). Ikusi 8 orrialdea.

[19] Moodle. *AI tools*. 2025eko urriaren 2a. Eskuragarri hemen: https://docs.moodle.org/501/en/AI_tools (azkenekoz ikusia 2026-03-19). Ikusi 8 orrialdea.

[20] Michael Feldstein. *Instructure Canvas: A New LMS Entrant*. 2010eko ekainaren 30a. Eskuragarri hemen: https://eliterate.us/instructure-canvas-a-new-lms-entrant (azkenekoz ikusia 2026-03-24). Ikusi 8 orrialdea.

[21] Instructure. *Canvas LMS*. 2011. Eskuragarri hemen: https://github.com/instructure/canvas-lms (azkenekoz ikusia 2026-03-25). Ikusi 8 orrialdea.

[22] Wabash EdTech. *IgniteAI in Canvas*. 2026ko martxoaren 18a. Eskuragarri hemen: https://edtech.wabash.edu/landing-page/blog-posts/blog-2-igniteai-in-canvas/ (azkenekoz ikusia 2026-03-24). Ikusi 8 orrialdea.

[23] Larry Magid. *Google Classroom Offers Assignment Center for Students and Teachers*. 2014ko maiatzaren 6a. Eskuragarri hemen: https://www.forbes.com/sites/larrymagid/2014/05/06/google-classroom-offers-control-center-for-students-and-teachers/ (azkenekoz ikusia 2026-03-24). Ikusi 9 orrialdea.

[24] Google. *Classroom*. 2026. Eskuragarri hemen: https://classroom.google.com/ (azkenekoz ikusia 2026-03-26). Ikusi 9 orrialdea.

[25] Cinthya Mohr. *Gemini in Classroom: No-cost AI tools that amplify teaching and learning*. 2025eko ekainaren 30a. Eskuragarri hemen: https://blog.google/products-and-platforms/products/education/classroom-ai-features/ (azkenekoz ikusia 2026-03-24). Ikusi 9 orrialdea.

[26] Maureen Heymans. *Guided Learning in Gemini: From answers to understanding*. 2025eko abuztuaren 6a. Eskuragarri hemen: https://blog.google/products-and-platforms/products/education/guided-learning/ (azkenekoz ikusia 2026-03-24). Ikusi 9 orrialdea.

[27] Codecademy. *Learn to Code - for Free*. 2025eko martxoaren 22a. Eskuragarri hemen: https://www.codecademy.com/ (azkenekoz ikusia 2026-03-24). Ikusi 9 orrialdea.

[28] Datacamp. *Learn Data Science and AI Online*. 2026. Eskuragarri hemen: https://www.datacamp.com/ (azkenekoz ikusia 2026-03-24). Ikusi 10 orrialdea.

[29] Similarweb. *leetcode.com Traffic Analytics, Ranking & Audience [February 2026]*. 2026. Eskuragarri hemen: https://www.similarweb.com/website/leetcode.com (azkenekoz ikusia 2026-03-27). Ikusi 10 orrialdea.

[30] LeetCode. *The World's Leading Online Programming Learning Platform*. 2026. Eskuragarri hemen: https://leetcode.com/ (azkenekoz ikusia 2026-03-24). Ikusi 10 orrialdea.

[31] The PostgreSQL Global Development Group. *PostgreSQL: The world's most advanced open source database*. 2019. Eskuragarri hemen: https://www.postgresql.org/ (azkenekoz ikusia 2026-03-10). Ikusi 59, 110 orrialdeak.

[32] Prisma. *Next-generation ORM for Node.js & TypeScript*. 2026. Eskuragarri hemen: https://www.prisma.io/ (azkenekoz ikusia 2026-06-05). Ikusi 59, 111 orrialdeak.

[33] OpenJS Foundation. *Express – Node.js web application framework*. 2026. Eskuragarri hemen: https://expressjs.com/ (azkenekoz ikusia 2026-03-10). Ikusi 60, 111 orrialdeak.

[34] Node.js Foundation. *Node.js*. 2019. Eskuragarri hemen: https://nodejs.org/ (azkenekoz ikusia 2026-03-10). Ikusi 60, 110 orrialdeak.

[35] Microsoft. *JavaScript With Syntax For Types*. 2026. Eskuragarri hemen: https://www.typescriptlang.org/ (azkenekoz ikusia 2026-06-06). Ikusi 60, 110 orrialdeak.

[36] Colin McDonnell. *TypeScript-first schema validation with static type inference*. 2026. Eskuragarri hemen: https://zod.dev/ (azkenekoz ikusia 2026-06-06). Ikusi 61, 112 orrialdeak.

[37] OpenAI. *Official JavaScript / TypeScript library for the OpenAI API*. 2026. Eskuragarri hemen: https://github.com/openai/openai-node (azkenekoz ikusia 2026-06-07). Ikusi 61 orrialdea.

[38] Express. *CORS*. 2026. Eskuragarri hemen: https://github.com/expressjs/cors (azkenekoz ikusia 2026-06-07). Ikusi 62, 110 orrialdeak.

[39] Adam Baldwin. *Helmet.js*. 2026. Eskuragarri hemen: https://helmet.js.org/ (azkenekoz ikusia 2026-06-07). Ikusi 62, 111 orrialdeak.

[40] Nathan Friedly. *express-rate-limit: Basic rate-limiting middleware for Express*. 2026. Eskuragarri hemen: https://github.com/express-rate-limit/express-rate-limit (azkenekoz ikusia 2026-06-07). Ikusi 62, 111 orrialdeak.

[41] Michael B. Jones, John Bradley eta Nat Sakimura. *JSON Web Token (JWT)*. RFC 7519. 2015eko maiatza. DOI: 10.17487/RFC7519. Eskuragarri hemen: https://www.rfc-editor.org/info/rfc7519. Ikusi 62 orrialdea.

[42] Docker. *Enterprise Application Container Platform*. 2024. Eskuragarri hemen: https://www.docker.com/ (azkenekoz ikusia 2026-03-10). Ikusi 63, 109 orrialdeak.

[43] Meta Open Source. *React*. 2025. Eskuragarri hemen: https://react.dev/ (azkenekoz ikusia 2026-03-10). Ikusi 66, 111 orrialdeak.

[44] Tailwind Labs. *Rapidly build modern websites without ever leaving your HTML*. 2026. Eskuragarri hemen: https://tailwindcss.com/ (azkenekoz ikusia 2026-06-08). Ikusi 66, 112 orrialdeak.

[45] Axios Collective. *Axios is a simple HTTP client for the browser and Node.js*. 2026. Eskuragarri hemen: https://axios.rest/ (azkenekoz ikusia 2026-06-09). Ikusi 68, 110 orrialdeak.

[46] Microsoft. *Monaco Editor*. 2026. Eskuragarri hemen: https://microsoft.github.io/monaco-editor/ (azkenekoz ikusia 2026-03-10). Ikusi 68, 111 orrialdeak.

[47] Daniel Fett et. al. *OAuth 2.0 Demonstrating Proof of Possession (DPoP)*. RFC 9449. 2023ko iraila. DOI: 10.17487/RFC9449. Eskuragarri hemen: https://www.rfc-editor.org/info/rfc9449. Ikusi 79 orrialdea.

[48] Jefatura del Estado. «Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales». Non: *Boletín Oficial del Estado* 294 (2018ko abenduaren 6a). BOE-A-2018-16673, orrk. 119788-119857. Eskuragarri hemen: https://www.boe.es/eli/es/lo/2018/12/05/3/con (azkenekoz ikusia 2026-06-12). Ikusi 79 orrialdea.

[49] European Parliament and the Council of the European Union. «Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)». Non: *Official Journal of the European Union* L 119 (2016ko apirilaren 27a), orrk. 1-88. ISSN: 1977-0677. Eskuragarri hemen: https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng (azkenekoz ikusia 2026-06-12). Ikusi 79 orrialdea.

[50] ISO/IEC JTC1/SC22/WG14. *C programming language standard (C23)*. 2025eko azaroaren 10a. Eskuragarri hemen: https://www.open-std.org/jtc1/sc22/wg14/ (azkenekoz ikusia 2026-06-21). Ikusi 109 orrialdea.

[51] World Wide Web Consortium. *CSS Snapshot 2026*. Standard. 2026ko martxoaren 26a. Eskuragarri hemen: https://www.w3.org/TR/css/ (azkenekoz ikusia 2026-06-21). Ikusi 109 orrialdea.

[52] Free Software Foundation. *GCC, the GNU Compiler Collection*. 2026. Eskuragarri hemen: https://gcc.gnu.org/ (azkenekoz ikusia 2026-06-21). Ikusi 109 orrialdea.

[53] WHATWG. *HTML Living Standard*. Standard. 2026ko ekainaren 17a. Eskuragarri hemen: https://html.spec.whatwg.org/multipage/ (azkenekoz ikusia 2026-06-21). Ikusi 109 orrialdea.

[54] Oracle. *Java*. 2026ko martxoaren 17a. Eskuragarri hemen: https://www.oracle.com/java/ (azkenekoz ikusia 2026-06-21). Ikusi 109 orrialdea.

[55] Ecma International. *ECMAScript® 2027 Language Specification*. 2026ko ekainaren 15a. Eskuragarri hemen: https://tc39.es/ecma262 (azkenekoz ikusia 2026-06-20). Ikusi 109 orrialdea.

[56] Oracle. *Java Development Kit*. 2026. Eskuragarri hemen: https://www.oracle.com/java/technologies/ (azkenekoz ikusia 2026-06-21). Ikusi 109 orrialdea.

[57] Kent Beck et. al. *JUnit: The programmer-friendly testing framework for Java and the JVM*. 2026. Eskuragarri hemen: https://junit.org/ (azkenekoz ikusia 2026-06-21). Ikusi 109 orrialdea.

[58] The LaTeX Project. *LaTeX – A document preparation system*. 2026. Eskuragarri hemen: https://www.latex-project.org/ (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[59] Holger Krekel eta The pytest development team. *pytest: helps you write better programs*. 2026. Eskuragarri hemen: https://docs.pytest.org/en/stable/ (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[60] Python Software Foundation. *Python is a programming language that lets you work quickly and integrate systems more effectively*. 2026. Eskuragarri hemen: https://www.python.org/ (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[61] ISO/IEC 9075-1:2023. *Information technology – Database languages SQL*. Standard. Geneva, CH: International Organization for Standardization, 2023. Eskuragarri hemen: https://www.iso.org/standard/76583.html (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[62] npm, Inc. *npm: Build amazing things*. 2026. Eskuragarri hemen: https://www.npmjs.com/ (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[63] Daniel Wirtz. *bcryptjs: Optimized bcrypt in JavaScript with zero dependencies, with TypeScript support*. 2026. Eskuragarri hemen: https://github.com/dcodeIO/bcrypt.js (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[64] Kimmo Brunfeldt. *concurrently: Run commands concurrently. Like 'npm run watch-js & npm run watch-less' but better*. 2026. Eskuragarri hemen: https://github.com/open-cli-tools/concurrently (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[65] TJ Holowaychuk eta Douglas Christopher Wilson. *cookie-parser: Parse HTTP request cookies*. 2026. Eskuragarri hemen: https://github.com/expressjs/cookie-parser (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[66] Kent C. Dodds. *cross-env: Run scripts that set and use environment variables across platforms*. 2026. Eskuragarri hemen: https://github.com/kentcdodds/cross-env (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[67] Pedro Dias. *dockerode: Node.js module for Docker's Remote API*. 2026. Eskuragarri hemen: https://github.com/apocas/dockerode (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[68] Scott Motte. *dotenv: Loads environment variables from .env for nodejs projects*. 2026. Eskuragarri hemen: https://github.com/motdotla/dotenv (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[69] Nicholas C. Zakas. *ESlint: Find and fix problems in your JavaScript code*. 2026. Eskuragarri hemen: https://eslint.org/ (azkenekoz ikusia 2026-06-21). Ikusi 110 orrialdea.

[70] Auth0. *jsonwebtoken: JsonWebToken implementation for node.js*. 2026. Eskuragarri hemen: https://github.com/auth0/node-jsonwebtoken (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[71] Andrey Okonetchnikov eta Iiro Jäppinen. *lint-staged: Run tasks like formatters and linters against staged git files*. 2026. Eskuragarri hemen: https://github.com/lint-staged/lint-staged (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[72] Anthropic. *modelcontextprotocol/sdk: The official TypeScript SDK for Model Context Protocol servers and clients*. 2026. Eskuragarri hemen: https://github.com/modelcontextprotocol/typescript-sdk (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[73] Lucas Merencia. *node-cron: A tiny, zero-dependency cron task scheduler for Node.js, written in TypeScript*. 2026. Eskuragarri hemen: https://github.com/node-cron/node-cron (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[74] Brian M. Carlson. *node-postgres: PostgreSQL client for node.js*. 2026. Eskuragarri hemen: https://github.com/brianc/node-postgres (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[75] James Long. *Prettier is an opinionated code formatter*. 2026. Eskuragarri hemen: https://prettier.io/ (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[76] Meta Open Source. *react-dom: package serving as the entry point to the DOM and server renderers for React*. 2026. Eskuragarri hemen: https://www.npmjs.com/package/react-dom (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[77] Espen Hovlandsdal. *react-markdown: Markdown component for React*. 2026. Eskuragarri hemen: https://github.com/remarkjs/react-markdown (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[78] Remix Software. *react-router-dom: Declarative routing for React*. 2026. Eskuragarri hemen: https://www.npmjs.com/package/react-router-dom (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[79] Titus Wormer. *remark-gfm: remark plugin to support GFM (autolink literals, footnotes, strikethrough, tables, tasklists)*. 2026. Eskuragarri hemen: https://github.com/remarkjs/remark-gfm (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[80] Mikhail Gorbunov. *simple-git-hooks: A simple git hooks manager for small projects*. 2026. Eskuragarri hemen: https://github.com/toplenboren/simple-git-hooks (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[81] Emil Kowalski. *Sonner: An opinionated toast component for React*. 2026. Eskuragarri hemen: https://github.com/emilkowalski/sonner (azkenekoz ikusia 2026-06-21). Ikusi 111 orrialdea.

[82] Mathias Buus. *tar-fs: fs bindings for tar-stream*. 2026. Eskuragarri hemen: https://github.com/mafintosh/tar-fs (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[83] EGOIST. *tsup: The simplest and fastest way to bundle your TypeScript libraries*. 2026. Eskuragarri hemen: https://github.com/egoist/tsup (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[84] Hiroki Osame. *TypeScript Execute | The easiest way to run TypeScript in Node.js*. 2026. Eskuragarri hemen: https://github.com/privatenumber/tsx (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[85] Evan You. *Vite: Next generation frontend tooling. It's fast!* 2026. Eskuragarri hemen: https://github.com/vitejs/vite (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[86] Charlie Robbins. *Winston: A logger for just about everything*. 2026. Eskuragarri hemen: https://github.com/winstonjs/winston (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[87] Charlie Robbins eta Matt Berther. *winston-daily-rotate-file: A transport for winston which logs to a rotating file each day*. 2026. Eskuragarri hemen: https://github.com/winstonjs/winston-daily-rotate-file (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[88] JGraph Ltd. *draw.io: Security-first diagramming for teams*. 2026. Eskuragarri hemen: https://app.diagrams.net/ (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[89] Neovim. *Neovim: hyperextensible Vim-based text editor*. 2026. Eskuragarri hemen: https://neovim.io/ (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[90] Online Gantt. *Online Gantt*. 2026. Eskuragarri hemen: https://www.onlinegantt.com/ (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[91] Overleaf Organization. *Overleaf: Online LaTeX Editor*. 2026. Eskuragarri hemen: https://www.overleaf.com/ (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[92] Microsoft. *VSCode: The open source AI code editor*. 2026. Eskuragarri hemen: https://code.visualstudio.com/ (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[93] Git Development Community. *Git*. 2024. Eskuragarri hemen: https://git-scm.com/ (azkenekoz ikusia 2026-03-10). Ikusi 112 orrialdea.

[94] GitHub, Inc. *GitHub: The future of building happens together*. 2026. Eskuragarri hemen: https://github.com/ (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[95] Microsoft. *Microsoft Teams: Video Conferencing, Meetings, Calling*. 2026. Eskuragarri hemen: https://www.microsoft.com/microsoft-teams/group-chat-software/ (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[96] Microsoft. *Microsoft OneDrive: Personal File Sharing & Cloud Storage*. 2026. Eskuragarri hemen: https://www.microsoft.com/microsoft-365/onedrive/online-cloud-storage (azkenekoz ikusia 2026-06-21). Ikusi 112 orrialdea.

[97] MZLA Technologies Corporation. *Thunderbird: Free Your Inbox*. 2026. Eskuragarri hemen: https://www.thunderbird.net/ (azkenekoz ikusia 2026-06-21). Ikusi 113 orrialdea.